

药材热风烘干过程的热质耦合建模与数值模拟

摘要

热风烘干是中药材加工的重要环节,烘房环境通过药材表面向内部传热,同时推动内部水分向外迁移。为描述这一过程,本文根据药材细长圆柱的形状特征,忽略轴向传递和端面影响,建立**一维轴对称径向传热传质模型**。烘房温度与水分浓度由附件数据插值得到,药材表面采用换热、传质边界。四个问题依次考虑**常物性预热**、**变物性热湿耦合**、**固定半径下的干燥达标**以及**半径收缩后的干燥达标**,空间上统一采用**圆柱径向有限体积法**,时间上采用**自适应 BDF 方法**,从而在同一框架内逐步加入物性变化、终止判据和移动边界。

针对问题一,首先将附件 1 中的离散环境数据处理为随时间连续变化的边界条件,再根据附录 2 的常物性关系分别建立 **Fourier 径向导热方程**和 **Fick 径向扩散方程**。由于水分扩散系数不含温度项,温度场与水分场可以分开求解。利用有限体积法处理圆柱中心和表面边界后,得到预热阶段各时刻、各半径位置的温度和水分浓度,并将完整结果保存至 result1.xlsx。在 1800 s 时,中心与表面温度分别为**33.5753°C**、**36.7856°C**,中心与表面水分浓度分别为**2.5500 kg/kg**、**1.5102 kg/kg**。温度变化已传入内部,而水分下降主要集中在近表面区域,说明预热阶段的传热速度快于内部水分扩散。

针对问题二,附录 3 中的密度、比热容和热导率随水分变化,水分扩散系数又同时受温度与水分影响。为反映这种相互作用,本文将**温度场和水分场合并为联合状态**,建立**变物性的热湿耦合模型**,并在每个时间步同步更新物性和界面通量。由此得到整个烘干过程前三小时的温度、水分分布,完整结果保存至 result2.xlsx。在 3 h 时,中心与表面温度分别为**49.8495°C**、**49.9664°C**,中心与表面水分浓度分别为**1.7662 kg/kg**、**1.0081 kg/kg**。此时温度已接近均匀,水分仍保持明显的径向梯度,表明恒温干燥后期主要受内部水分迁移限制。

针对问题三,仍采用**固定半径热湿耦合模型**,并从烘干初始状态重新计算。题目要求药材各处水分浓度均低于规定阈值,因此本文以全截面最大水分浓度构造终止事件,通过连续事件搜索确定首次达标时间,而不以表面值或平均值代替。附件 1 结束后的烘房条件采用末值延续,并用不同边界情景检验这一假设的影响;同时采用另一种隐式方法进行交叉验证。计算得到连续达标时间为 $t^* = 205821.7420 \text{ s} = 57.1727 \text{ h}$,第一个满足要求的 60 s 输出时刻为**57.1833 h**,完整水分结果保存至 result3.xlsx。终止时全域最大水分位于中心,说明药材中心的干燥滞后决定了最终停机时刻;边界扰动下的达标时间范围为**53.6154–61.0702 h**。

针对问题四,在问题三的达标判据上进一步引入附件 2 给出的半径变化,并采用附录 4 的物性关系。本文利用 **Landau 变换**将随时间收缩的物理区域映射到固定计算区域,以无量纲物质坐标标记药材内部位置,再根据当前半径更新传热传质尺度,实现收缩条件下温度场和水分场的同步求解。计算得到连续达标时间为 $t^* = 182968.2250 \text{ s} = 50.8245 \text{ h}$,第一个满足要求的 60 s 输出时刻为**50.8333 h**,完整水分结果保存至 result4.xlsx。与问题三相比,达标时间缩短约**6.35 h**;边界扰动下的终止时间范围为**47.6834–54.2753 h**。结果表明,在题给半径轨迹和物性关系下,尺寸收缩缩短了水分迁移路径,使药材更早达到干燥要求。

关键词: 药材热风干燥 热质耦合 有限体积法 向后微分公式 Landau 变换

一、 问题重述

1.1 问题背景

热风烘干是中药材加工中的常见工序。烘干不足时，药材内部残余水分偏高，不利于后续贮藏；烘干过度时，又会增加能耗，并可能影响药材品质。因此，如何根据药材内部温度和水分分布判断烘干进程，是本题需要解决的核心问题。

从物理过程看，热风首先作用于药材表面，表面温度和含水率变化较快；中心区域只能依靠内部导热和水分扩散逐步响应，通常存在明显滞后。若后期药材发生收缩，半径减小又会改变扩散距离和空间边界。由此可知，仅根据表面含水率或平均含水率判断停机时间并不充分，必须建立能够描述径向分布的传热传质模型。

题目将药材近似为长 25 cm、初始半径 2 cm 的圆柱体，给定初始温度、初始干基含水率、烘房温湿度数据以及不同阶段的物性经验式；附件 2 还给出了半径随时间变化的数据。本文据此把药材内部过程简化为一维径向问题，按常物性、变物性、固定半径达标和收缩半径达标四个层次建立模型，逐步分析物性变化、边界外推和半径收缩对计算结果的影响。

1.2 问题提出

四个问题的建模要求具有递进关系，可整理为以下任务。

问题一：在 0 到 1800 s 的预热阶段内，基于附录 2 物性关系建立温度场和水分场模型，给出表 2、表 3 所需结果，并按 1 s 时间分辨率和 0.1 cm 径向分辨率输出完整数据。

问题二：在固定半径条件下，将物性关系改为附录 3 经验式，建立温度与水分相互影响的热湿耦合模型，给出前 3 h 内每隔 0.5 h、半径 0, 0.5, 1, 1.5, 2 cm 处的温度和水分浓度。

问题三：沿用问题二的固定半径耦合模型，继续计算至药材各处水分浓度均低于 0.15 kg/kg，确定达标烘干时间，并输出每隔 6 h 以及终止时刻的水分浓度分布。

问题四：在问题三的达标判据基础上引入附件 2 给出的半径收缩轨迹 $R(t)$ ，同时将物性关系改为附录 4，求移动边界条件下药材全截面水分浓度达标所需时间，并输出每隔 6 h 以及终止时刻、含移动表面位置的数据。

二、 问题分析

四个问题均围绕圆柱药材内部的径向传热、传质展开，区别主要体现在物性关系、计算时长和几何边界三个方面。问题一是常物性预热阶段，用于确定空间离散、表面边界和输出插值方法；问题二引入状态相关物性，需要同步推进温度场和水分场；问题三在固定半径下延长计算时间，重点转为达标判据和边界外推；问题四进一步考虑半径收缩，将固定区域问题变为移动边界问题。因此，本文不把四问割裂处理，而是在同一径向守恒框架上逐层加入新的物理因素。

2.1 问题一分析

问题一只计算 0 到 1800 s 的预热过程，该时间段完全处在附件 1 给出的烘房边界范围内。附录 2 中密度、比热容和热导率均为常数，水分扩散系数只与水分浓度有关，不含温度项，因此温度方程和水分方程在本问中可以分别求解。该问的关键不是寻找停机时刻，而是建立可靠的径向控制体格式、第三类边界处理和结果输出方法，并观察表面快速响应与中心滞后之间的差异。

2.2 问题二分析

问题二仍采用固定半径和相同类型的表面边界，但物性关系由附录 2 改为附录 3。此时 ρ, c_p, k 随水分浓度变化， D 同时依赖水分浓度和绝对温度。也就是说，温度会影响水分扩散速率，水分变化又会通过物性参数影响能量方程。若先求温度再代入水分方程，容易在非线性物性上产生时间滞后。因此，本问应把温度和水分作为联合状态同步积分，并沿用问题一建立的径向守恒离散结构。

2.3 问题三分析

问题三把问题二的固定半径耦合模型从前三小时延长到烘干终点。题目要求药材“各处”水分浓度均低于 0.15 kg/kg ，因此终止判据必须作用于全截面的最大水分浓度。若只检查表面值或平均值，将会提前停机，无法反映中心区域的干燥滞后。该问还涉及边界外推：附件 1 只覆盖前 4h，而达标时间预计远超过 4h，因此需要给出后续烘房工况假设，并在结果中区分数值误差与边界假设造成的不确定性。

2.4 问题四分析

问题四在问题二、问题三的基础上同时改变物性和几何边界。附录 4 给出新的物性经验式，附件 2 给出随时间减小的半径 $R(t)$ ，计算区域由固定区间 $[0, R]$ 变为移动区间 $[0, R(t)]$ 。若直接在物理半径上计算，每个时间步都需要处理移动网格，数值实现较繁琐。为此，可引入 $\xi = r/R(t)$ ，把移动区域映射到固定区间 $[0, 1]$ 。在均匀径向收缩假设下， ξ 可视为同一物质层的位置标签，半径变化主要通过扩散尺度项影响水分迁移。题给半径轨迹与附录 4 密度经验式是否完全满足干物质守恒，将在模型检验部分进一步讨论。

综合上述分析，四个问题并非相互独立，而是在统一径向传热传质框架下逐步增加物性非线性、终点判据与移动边界效应。本文按照“预热平衡—热质耦合—终点判定—移动边界”的递进思路展开建模，并在各阶段采用与模型复杂度相适应的数值方法。整体建模与求解技术路线如图 1 所示。

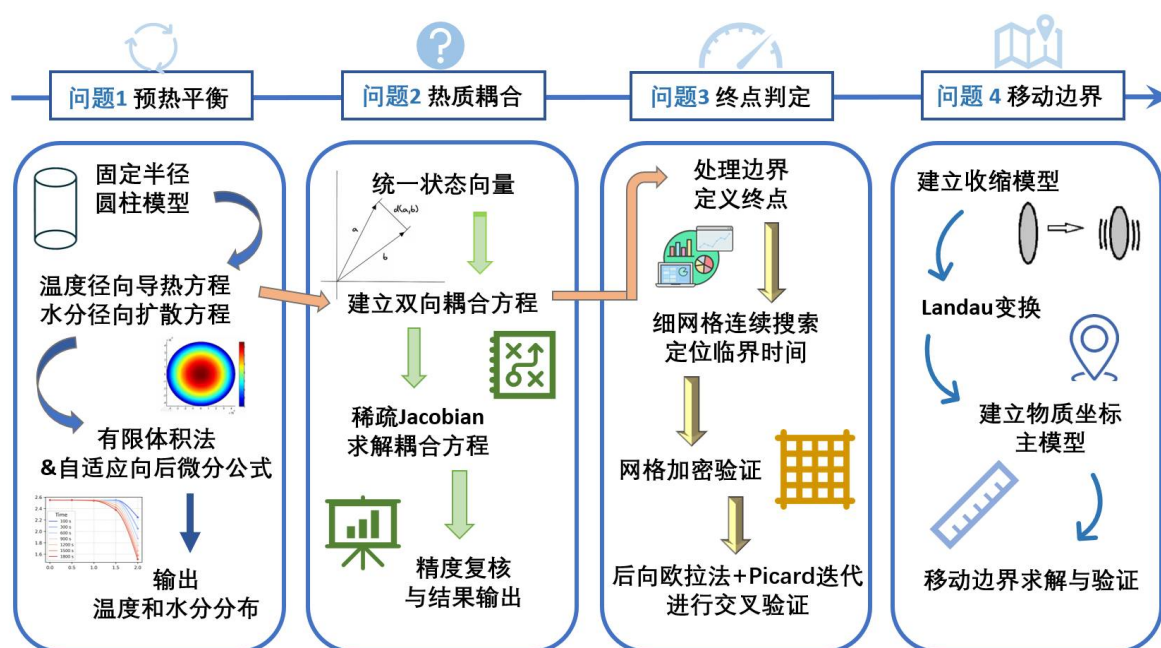


图 1 四个问题的递进式建模与求解技术路线

如图 1 所示，问题一首先在固定半径和常物性条件下建立径向传热、传质模型，为

后续计算提供基础；问题二进一步引入状态相关物性，将温度场与水分场统一为耦合状态进行求解；问题三在热湿耦合模型基础上引入全截面含水率终止判据，通过连续搜索确定临界干燥时间；问题四进一步考虑药材半径随时间收缩的移动边界，通过坐标变换将变区域问题转化为固定计算域上的耦合问题。由此形成由基础物理场模拟向实际停机判定逐级推进的完整建模体系。

三、 模型假设

1. 药材近似为轴对称长圆柱体。由于长度明显大于半径，忽略轴向传递和端面效应，只考虑径向一维传热、传质过程。
2. 初始时刻药材内部状态均匀， $T(r, 0)$ 与 $C(r, 0)$ 均不随半径变化；烘房环境视为空间均匀混合。
3. 题目未给出显式蒸发潜热源项所需参数，本文在题设经验物性体系内建模，不额外加入相变潜热源项。
4. 问题四采用前置固定变换处理移动边界，认为半径收缩为均匀径向缩放。

四、 符号说明

表 1 主要符号说明

符号	含义	单位
r	到药材中心轴的径向距离	m
$R, R(t)$	药材半径；前三问取常数 0.02 m，问题四为随时间变化的半径	m
ξ	问题四采用的物质坐标， $\xi = r/R(t) \in [0, 1]$	无量纲
t	烘干时间	s
$T(r, t)$	药材内部温度	°C
T_K	绝对温度， $T_K = T + 273.15$	K
$C(r, t)$	干基水分浓度	kg/kg
$T_\infty(t)$	烘房环境温度	°C
$C_\infty(t)$	烘房环境水分浓度	kg/kg
$\rho(C)$	密度	kg/m ³
$c_p(C)$	比热容	J/(kg·K)
$k(C)$	热导率	W/(m·K)
$D(C, T_K)$	有效水分扩散系数	m ² /s
h	对流换热系数	W/(m ² ·K)
h_m	对流传质系数	m/s
N	径向离散节点数	无量纲
Δr	固定半径模型中的径向网格尺度	m
Δt	输出或时间推进步长	s
$g(t)$	终止事件函数， $g(t) = \max C - 0.15$	kg/kg
t^*	达到水分阈值所需烘干时间，即连续事件定位的临界时刻	h

五、 模型准备

四个问题在物性关系、计算时长和半径条件上逐步变化，但数据调用和空间离散可以统一处理。本节先说明附件中离散时间序列的插值与外推方式，再给出采用一维径向模型的依据，最后建立后续各问共同使用的有限体积离散格式。这样处理可以保证四问之间只改变必要的物理条件，而不因离散方法变化引入额外误差。

5.1 边界数据插值

附件 1 只给出了若干离散时刻的烘房温度和环境水分浓度，而隐式时间积分器会在自适应时间步上反复调用边界值。因此，需先将附件数据扩展为连续时间函数。本文对烘房温度 $T_\infty(t)$ 和环境水分浓度 $C_\infty(t)$ 均采用分段线性插值：

$$y_\infty(t) = y_j + \frac{y_{j+1} - y_j}{t_{j+1} - t_j}(t - t_j), \quad t_j \leq t \leq t_{j+1}, \quad (1)$$

其中 y 可表示温度或水分浓度。问题一和问题二的报告时段均在附件 1 覆盖范围内，边界值直接由上述插值式给出。问题三、问题四需要继续计算到水分达标，计算时间可能超过 4h，超出附件 1 范围后采用末值保持：

$$T_\infty(t) = T_\infty(4h), \quad C_\infty(t) = C_\infty(4h), \quad t > 4h. \quad (2)$$

这一处理等价于假定 4h 后烘房进入稳定运行阶段。由于该假设会直接影响后期干燥速率，本文在模型检验中另设边界扰动情景，用于衡量外推方式对达标时间的影响。

5.2 半径收缩数据插值

问题四需要考虑药材半径随时间收缩，因此必须在积分过程中实时更新 $R(t)$ 。附件 2 给出了离散时刻的半径数据，本文仍采用分段线性插值：

$$R(t) = R_j + \frac{R_{j+1} - R_j}{t_{j+1} - t_j}(t - t_j), \quad t_j \leq t \leq t_{j+1}, \quad (3)$$

附件 2 覆盖 0 到 72h，采样间隔为 1800s，半径由 2.0cm 单调减小到 1.198cm。若计算时间超过 72h，半径取附件末值；本文问题四得到的终止时间小于 72h，实际计算中没有触发该外推。需要说明的是， $R(t)$ 在本文中作为题目给定的经验轨迹使用，不再由质量守恒方程反求。

5.3 几何简化依据

将圆柱药材简化为一维径向模型，需要同时满足两个判断：一是内部温度和水分梯度不能忽略，二是端面效应相对较弱。前者可用热 Biot 数和质 Biot 数估计：

$$Bi_T = \frac{hR}{k}, \quad Bi_C = \frac{h_m R}{D(C_0)}. \quad (4)$$

按问题一参数估计， $Bi_T \approx 1.389$ 、 $Bi_C \approx 3.24$ ，均大于 0.1。这说明药材内部存在明显梯度，不能用集总参数模型代替空间分布模型。另一方面，1800s 内热扩散尺度和水分扩散尺度分别为

$$l_T = \sqrt{\alpha t}, \quad l_C = \sqrt{D(C_0)t}, \quad \alpha = \frac{k}{\rho c_p}. \quad (5)$$

代入参数可得 $l_T \approx 1.74\text{cm}$ 、 $l_C \approx 0.30\text{cm}$ ，均小于圆柱半长 12.5cm。因此，在题目给定的主要计算尺度内，端面影响不会主导整体干燥过程，忽略轴向传递是合理的。

5.4 径向有限体积离散

柱坐标方程中含有 $1/r$ 项，若直接差分，中心点和表面通量处理容易不一致。为保持守恒结构，本文采用有限体积法。取单位长度圆柱截面，节点 $r_i = i\Delta r$ ，控制体体积和界面面积分别为

$$V_i = \pi (r_{i+1/2}^2 - r_{i-1/2}^2), \quad A_{i+1/2} = 2\pi r_{i+1/2}. \quad (6)$$

对具有径向扩散形式的一般方程

$$a(u) \frac{\partial u}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r b(u) \frac{\partial u}{\partial r} \right), \quad (7)$$

在每个控制体上积分并用相邻节点差商表示界面通量，可得内部节点的半离散形式：

$$a_i V_i \frac{du_i}{dt} = b_{i-1/2} A_{i-1/2} \frac{u_{i-1} - u_i}{\Delta r} + b_{i+1/2} A_{i+1/2} \frac{u_{i+1} - u_i}{\Delta r}. \quad (8)$$

其中 $a(u)$ 对应热容项或水分储存项， $b(u)$ 对应热导率或水分扩散系数。中心处因界面面积为零，自然满足对称零通量条件；表面控制体则在扩散通量之外加入对流换热或传质通量，从而实现第三类边界。后续各问只需替换物性函数、边界函数以及是否引入 $R(t)$ ，空间离散框架保持不变。

六、 问题一模型的建立与求解

6.1 本问任务与前文承接

问题一研究固定半径、常物性条件下的预热过程，计算时间为 0 到 1800 s。该问采用附录 2 物性关系，温度方程和水分方程尚未形成双向耦合，因此可作为全文数值框架的基准算例。本文先通过本问确定径向有限体积离散、第三类边界和指定位置输出方法，后续问题再在同一框架上加入变物性、达标事件函数和移动边界。

6.2 模型建立

在固定半径 $R = 0.02 \text{ m}$ 内，药材温度场满足径向 Fourier 热传导方程：

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(kr \frac{\partial T}{\partial r} \right), \quad 0 < r < R. \quad (9)$$

水分场满足径向 Fick 扩散方程：

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left[r D(C) \frac{\partial C}{\partial r} \right], \quad D(C) = 7 \times 10^{-9} \exp \left(-\frac{0.89}{C} \right). \quad (10)$$

初始条件为

$$T(r, 0) = 28, \quad C(r, 0) = 2.55. \quad (11)$$

中心对称边界为

$$\left. \frac{\partial T}{\partial r} \right|_{r=0} = 0, \quad \left. \frac{\partial C}{\partial r} \right|_{r=0} = 0. \quad (12)$$

表面 Robin 边界为

$$-k \left. \frac{\partial T}{\partial r} \right|_{r=R} = h [T(R, t) - T_\infty(t)], \quad (13)$$

$$-D(C_s) \left. \frac{\partial C}{\partial r} \right|_{r=R} = h_m [C(R, t) - C_\infty(t)]. \quad (14)$$

上述两个表面条件分别描述药材表面与热风之间的换热和换质。由于本问中 ρ, c_p, k 为常数，且 $D(C)$ 不含温度项，温度场与水分场之间没有直接耦合，可以分别积分求解。

6.3 模型求解

空间方向采用前文建立的径向有限体积格式，时间方向采用自适应 BDF 隐式积分。计算以几何半径 R 、初值 T_0, C_0 、附件 1 边界序列和附录 2 物性参数为已知条件，先对烘房温度和环境水分浓度建立线性插值函数，再分别组装温度方程和水分方程的右端项，积分至 1800 s 后抽取题目指定时间和半径位置的结果。

内部计算网格取 $\Delta r = 0.0025$ cm，使表面附近的温度和水分梯度能够被充分分辨；得到连续半径网格上的数值解后，再抽取到题目要求的 0.1 cm 输出位置。水分扩散系数 $D(C)$ 随含水率变化较快，界面扩散系数采用调和平均，以避免在低扩散区域高估界面通量。

输出前设置 $C \leftarrow \max(C, 0)$ 作为非负保护。

6.4 结果分析

表 2 和表 3 给出了问题一的代表性输出。温度场在半小时内整体升高，并由表面向中心逐步传递。1800 s 时，表面温度为 36.7856°C，中心温度为 33.5753°C，说明热量已进入药材内部，但径向温差仍然存在。水分场的响应慢于温度场：同一时刻表面含水率降至 1.5102 kg/kg，中心仍保持 2.5500 kg/kg，脱水主要集中在近表面区域。

表 2 30 分钟内药材的温度 /°C

时间/s	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
100	28.0001	28.0003	28.0040	28.0327	28.1801
300	28.0408	28.0635	28.1514	28.3681	28.8489
600	28.4533	28.5360	28.8039	29.3159	30.1652
900	29.3243	29.4583	29.8755	30.6161	31.7304
1200	30.5427	30.7098	31.2223	32.1126	33.4276
1500	31.9957	32.1867	32.7660	33.7463	35.1203
1800	33.5753	33.7720	34.3642	35.3621	36.7856

表 3 30 分钟内药材的水分浓度 /kg · kg⁻¹

时间/s	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
100	2.5500	2.5500	2.5500	2.5500	2.2471
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

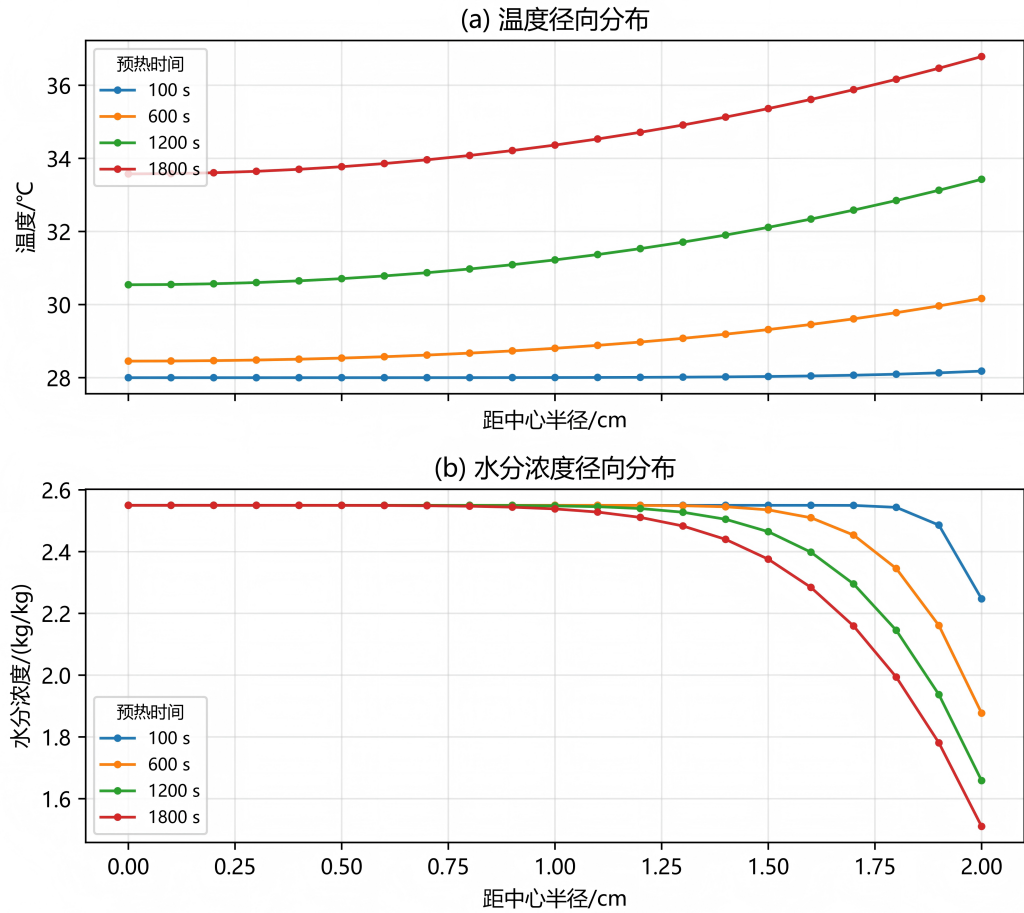


图 2 问题一预热阶段药材温度与水分浓度的径向演化。(a) 100 s、600 s、1200 s 与 1800 s 的温度径向分布；(b) 对应时刻的水分浓度径向分布。

由图 2 可见，各时刻温度均由中心向表面升高，且温度曲线随时间整体上移；水分浓度的明显下降则主要集中在近表面区域，中心在 1800 s 内基本保持初值。两类剖面的差异说明，预热阶段的热量传递快于内部水分扩散，这也为后续将干燥过程的控制环节归结为内部传质提供了依据。

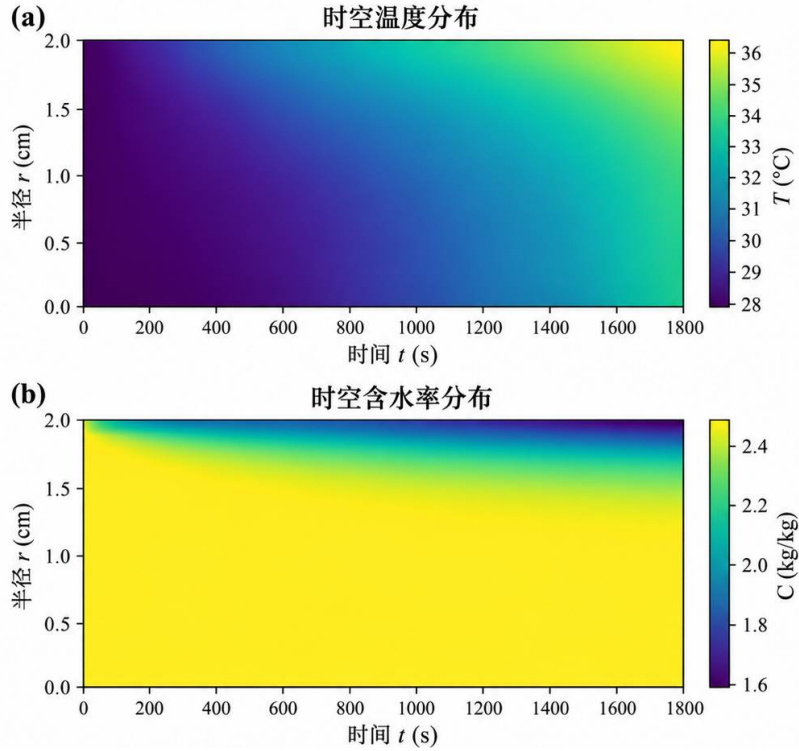


图3 预热平衡阶段（0–1800 s）药材内部温度场与含水率场沿径向位置的时空演化特征。子图 (a) 展示温度分布，子图 (b) 展示含水率分布。

图3用二维时空热图展示了预热阶段药材内部热湿状态的演化。可以看出，表层区域首先升温，随后热量向中心传递，形成由表面到中心递减的温度梯度；含水率则先在外层下降，并逐渐向内部推进。上述现象与第三类边界条件下的物理过程一致：外部热风先作用于表面，内部传递速度受有限热扩散和水分扩散限制。该结果说明本文的一维轴对称模型能够刻画预热阶段的主要热湿迁移特征。

6.5 本问小结与后续衔接

问题一表明，即使在常物性预热阶段，药材内部也已形成不可忽略的径向梯度，后续模型不能采用整体均匀近似。由本问确定的几何处理、第三类边界和守恒型径向离散将直接用于问题二；区别在于问题二的物性随状态变化，温度场和水分场需要作为联合系统同步求解。

七、 问题二模型的建立与求解

7.1 本问任务与前文承接

问题二仍在固定半径圆柱区域内计算，但物性关系由附录2常物性改为附录3经验式，并要求给出0.5–3.0 h内若干位置的温度 and 水分浓度。与问题一相比，本问的关键变化是温度场和水分场不再相互独立：水分浓度会改变密度、比热容和热导率，温度又通过绝对温度项影响水分扩散系数。因此，本问沿用问题一建立的径向有限体积空间离散，但在时间推进时将温度和水分合并为联合状态，同步求解热湿耦合系统。

7.2 模型建立

附录 3 给出的状态相关物性如下：

$$\rho(C) = 650 + 128C, \quad (15)$$

$$c_p(C) = 1450 + \frac{2736C}{C+1}, \quad (16)$$

$$k(C) = 0.21 + \frac{0.38C}{C+1}, \quad (17)$$

$$D(C, T) = 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}\right) \exp\left(-\frac{3850}{T_K}\right), \quad T_K = T + 273.15. \quad (18)$$

在该物性体系下，能量方程写为

$$\rho(C)c_p(C)\frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left[rk(C)\frac{\partial T}{\partial r}\right]. \quad (19)$$

水分迁移仍采用守恒型径向扩散方程：

$$\frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left[rD(C, T)\frac{\partial C}{\partial r}\right]. \quad (20)$$

初始条件和中心对称边界沿用问题一。表面边界中的物性取表面状态值，写为

$$-k(C_s)\frac{\partial T}{\partial r}\bigg|_{r=R} = h[T(R, t) - T_\infty(t)], \quad (21)$$

$$-D(C_s, T_s)\frac{\partial C}{\partial r}\bigg|_{r=R} = h_m[C(R, t) - C_\infty(t)]. \quad (22)$$

与问题一的常物性模型不同，问题二中材料热物性与传质物性均随当前状态发生变化。其中，水分浓度不仅直接决定密度、比热容和热导率，还与温度共同影响有效扩散系数。因此，温度场和水分场之间通过状态相关物性形成双向反馈，二者不能再视为相互独立的演化过程，其耦合关系如图 4 所示。

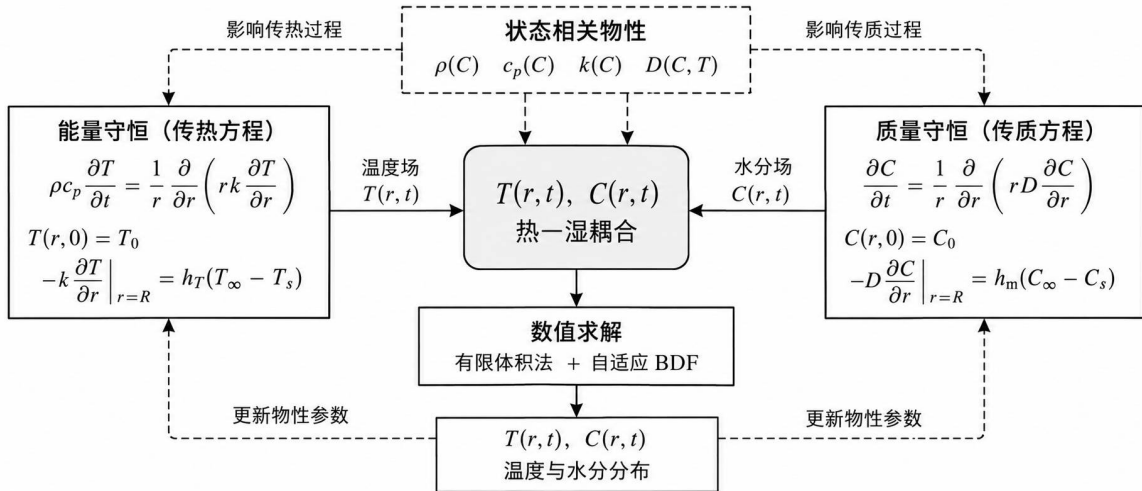


图 4 状态相关物性作用下的热湿双向耦合机制

如图 4 所示，能量守恒方程决定温度场 $T(r, t)$ ，质量守恒方程决定水分场 $C(r, t)$ ，二者又共同作用于状态相关物性参数。其中， $\rho(C)$ 、 $c_p(C)$ 和 $k(C)$ 的变化反馈至传热过程，而 $D(C, T)$ 同时受温度和水分状态影响并反馈至传质过程，由此形成闭环热湿耦合系统。基于这一结构，本文将温度与水分拼接为统一状态向量，并在每个隐式时间步同步更新物性参数和两个物理场，避免“先求温度、再求水分”的分步计算引入额外滞后。

7.3 模型求解

问题二仍采用径向有限体积法进行空间离散。内部网格取 $\Delta r = 0.025 \text{ cm}$ ，共 81 个径向节点；温度和水分拼接后，联合状态维度为 162。控制体体积 $V_i = \pi(r_{i+1/2}^2 - r_{i-1/2}^2)$ 、界面面积 $A_{i+1/2} = 2\pi r_{i+1/2}$ 均按圆柱几何组装。界面热导率 k 和扩散系数 D 采用调和平均，以便在相邻控制体物性差异较大时保持通量计算稳定。

时间方向采用 `scipy.integrate.solve_ivp` 中的自适应 BDF 隐式积分器。每次推进以上一时刻联合状态 $u^n = [T^n; C^n]$ 、附件 1 边界和附录 3 物性公式为基础，先计算 $\rho(C), c_p(C), k(C), D(C, T + 273.15)$ ，再用调和平均得到界面 k, D 。据此计算各控制体的传热、传质通量，并构成待求解方程。中心控制体自然满足对称零通量，表面控制体加入 Robin 对流通量，最终得到下一输出时刻联合状态 $u^{n+1} = [T^{n+1}; C^{n+1}]$ 。

计算设置为 $\text{rtol} = 2 \times 10^{-8}$ ，温度和水分的绝对容差分别为 2×10^{-9} 、 2×10^{-10} ，最大内部步长为 5 s。

由于 T 与 C 位于同一个状态向量内，BDF 在每个隐式步中同时处理热方程和水分方程的非线性残差，不再采用“先求温度、再求水分”的分步方式。为提高稳定性，非线性残差对应的块三对角稀疏 Jacobian 由解析式给出。

BDF 积分覆盖 0–3 h，并输出 10801 个 1 s 时刻。内部最大步长受 5 s 上限约束，最小步长出现在初期薄边界层。为检验数值稳定性，本文将网格由 $\Delta r = 0.025 \text{ cm}$ 加密至 0.0125 cm ，在 30 个论文采样点上得到温度最大绝对差 $4.16 \times 10^{-5} \text{ }^\circ\text{C}$ 、水分最大绝对差 $4.67 \times 10^{-5} \text{ kg/kg}$ 。进一步收紧 BDF 容差并缩小最大步长后，温度最大绝对差为 $1.91 \times 10^{-5} \text{ }^\circ\text{C}$ ，水分最大绝对差为 $4.84 \times 10^{-8} \text{ kg/kg}$ 。这些差异均低于四位小数显示精度，说明表 4 和表 5 的结果对网格和时间容差不敏感。

7.4 结果分析

表 4 和表 5 给出了问题二的计算结果。药材温度在前 2 h 上升较快，随后逐渐接近烘房温度；到 3 h 时，中心温度为 49.8495°C ，表面温度为 49.9664°C ，二者仅相差约 0.1169°C 。相比之下，水分浓度的径向差异仍然明显：3 h 时中心为 1.7662 kg/kg ，表面为 1.0081 kg/kg ，中心与表面相差 0.7581 kg/kg 。因此，恒温干燥后期的主要限制不再是升温，而是内部水分向表面的扩散。

图 5 从时空角度展示了上述差异。温度等值线主要集中在前半段，随后逐渐稀疏，说明温度场较快趋于准均匀；水分等值线则在整个 3 h 内持续向中心推进，表明干燥前沿仍未到达中心区域。该结果也解释了问题三中停机时刻必须由全截面最大水分浓度控制，而不能用表面含水率代替。

表 4 3 小时内药材的温度 / $^\circ\text{C}$

时间/h	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
0.5	32.1893	32.3821	32.9659	33.9608	35.4131
1.0	40.3816	40.5536	41.0605	41.8783	42.9977
1.5	45.8468	45.9348	46.1930	46.6051	47.1401
2.0	48.4502	48.4881	48.5985	48.7736	49.0034
2.5	49.4670	49.4792	49.5137	49.5653	49.6609
3.0	49.8495	49.8553	49.8746	49.9101	49.9664

表 5 3 小时内药材的水分浓度 / $\text{kg} \cdot \text{kg}^{-1}$

时间/h	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
0.5	2.5499	2.5489	2.5255	2.3258	1.6486
1.0	2.5257	2.4947	2.3578	2.0230	1.4711
1.5	2.3860	2.3256	2.1344	1.8020	1.3475
2.0	2.1708	2.1084	1.9236	1.6259	1.2311
2.5	1.9566	1.9006	1.7360	1.4720	1.1166
3.0	1.7662	1.7165	1.5702	1.3333	1.0081

图3 变物性热湿耦合条件下温度与水分浓度的时空演化

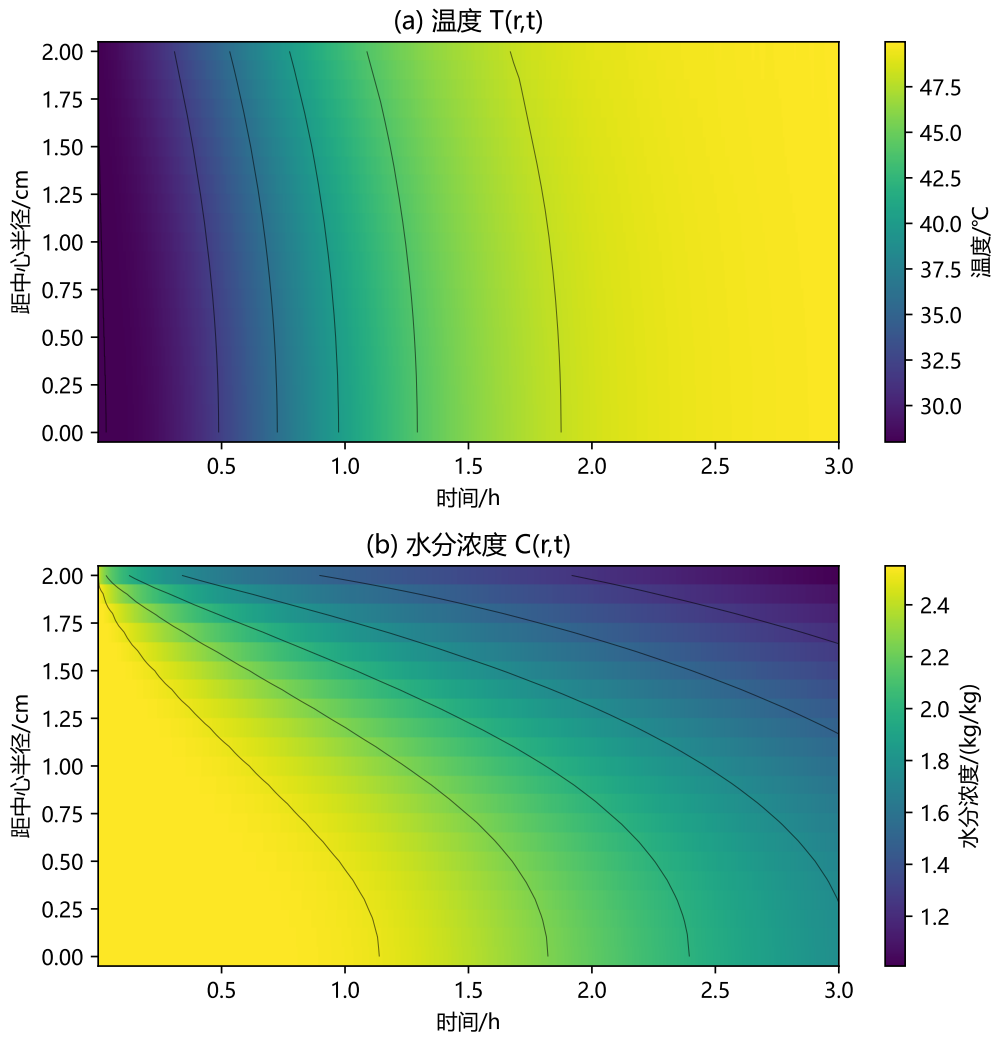


图 5 变物性热湿耦合条件下温度与水分浓度的时空演化。(a) 温度 $T(r,t)$ ；(b) 水分浓度 $C(r,t)$ ；黑色细线为等值线。温度场约 2h 内接近烘房温度、径向梯度消失；水分场在整个 3h 内保持明显径向梯度，说明恒温干燥阶段受内部水分扩散控制。

7.5 本问小结与后续衔接

问题二完成了由常物性预热模型向变物性热湿耦合模型的过渡。计算结果显示，三小时内温度趋于均匀的速度明显快于水分扩散，后续达标时间将主要受中心区域含水率控制。因此，问题三不需要重新建立控制方程，只需在本问耦合模型基础上继续积分，并加入全截面达标事件。

八、 问题三模型的建立与求解

8.1 本问任务与前文承接

问题三沿用问题二的固定半径耦合模型，目标由给定时刻输出转为达标时间定位。题目要求药材“各处”水分浓度均低于 0.15 kg/kg ，因此本文不使用表面值或平均值作停机标准，而采用全截面最大水分浓度判据。

8.2 模型建立

令固定半径区域为

$$\Omega_R = \{r \mid 0 \leq r \leq R, R = 0.02 \text{ m}\}, \quad (23)$$

并定义连续全域最大含水率

$$C_{\max}(t) = \max_{r \in \Omega_R} C(r, t). \quad (24)$$

达标时间写为

$$t^* = \inf\{t \geq 0 : C_{\max}(t) < C_{\text{cr}}\}, \quad C_{\text{cr}} = 0.15 \text{ kg/kg}. \quad (25)$$

数值离散后，径向节点集为 $\mathcal{R}_h = \{r_i\}_{i=0}^{N-1}$ ，相应离散判据为

$$C_{\max,h}(t) = \max_{r_i \in \mathcal{R}_h} C_i(t), \quad g_h(t) = C_{\max,h}(t) - C_{\text{cr}}. \quad (26)$$

当 $g_h(t)$ 由正变负时，药材全截面达到题设水分要求。由于径向扩散路径最长的位置在中心，计算中通常有

$$C_{\max,h}(t) = C(0, t), \quad (27)$$

该关系也可作为结果合理性的检查。热量方程、水分方程、初始条件和第三类边界均沿用问题二；外部边界在 $t > 4 \text{ h}$ 时取附件 1 末值恒定外推，即仍采用式 (2)。

8.3 模型求解

本问仍采用径向守恒 FVM。生产网格取

$$\Delta r = 0.00078125 \text{ cm}, \quad N = 2561, \quad (28)$$

联合状态为

$$u(t) = [T_0(t), \dots, T_{N-1}(t), C_0(t), \dots, C_{N-1}(t)]^T. \quad (29)$$

时间推进采用自适应 BDF，得到连续临界时间和离散临界时间，二者分别对应“连续事件定位”和“结果文件首个达标行”，后文分开报告。

实际求解时，固定半径网格取 $\Delta r = 0.00078125 \text{ cm}$ 、共 2561 个节点，联合状态初始化为 $u(0) = [T_0; C_0]$ 。BDF 积分器在 $g_h(t)$ 由正变负处进行步内事件定位，得到连续临界时间 t^* ；随后按 60 s 输出网格保存水分场，并抽取到题目指定半径位置。

为避免停机时间只依赖单一积分器，本文另用 backward-Euler + Picard 格式进行交叉验证。

8.4 结果分析

在附件 1 末值外推边界下，连续事件定位给出

$$t^* = 205821.7420 \text{ s} = 57.1727 \text{ h}. \quad (30)$$

终止时刻最大水分浓度出现在中心点，且

$$C_{\max,h}(t^*) = C(0, t^*) = 0.1499888 \text{ kg/kg} < C_{\text{cr}}. \quad (31)$$

按 60 s 输出网格向后取首个达标行，得到离散达标时刻

$$t_{60} = 205860 \text{ s} = 57.1833 \text{ h}. \quad (32)$$

因此，正文报告连续临界时间 t^* ，结果文件末行采用 t_{60} 。表 6 中终止行中心值显示为 0.1500，来源于四舍五入；全精度值满足全域达标条件。

表 6 药材烘干过程的水分浓度 / $\text{kg} \cdot \text{kg}^{-1}$

时间/h	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
6	1.0156	0.9868	0.9006	0.7546	0.5335
12	0.4548	0.4418	0.4019	0.3289	0.1639
18	0.2979	0.2906	0.2679	0.2247	0.0842
24	0.2371	0.2321	0.2161	0.1851	0.0663
30	0.2053	0.2013	0.1887	0.1637	0.0597
36	0.1854	0.1821	0.1714	0.1501	0.0565
42	0.1717	0.1687	0.1593	0.1404	0.0547
48	0.1615	0.1588	0.1504	0.1332	0.0536
54	0.1535	0.1511	0.1433	0.1275	0.0528
57.1727	0.1500	0.1477	0.1402	0.1249	0.0525

表 6 中，表面点在 18 h 已降至 0.0842 kg/kg，中心点仍为 0.2979 kg/kg；到 54 h，表面点为 0.0528 kg/kg，中心点仍为 0.1535 kg/kg。这说明固定半径模型的停机时间主要由中心控制，可概括为

$$C(R, t) < C_{\text{cr}} \not\Rightarrow C_{\max}(t) < C_{\text{cr}}. \quad (33)$$

图 6 中， $C = C_{\text{cr}}$ 等值线由表面向中心收缩，进一步说明干燥前沿自外向内推进；连续临界时间 t^* 对应积分器的连续事件定位，离散输出时刻 t_{60} 对应结果文件中的首个达标行。

图4 固定半径条件下水分浓度演化及全域达标时刻

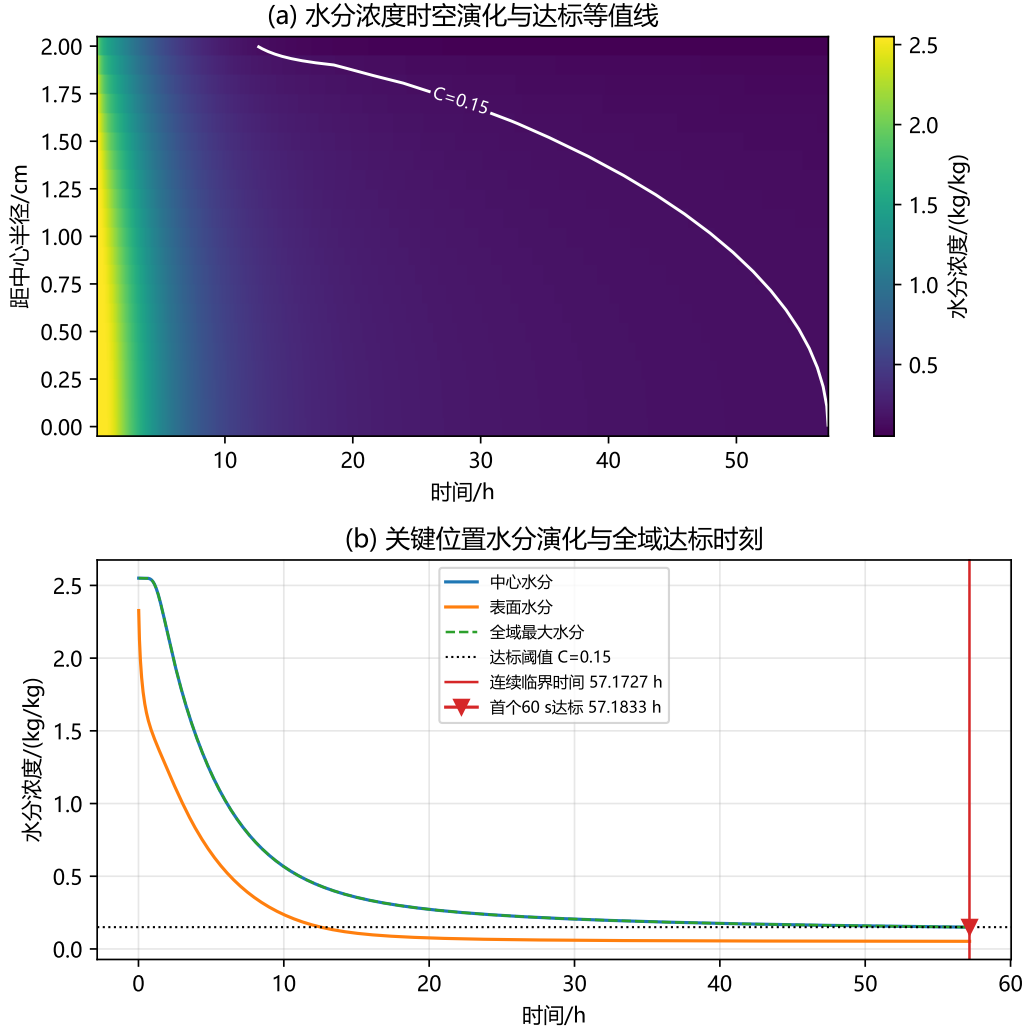


图 6 固定半径条件下水分浓度演化及全域达标时刻。(a) 水分浓度时空热力图，白色曲线为 $C = 0.15 \text{ kg/kg}$ 达标等值线；(b) 中心、表面与全域最大水分随时间演化，水平点线为达标阈值，竖线与三角标记分别给出连续临界时间与首个 60 s 达标时刻。

为区分边界假设与数值误差，记边界情景集合为

$$\mathcal{S} = \{(\Delta T_\infty, \eta_C) : \Delta T_\infty \in \{-2, 0, 2\}, \eta_C \in \{0.9, 1.0, 1.1\}\}. \quad (34)$$

在该集合上重算得

$$t^*(\mathcal{S}) \in [53.6154, 61.0702] \text{ h}. \quad (35)$$

若只扰动 h_m 的 $\pm 10\%$ ，则

$$t^* \in [56.5955, 57.9086] \text{ h}. \quad (36)$$

而数值误差量级为

$$|t_{\text{BDF}}^* - t_{\text{BE}}^*| = 7.95 \text{ s}, \quad |t_{\text{h}}^* - t_{\text{Rich}}^*| \approx 10 \text{ s}, \quad |t_{\text{h}}^* - t_{2\text{h}}^*| = 39.34 \text{ s}. \quad (37)$$

可见，边界外推带来的小时级变化显著大于数值离散误差。因此，57.1727 h 是末值外推边界下的点估计，上述边界情景区间用于描述外部边界不确定性。

8.5 本问小结与后续衔接

问题三在固定半径、附录 3 物性和末值外推边界下得到

$$t^* = 57.1727 \text{ h}, \quad t^*(S) \in [53.6154, 61.0702] \text{ h}. \quad (38)$$

该结果构成问题四移动边界模型的固定半径基准。问题四将在同一全截面判据下引入 $R(t)$ 和附录 4 物性，比较收缩对达标时间的影响。

九、 问题四模型的建立与求解

9.1 本问任务与前文承接

问题四在问题三达标停机框架上进一步引入半径收缩。与问题三相比，本问同时发生两类变化：物性关系由附录 3 改为附录 4，几何边界由固定半径 $R = 2.0 \text{ cm}$ 改为附件 2 给定的时变半径 $R(t)$ 。半径收缩会改变扩散距离和表面位置，若仍在物理半径上直接离散，需要不断移动网格。为保持计算网格稳定，本文将移动物理域映射到固定计算域，并在相同全截面含水率阈值下求解临界时间。

9.2 建模思路

移动边界处理的核心，是把随时间变化的物理区间 $r \in [0, R(t)]$ 化为固定区间。本文采用 Landau 前置固定变换，引入物质坐标

$$\xi = \frac{r}{R(t)} \in [0, 1], \quad (39)$$

从而把收缩域映射到 $\xi \in [0, 1]$ 。在均匀径向收缩假设下， ξ 表示同一物质层的位置标签；干基含水率 C 以单位干物质质量为基准，也适合在物质坐标下跟踪。对守恒式作物质导数处理后，收缩效应主要体现在两个位置：内部扩散项出现尺度因子 $1/R^2(t)$ ，表面通量项出现尺度因子 $R(t)$ 。采用这一坐标后，无需额外引入由网格收缩产生的对流项。

需要说明的是，附件 2 的 $R(t)$ 在本文中作为题目给定的经验输入，并非由含水率场反推的未知量。附录 4 密度律与附件 2 半径轨迹是否严格满足干物质质量守恒，不作为本问求解的约束条件；该一致性问题在后文模型检验与评价中单独讨论。

与固定半径模型相比，问题四临界时间由 57.1727 h 缩短至 50.8245 h 。这一差异并非单纯来自物性参数改变，而与半径收缩引起的特征扩散尺度变化密切相关。为直观说明半径收缩对内部水分迁移及最终达标时间的作用路径，其主要机制概括如图 7 所示。

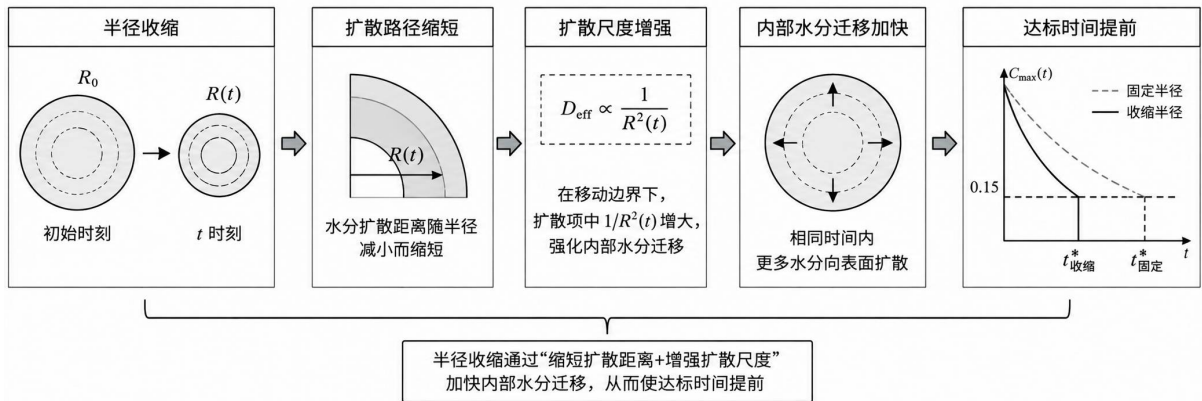


图 7 半径收缩促进内部水分迁移并缩短达标时间的作用机制

如图 7 所示，随着药材半径由 R_0 缩小至 $R(t)$ ，中心水分迁移至表面的有效扩散距离随之缩短；在物质坐标表示下，扩散项的特征尺度又随

$$\frac{1}{R^2(t)}$$

增大，从而强化单位物质坐标尺度上的水分迁移。两种作用共同促进内部高含水区域更快向表面释放水分，使全域最大含水率更早降至规定阈值。因此，在题设给定半径轨迹及附录 4 物性关系下，收缩模型的连续达标时间为 50.8245 h，较问题三固定半径模型的 57.1727 h 提前约 6.35 h。

9.3 模型建立

附录 4 给出的状态相关物性为

$$\rho(C) = 760 + 90C, \quad c_p(C) = 1850 + \frac{2150C}{C+1}, \quad k(C) = 0.12 + \frac{0.20C}{C+1}, \quad (40)$$

$$D(C, T) = 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{C}\right) \exp\left(-\frac{3850}{T_K}\right), \quad T_K = T + 273.15. \quad (41)$$

在物质坐标 ξ 下，温度场与水分场可统一写为压缩扩散方程：

$$\frac{\partial u}{\partial t} \Big|_{\xi} = \frac{1}{\text{cap} \cdot \xi R^2} \frac{\partial}{\partial \xi} \left(\xi \text{cond} \frac{\partial u}{\partial \xi} \right), \quad (42)$$

其中 $u = T$ 时 $\text{cap} = \rho(C)c_p(C)$ 、 $\text{cond} = k(C)$ ； $u = C$ 时 $\text{cap} = 1$ 、 $\text{cond} = D(C, T)$ ； $R(t)$ 由附件 2 线性插值得到。初始条件为 $T(\xi, 0) = 28$ 、 $C(\xi, 0) = 2.55$ 。中心 $\xi = 0$ 为对称零通量边界；表面 $\xi = 1$ （即 $r = R(t)$ ）为第三类边界

$$-\text{cond} \frac{\partial u}{\partial \xi} \Big|_{\xi=1} = \text{transfer} \cdot R(t) \cdot (u_s - u_{\text{ext}}), \quad (43)$$

其中温度场 $\text{transfer} = h$ 、 $u_{\text{ext}} = T_{\infty}(t)$ ，水分场 $\text{transfer} = h_m$ 、 $u_{\text{ext}} = C_{\infty}(t)$ 。4 h 后的烘房边界与问题三一致，取附件 1 末值恒定外推。

收缩和物性切换对达标时间的影响方向并不完全相同。半径收缩通过 $1/R^2(t)$ 增大有效扩散尺度，并缩短中心到表面的距离；而附录 4 中的扩散系数基值 4.2×10^{-4} 小于附录 3 中的 2.4×10^{-3} ，会降低相同状态下的水分迁移速率。二者共同决定本问相对于问题三是提前还是延后达标。停机判据仍取全剖面最大水分浓度，首次满足 $C_{\max}(t) < 0.15 \text{ kg/kg}$ 时停止积分。

9.4 模型求解

在完成移动边界坐标变换及控制方程构建后，问题四的数值求解还需同步处理时变环境边界、半径轨迹、状态相关物性以及全域达标事件。为明确各计算模块之间的数据传递关系，本文将移动边界模型的完整数值求解过程整理如图 8 所示。

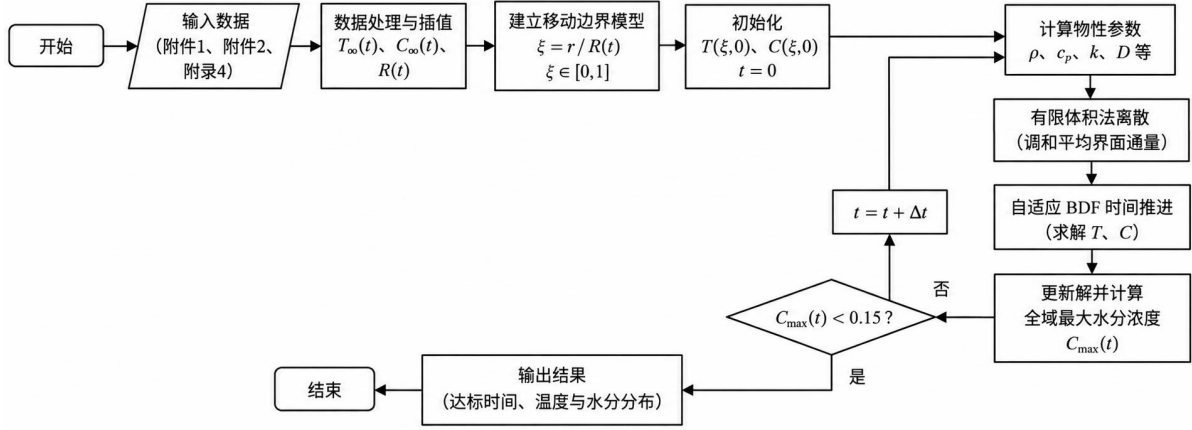


图 8 基于物质坐标变换与自适应 BDF 的问题四数值求解流程

如图 8 所示，首先对附件中的烘房温度、环境水分浓度及半径数据进行处理与插值，并利用

$$\xi = \frac{r}{R(t)}$$

将时变物理区域映射至固定计算域；随后根据当前温度、水分状态更新密度、比热容、热导率及扩散系数，并采用守恒型有限体积法完成空间离散。时间方向由自适应 BDF 隐式推进，每一时刻更新温度场与水分场后计算全域最大水分浓度 $C_{\max}(t)$ 。当

$$C_{\max}(t) < 0.15$$

时触发终止事件并输出达标时间，否则继续推进至下一时刻。该流程实现了移动边界、变物性热湿耦合与连续终点判定的统一求解。

问题四在固定 ξ 网格上采用严格径向守恒有限体积法。计算条件包括初值 T_0, C_0 、附录 4 物性、附件 1 环境边界及末值外推规则，以及附件 2 半径序列 $R(t)$ ；计算结果包括连续临界时间 t^* 和含移动表面列的水分浓度过程表。

生产计算采用固定的 ξ 网格：

$$N = 3201, \quad \Delta\xi = \frac{1}{N-1}. \quad (44)$$

控制体几何量和界面面积分别为

$$\hat{V}_i = \frac{1}{2} (\xi_{i+1/2}^2 - \xi_{i-1/2}^2), \quad \hat{A}_{i+1/2} = \xi_{i+1/2}. \quad (45)$$

上述几何量相当于在单位 ξ 圆盘上组装径向守恒通量。界面热导率 k 和扩散系数 D 均采用调和平均，以降低强梯度区域的界面通量高估风险。

时间推进仍采用 `scipy.integrate.solve_ivp` 的联合状态自适应 BDF 隐式积分器，计算参数为

$$\text{rtol} = 10^{-8}, \quad \text{atol}_T = 10^{-8}, \quad \text{atol}_C = 10^{-10}, \quad \Delta t_{\max} = 30 \text{ s}. \quad (46)$$

每个时间步由附件 2 线性插值得到当前 $R(t)$ ，并据此更新有效扩散系数 cond/R^2 和表面对流尺度 $\text{transfer} \cdot R$ ；同时由当前联合状态计算 $\rho(C), c_p(C), k(C), D(C, T + 273.15)$ ，再用调和平均组装界面通量。中心 $\xi = 0$ 用对称零通量条件消除奇点，表面 $\xi = 1$ 施加第三类边界；块三对角稀疏 Jacobian 由解析式提供。

终止事件及其搜索方向设为

$$g(t) = \max_{\xi} C(\xi, t) - 0.15, \quad \text{terminal}=\text{True}, \quad \text{direction} = -1. \quad (47)$$

到达 60 s 输出间隔时，程序按固定距离 $d = \xi R(t)$ 记录水分浓度，若指定距离已超过当前半径，则该位置不再属于药材内部。

为验证物质坐标实现，程序还进行了 $D = 0$ 时物质剖面守恒自检，并以 backward-Euler + Picard 固定步隐式格式作为独立对照。

BDF 积分触发终止事件后得到连续临界时间。内部通量平衡的最大归一化残差为 2.087×10^{-16} ，处于机器精度量级。

最小原始水分为 0.0525 kg/kg，计算过程中未触发非负裁剪；全域最大含水率始终位于中心 $\xi = 0$ ，与圆柱径向扩散的滞后规律一致。

backward-Euler + Picard 对照在 $\Delta t = 20/10/5/2.5$ s 四档时间步下分别给出 $t^* = 50.8377/50.8312/50.8279/50.8262$ h，随时间步减小向 BDF 结果收敛；最细 BE 与 BDF 相差 6.15 s，相对差为 3.36×10^{-5} 。

9.5 结果分析

计算得到

$$t^* = 182968.2250 \text{ s} = 50.8245 \text{ h}. \quad (48)$$

半径从首次采样时刻的 1.9958 cm 收缩到临界时刻的约 1.2000 cm。临界时刻中心全精度含水率已低于 0.15 kg/kg，表中显示 0.1500 来自四舍五入。第一个在 60 s 输出网格上严格达标的时刻为 183000 s = 50.8333 h，即 result4.xlsx 的末行。表 7 按题目要求给出每 6 h 及临界时刻、固定距离 0、0.5、1 cm 与移动表面处的水分浓度。由于半径在后期已降至 1.5 cm 以下，更远的固定距离已不在药材内部，该处状态应由烘房环境而非药材内部方程描述，故表中不再列出，不属于数据缺失。

表 7 药材烘干过程的水分浓度（含半径收缩 · 移动表面）/kg · kg⁻¹

时间/h	0 cm	0.5 cm	1 cm	药材表面
6	1.7172	1.5357	1.0217	0.4217
12	0.7343	0.6518	0.4069	0.1667
18	0.4066	0.3668	0.2388	0.0889
24	0.2837	0.2599	0.1783	0.0671
30	0.2253	0.2085	0.1488	0.0593
36	0.1921	0.1790	0.1312	0.0558
42	0.1706	0.1598	0.1196	0.0539
48	0.1556	0.1463	0.1113	0.0529
50.8245	0.1500	0.1413	0.1081	0.0525

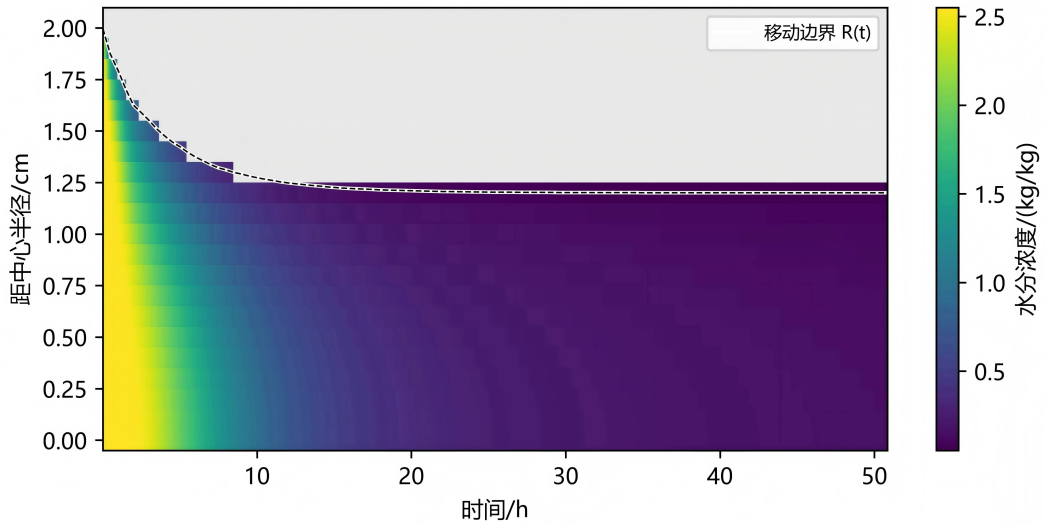


图9 半径收缩条件下物理坐标中的水分浓度演化。白色曲线为移动边界 $R(t)$ ，灰色为已收缩退出的药材区域。

图9中，移动边界 $R(t)$ 从2.0 cm收缩至临界时刻的约1.2 cm，灰色区域为已收缩退出的药材区域。剩余区域内水分浓度仍呈中心高、表面低的分布形态；表面含水率后期接近0.0525 kg/kg，与环境末值0.04986 kg/kg相近，而中心含水率下降较慢并继续控制终止时刻。

边界情景分析给出了默认答案的外部适用范围。将4 h后温度扰动 $\pm 2^\circ\text{C}$ 、水分扰动 $\pm 10\%$ 组成 3×3 情景时， t^* 落在47.6834–54.2753 h；传质系数 h_m 扰动 $\pm 10\%$ 时， t^* 落在50.6035–51.1159 h；用附件最后30分钟或1 h均值延拓代替末值外推时， t^* 为51.0757–51.0972 h。与这些小时量级的边界差异相比，数值离散误差较小：网格201 $\rightarrow$ 3201的末级差为3.30 s，BDF容差三档差小于 2×10^{-4} s，BDF与BE-Picard对照差为6.15 s。因此，50.8245 h可作为题设默认边界下的点估计，边界情景区间用于说明后期烘房条件外推带来的不确定性。

9.6 本问小结与后续衔接

问题四通过物质坐标前置固定，将半径收缩问题转化为固定域上的压缩扩散问题，并在同一全截面达标判据下得到连续临界时间 $t^* = 50.8245$ h。该时间短于问题三固定半径结果57.1727 h，说明在题设半径轨迹和附录4物性下，扩散路径缩短带来的加速作用占主导。后续模型检验与评价将进一步讨论数值收敛、坐标解释差异以及附件2半径轨迹与附录4密度律之间的闭合关系。

十、模型的检验与分析

10.1 数值收敛检验

为确认计算结果不受离散设置支配，本文从空间网格、时间精度和独立求解器三个方面进行检验。

为排除空间离散尺度和时间推进策略对临界时间结果的主导影响，本文进一步从空间网格加密和独立时间积分格式两个层面开展收敛性检验。问题三考察固定半径条件下临界时间随径向网格尺度的变化，问题四考察移动边界模型随物质坐标节点数增加的收敛趋势，并采用后向欧拉（BE）与BDF两种隐式格式进行交叉验证，结果如图10所示。

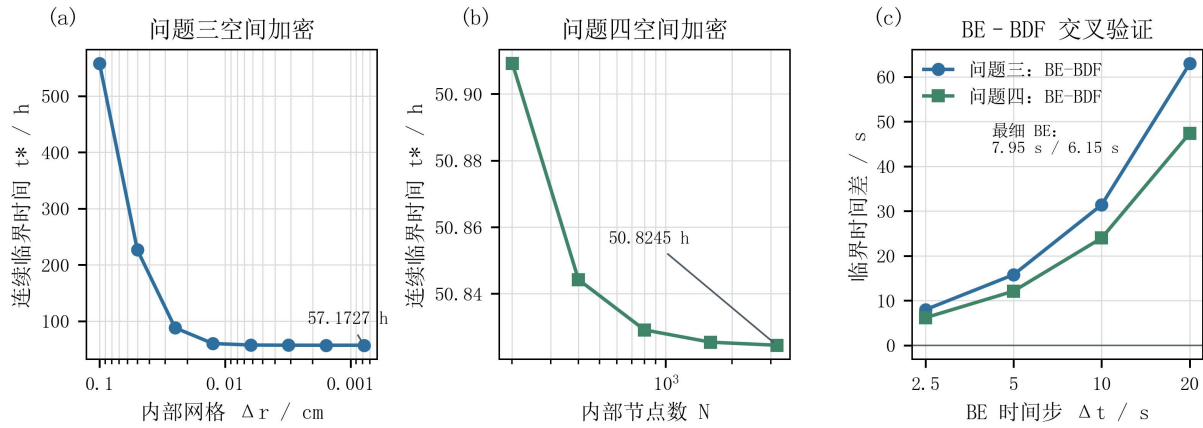


图 10 空间离散与时间积分的数值收敛及交叉验证结果

图 10(a) 表明，随着问题三内部径向网格持续加密，连续临界时间逐渐收敛至 57.1727 h；图 10(b) 显示，问题四在增加物质坐标节点数后，临界时间稳定于 50.8245 h 附近。与此同时，图 10(c) 中 BE 与 BDF 两种时间推进方法在步长减小时均表现出一致的收敛趋势。上述结果说明，本文最终临界时间并非由特定网格尺度或单一时间积分器偶然产生，数值离散误差已被控制在远低于边界条件不确定性的水平。

问题一将内部网格继续加密后，温度和水分的最大变化均不超过 1.90×10^{-4} ；界面扩散系数改用调和平均所引起的变化也低于报告精度。因此，正文表格中的四位小数保持稳定。

问题二采用两级网格比较，温度和水分的最大差均小于 5×10^{-5} 。进一步收紧 BDF 容差并缩小最大步长后，表中数值在四位小数下没有变化。

问题三对网格更敏感。末两级网格的事件时间相差 39.34 s，生产网格与 Richardson 外推值相差 10.01 s；严格 BDF 与后向欧拉–Picard 的结果相差 7.95 s。三种检验给出的误差均处于可接受范围。

调和平均需要与足够细的网格配合。若直接在 0.1 cm 粗网格上计算，会把近表面的低扩散区域集中为过大的界面阻力，得到 557.4174 h 的异常结果；持续加密后，终止时间稳定收敛到 57.1727 h。因此，粗网格结果不具有物理解释意义。

问题四的末两级网格结果相差 3.30 s，最细后向欧拉结果与 BDF 相差 6.15 s。计算过程中水分剖面保持单调，最大含水率始终位于中心，说明事件判据与径向扩散规律一致。

10.2 问题四两种坐标模型的结果比较

问题四以物质坐标 FVM-BDF 为主模型。由于题目未给出内部形变速度场，本文另计算 Euler 空间坐标模型，用于考察坐标解释对结果的影响。表 8 仅作敏感性比较，不表示两个结果均为答案。

表 8 问题四两种坐标模型的结果比较

模型	t^*/h	说明
物质坐标 FVM-BDF 模型	50.8245	问题四最终结果
Euler 空间坐标模型	52.3842	坐标处理敏感性对照，不纳入主答案

物质坐标模型以 $\xi = r/R(t)$ 标记同一物质层，符合均匀径向收缩假设，得到 $t^* = 50.8245$ h。Euler 空间坐标模型保留收缩对流项，结果比主模型晚 1.5597 h，反映了内部形变场未知带来的模型形式敏感性。

作为数值实现互证，固定步后向欧拉–Picard 方案与 FVM-BDF 主结果仅相差 0.0301 h，约为 0.06%。由此可见，数值实现差异小于坐标解释和边界假设造成的影响。

10.3 边界敏感性讨论

问题三和问题四的计算时长均超过附件 1 覆盖的前 4h，后期结果因而依赖边界末值外推。边界敏感性分析用于把这一外部假设的影响与数值误差区分开。

当后期温度扰动 $\pm 2^{\circ}\text{C}$ 、环境水分扰动 $\pm 10\%$ 时，问题三的达标时间为 53.6154–61.0702 h，问题四为 47.6834–54.2753 h。其中温度变化的影响更明显，传质系数扰动和不同末值平均方式造成的变化相对较小。

上述情景变化达到小时量级，而网格和求解器误差主要处于秒级。因此，57.1727 h 和 50.8245 h 应理解为末值外推条件下的点估计；敏感性范围只说明边界假设的影响，不代表统计置信区间。

问题四表格只列至 1 cm 固定距离并附移动表面，是因为更远的位置在后期已退出药材内部，并非数据缺失。

10.4 物理合理性检验

四问呈现出一致的物理规律：外部热风先影响表面，中心响应较慢。温度场较早趋于均匀，而水分梯度持续时间更长，最终由中心含水率控制停机。

问题四中，半径收缩缩短了水分传递路径，使达标时间早于固定半径情形。这些现象符合圆柱径向扩散和移动边界的基本规律。

十一、 模型评价

11.1 模型优点

- 1、 四问采用统一的径向有限体积框架，模型层次清楚，前后衔接自然。
- 2、 联合求解温度与水分状态，能够反映变物性条件下的热湿相互影响。
- 3、 停机判据面向全截面，并通过网格加密和独立求解器检验结果稳定性。

11.2 模型不足

- 1、 后期环境条件采用末值外推，难以反映实际烘房温湿度的持续波动。
- 2、 受题给参数限制，模型未显式描述蒸发潜热，收缩与密度关系也未完全闭合。

11.3 模型的推广

本模型可用于其他圆柱状物料的热风干燥预测；更换物性、尺寸、边界条件和达标阈值后，即可重新计算温湿分布与停机时间。

十二、 AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于思路梳理、代码辅助编写与调试、数值结果复核、格式检查，详细使用情况见支撑材料。

十三、 参考文献

- [1] 刘琦. 基于有限体积法非傅里叶导热及热粘弹性耦合问题的研究 [D]. 哈尔滨工程大学, 2020.
- [2] 单惠敏. 香菇热风烘干实验与数值模拟 [D]. 湖南科技大学, 2021.
- [3] 庞凌云, 袁志华, 詹丽娟, 等. 铁棍山药片热风干燥过程中传热传质规律 [J]. 食品与机械, 2024, 40(8): 157–165.
- [4] Morris D E. Factorial sampling plans for preliminary computational experiments. *Technometrics*, 1991, 33(2): 161–174.
- [5] Xiu D, Karniadakis G E. The Wiener-Askey polynomial chaos for stochastic differential equations. *SIAM Journal on Scientific Computing*, 2002, 24(2): 619–644.
- [6] Sobol I M. Global sensitivity indices for nonlinear mathematical models and their Monte Carlo estimates. *Mathematics and Computers in Simulation*, 2001, 55(1–3): 271–280.
- [7] Saltelli A, Annoni P, Azzini I, Campolongo F, Riva M, Tarantola S. Variance based sensitivity analysis of model output: Design and estimator for the total sensitivity index. *Computer Physics Communications*, 2010, 181(2): 259–270.

附录 A：支撑材料清单

文件名	说明
problem1.py	问题一求解器：径向 FVM + BDF 时间推进，输出温度场与水分场
problem2.py	问题二求解器：变物性耦合模型，输出中心温度与干燥时间
problem3.py	问题三求解器：严格径向 FVM + BDF，含后向欧拉交叉验证
problem4.py	问题四求解器：动边界 FVM + BDF，多情景模拟
sensitivity.py	全局敏感性分析：Morris 筛选 + PCE-Sobol 指数 + Radau 交叉验证
figures.py	论文插图生成脚本（图 1-图 5）
limitations.py	模型局限性分析：干物质闭合、潜热影响、半径上限证明
mass_closure.py	质量闭合验证：干物质守恒与水分平衡核算
bonus.py	附加验证：Bessel 解析模态、时间推进阶数、参数敏感性
交叉验证.py	BE-BDF 交叉验证绘图与误差分析
result1.xlsx	问题一数值结果（温度场、水分场随时间演化）
result2.xlsx	问题二数值结果（中心温度、干燥时间表）
result3.xlsx	问题三数值结果（径向温度/水分分布）
result4.xlsx	问题四数值结果（多情景半径、温度、水分演化）
requirements.txt	Python 依赖包清单（numpy/scipy/pandas/matplotlib/openpyxl/Pillow）
Factorial Sampling.pdf	Morris (1991) 因子筛选方法原文
Sobol_2001.pdf	Sobol (2001) 全局敏感性指数原文
Variance based.pdf	Saltelli 等 (2010) 方差敏感性分析综述
XiuK_SISC02.pdf	Xiu & Karniadakis (2002) 多项式混沌方法原文

附录 B：代码清单

```
1 """问题一：一维径向守恒有限体积 + 隐式 BDF，求 30 min 预热阶段的温度场与水分场。
2 """
3 from __future__ import annotations
4
5 import json
6 from dataclasses import asdict, dataclass, replace
7 from pathlib import Path
8
9 import numpy as np
10 import openpyxl
11 from scipy.integrate import solve_ivp
12 from scipy.sparse import diags
13
14
15 # ===== 求解核心 =====
16 @dataclass(frozen=True)
17 class EnvironmentSeries:
18     """Piecewise-linear oven boundary data."""
19
20     time_s: np.ndarray
21     temperature_c: np.ndarray
22     moisture: np.ndarray
23
24     def __post_init__(self) -> None:
25         sizes = {len(self.time_s), len(self.temperature_c), len(self.moisture)}
26         if len(sizes) != 1 or not len(self.time_s):
27             raise ValueError("environment arrays must have the same non-zero length")
28         if np.any(np.diff(self.time_s) <= 0):
29             raise ValueError("environment times must be strictly increasing")
30
31     def temperature_at(self, t_s: float) -> float:
32         return float(np.interp(t_s, self.time_s, self.temperature_c))
33
34     def moisture_at(self, t_s: float) -> float:
35         return float(np.interp(t_s, self.time_s, self.moisture))
36
37
38 @dataclass(frozen=True)
39 class SolverConfig:
40     radius_cm: float = 2.0
41     internal_dr_cm: float = 0.0025
42     output_dr_cm: float = 0.1
43     duration_s: int = 1800
44     output_dt_s: float = 1.0
45     initial_temperature_c: float = 28.0
46     initial_moisture: float = 2.55
47     density_kg_m3: float = 820.0
48     heat_capacity_j_kg_k: float = 2600.0
49     thermal_conductivity_w_m_k: float = 0.36
50     heat_transfer_w_m2_k: float = 25.0
51     mass_transfer_m_s: float = 8e-7
52     relative_tolerance: float = 1e-7
53     absolute_tolerance: float = 1e-9
54     max_time_step_s: float = 10.0
55     interface_mean: str = "harmonic" # 改进①：界面 D 谐波平均
56     clip_negative_moisture: bool = True # 改进②：输出前 C≥0 裁剪
57
58
59 @dataclass(frozen=True)
60 class SimulationResult:
61     time_s: np.ndarray
62     radius_cm: np.ndarray
63     temperature_c: np.ndarray
64     moisture: np.ndarray
65
66
67 def build_radial_grid_cm(radius_cm: float, dr_cm: float) -> np.ndarray:
68     count = int(round(radius_cm / dr_cm)) + 1
69     if count < 2 or not np.isclose(radius_cm / dr_cm, count - 1):
70         raise ValueError("radius_cm must be an integer multiple of dr_cm")
71     return np.linspace(0.0, radius_cm, count)
72
73
74 def moisture_diffusivity(c: np.ndarray) -> np.ndarray:
75     """Problem-1 diffusivity, D(C)=7e-9 exp(-0.89/C), in m²/s."""
76
77     safe_c = np.maximum(np.asarray(c, dtype=float), 1e-12)
78     return 7e-9 * np.exp(-0.89 / safe_c)
```

```

79
80
81 def interface_average(d_left: np.ndarray, d_right: np.ndarray, kind: str) -> np.ndarray:
82     """控制体积界面扩散系数平均。
83
84     kind="arithmetic":  $0.5 \times (D_l + D_r)$  (原版)。
85     kind="harmonic":  $2 \times D_l \times D_r / (D_l + D_r)$  (改进①); 对 D 跨数量级的串联扩散更准;
86         且当任一侧  $D \rightarrow 0$  (表面强脱水) 时界面  $D \rightarrow 0$ , 比算术平均更物理。防  $0/0$ : 分母为 0 时取 0。
87     """
88
89     if kind == "arithmetic":
90         return 0.5 * (d_left + d_right)
91     if kind != "harmonic":
92         raise ValueError(f"unknown interface_mean: {kind!r} (expected 'harmonic' or 'arithmetic')")
93     denom = d_left + d_right
94     safe = np.where(denom > 0.0, denom, 1.0)
95     return np.where(denom > 0.0, 2.0 * d_left * d_right / safe, 0.0)
96
97
98 def _radial_geometry(radius_m: float, dr_m: float) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
99     node_count = int(round(radius_m / dr_m)) + 1
100     radius = np.linspace(0.0, radius_m, node_count)
101     left_faces = np.maximum(0.0, radius - dr_m / 2.0)
102     right_faces = np.minimum(radius_m, radius + dr_m / 2.0)
103     volumes = np.pi * (right_faces**2 - left_faces**2)
104     interface_areas = 2.0 * np.pi * (radius[:-1] + dr_m / 2.0)
105     return radius, volumes, interface_areas
106
107
108 def _add_internal_fluxes(
109     derivative_numerator: np.ndarray,
110     values: np.ndarray,
111     interface_transport: np.ndarray,
112     interface_areas: np.ndarray,
113     dr_m: float,
114 ) -> None:
115     flux = interface_transport * interface_areas * (values[1:] - values[:-1]) / dr_m
116     derivative_numerator[:-1] += flux
117     derivative_numerator[1:] -= flux
118
119
120 def _sample_to_output_grid(
121     internal_radius_cm: np.ndarray,
122     output_radius_cm: np.ndarray,
123     states_by_time: np.ndarray,
124 ) -> np.ndarray:
125     spacing_cm = internal_radius_cm[1] - internal_radius_cm[0]
126     indices = np rint(output_radius_cm / spacing_cm).astype(int)
127     if np.allclose(internal_radius_cm[indices], output_radius_cm):
128         return states_by_time[:, indices]
129     return np.vstack(
130         [np.interp(output_radius_cm, internal_radius_cm, row) for row in states_by_time]
131     )
132
133
134 def solve_problem1(environment: EnvironmentSeries, config: SolverConfig | None = None) -> SimulationResult:
135     """Solve the decoupled heat/moisture model by radial FVM + implicit BDF."""
136
137     config = config or SolverConfig()
138     internal_radius_cm = build_radial_grid_cm(config.radius_cm, config.internal_dr_cm)
139     output_radius_cm = build_radial_grid_cm(config.radius_cm, config.output_dr_cm)
140     dr_m = config.internal_dr_cm / 100.0
141     radius_m = config.radius_cm / 100.0
142     _, volumes, interface_areas = _radial_geometry(radius_m, dr_m)
143     surface_area = 2.0 * np.pi * radius_m
144     node_count = len(internal_radius_cm)
145     jacobian_pattern = diags(
146         [np.ones(node_count - 1), np.ones(node_count), np.ones(node_count - 1)],
147         offsets=[-1, 0, 1],
148         shape=(node_count, node_count),
149         format="csc",
150     )
151     output_time_s = np.arange(
152         0.0,
153         config.duration_s + config.output_dt_s / 2.0,
154         config.output_dt_s,
155     )
156
157     def temperature_rhs(t_s: float, temperature: np.ndarray) -> np.ndarray:

```

```

158     numerator = np.zeros_like(temperature)
159     _add_internal_fluxes(
160         numerator,
161         temperature,
162         np.full(node_count - 1, config.thermal_conductivity_w_m_k),
163         interface_areas,
164         dr_m,
165     )
166     numerator[-1] += (
167         config.heat_transfer_w_m2_k
168         * surface_area
169         * (environment.temperature_at(t_s) - temperature[-1])
170     )
171     return numerator / (config.density_kg_m3 * config.heat_capacity_j_kg_k * volumes)
172
173 def moisture_rhs(t_s: float, moisture: np.ndarray) -> np.ndarray:
174     numerator = np.zeros_like(moisture)
175     node_diffusivity = moisture_diffusivity(moisture)
176     # 改进①: 界面 D 用谐波平均 (config.interface_mean="arithmetic" 时退化为原版算术平均)
177     interface_diffusivity = interface_average(
178         node_diffusivity[:-1], node_diffusivity[1:], config.interface_mean
179     )
180     _add_internal_fluxes(
181         numerator,
182         moisture,
183         interface_diffusivity,
184         interface_areas,
185         dr_m,
186     )
187     numerator[-1] += (
188         config.mass_transfer_m_s
189         * surface_area
190         * (environment.moisture_at(t_s) - moisture[-1])
191     )
192     return numerator / volumes
193
194 common_options = {
195     "method": "BDF",
196     "t_eval": output_time_s,
197     "rtol": config.relative_tolerance,
198     "atol": config.absolute_tolerance,
199     "max_step": config.max_time_step_s,
200     "jac_sparsity": jacobian_pattern,
201 }
202 temperature_solution = solve_ivp(
203     temperature_rhs,
204     (0.0, float(config.duration_s)),
205     np.full(node_count, config.initial_temperature_c),
206     **common_options,
207 )
208 moisture_solution = solve_ivp(
209     moisture_rhs,
210     (0.0, float(config.duration_s)),
211     np.full(node_count, config.initial_moisture),
212     **common_options,
213 )
214 if not temperature_solution.success:
215     raise RuntimeError(f"temperature integration failed: {temperature_solution.message}")
216 if not moisture_solution.success:
217     raise RuntimeError(f"moisture integration failed: {moisture_solution.message}")
218
219 temperature = _sample_to_output_grid(
220     internal_radius_cm,
221     output_radius_cm,
222     temperature_solution.y.T,
223 )
224 moisture = _sample_to_output_grid(
225     internal_radius_cm,
226     output_radius_cm,
227     moisture_solution.y.T,
228 )
229 # 改进②: 表面强脱水可能出现非物理负水分, 输出前裁剪到 C≥0 (problem1 工况实测不触发, 防御性)
230 if config.clip_negative_moisture:
231     moisture = np.maximum(moisture, 0.0)
232 return SimulationResult(output_time_s, output_radius_cm, temperature, moisture)
233
234
235 # ===== 运行与产物 (全部默认, 无命令行参数) =====
236 BASE = Path( file ).resolve().parent # 代码同根目录

```

```

237 ATTACHMENT1 = BASE / "附件" / "附件1.xlsx" # 环境数据（输入）
238 TEMPLATE = BASE / "附件" / "附件3" / "result1.xlsx" # 结果格式模板（题目附件3）
239 OUT_XLSX = BASE / "result1.xlsx" # 输出：同根目录
240 OUT_JSON = BASE / "verification1.json"
241 SAMPLE_TIMES = np.array([100, 300, 600, 900, 1200, 1500, 1800], dtype=float)
242 SAMPLE_RADII_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0], dtype=float)
243 TEMP_SHEET = "温度"
244 MOIST_SHEET = "水分浓度"
245
246
247 def load_environment(path: Path) -> EnvironmentSeries:
248     """openpyxl 读附件1 三列（时间/温度/水分浓度）→ EnvironmentSeries。"""
249
250     worksheet = openpyxl.load_workbook(path, data_only=True).worksheets[0]
251     rows = [r for r in worksheet.iter_rows(min_row=2, values_only=True) if r[0] not in (None, "")]
252     return EnvironmentSeries(
253         time_s=np.asarray([float(r[0]) for r in rows], dtype=float),
254         temperature_c=np.asarray([float(r[1]) for r in rows], dtype=float),
255         moisture=np.asarray([float(r[2]) for r in rows], dtype=float),
256     )
257
258
259 def selected_values(result: SimulationResult, values: np.ndarray) -> np.ndarray:
260     time_indices = [int(np.where(np.isclose(result.time_s, value))[0][0]) for value in SAMPLE_TIMES]
261     radius_indices = [int(np.where(np.isclose(result.radius_cm, value))[0][0]) for value in SAMPLE_RADII_CM]
262     return values[np.ix_(time_indices, radius_indices)]
263
264
265 def difference_summary(left: np.ndarray, right: np.ndarray) -> dict[str, float]:
266     """两场逐点对照：最大绝对差 + 平均绝对差（论文模型检验章引用）。"""
267
268     absolute = np.abs(left - right)
269     return {
270         "maximum_absolute_difference": float(absolute.max()),
271         "mean_absolute_difference": float(absolute.mean()),
272     }
273
274
275 def write_result_xlsx(result: SimulationResult, template: Path, out_path: Path) -> None:
276     """读附件3 格式模板 → 按完整时间×半径网格覆盖写温度/水分两表（全精度存储，四位小数显示）。"""
277
278     workbook = openpyxl.load_workbook(template)
279     radius = result.radius_cm
280     for sheet_name, field in ((TEMP_SHEET, result.temperature_c), (MOIST_SHEET, result.moisture)):
281         sheet = workbook[sheet_name]
282         sheet.cell(1, 1).value = "时间\\到药材中心的距离"
283         for j in range(radius.size):
284             sheet.cell(1, 2 + j).value = round(float(radius[j]), 10)
285         for i in range(1, result.time_s.size): # t=1..1800（跳过 t=0 初值行）
286             sheet.cell(i + 1, 1).value = int(round(result.time_s[i]))
287             for j in range(radius.size):
288                 cell = sheet.cell(i + 1, 2 + j)
289                 cell.value = float(field[i, j]) # 全精度存储，仅用 number_format 控制四位小数显示（与原 result1.xlsx 一致）
290                 cell.number_format = "0.0000"
291             sheet.freeze_panes = "B2"
292     out_path.parent.mkdir(parents=True, exist_ok=True)
293     workbook.save(out_path)
294
295
296 def main() -> None:
297     if not ATTACHMENT1.exists():
298         raise SystemExit(
299             f"未找到附件: {ATTACHMENT1}\n"
300             "请把题目附件 附件1.xlsx 放到本脚本同根目录的 附件/ 文件夹下，再运行: python problem1.py"
301         )
302     if not TEMPLATE.exists():
303         raise SystemExit(
304             f"未找到结果格式模板: {TEMPLATE}\n"
305             "请把题目附件3 的 result1.xlsx 放到 附件/附件3/ 下，再运行: python problem1.py"
306         )
307     environment = load_environment(ATTACHMENT1)
308
309     improved_config = SolverConfig() # 默认：谐波平均 + C≥0 裁剪（主结果）
310     baseline_config = SolverConfig(interface_mean="arithmetic", clip_negative_moisture=False) # 原版行为（对照）
311     grid_config = replace(improved_config, internal_dr_cm=improved_config.internal_dr_cm / 2.0) # 网格加密一倍
312
313     print("求解 baseline（算术平均、无裁剪，对照）...", flush=True)

```

```

314 baseline = solve_problem1(environment, baseline_config)
315 print("求解 improved (谐波平均、C≥0 裁剪, 主结果) ...", flush=True)
316 improved = solve_problem1(environment, improved_config)
317 print(f"求解 grid-refined (内部网格 {grid_config.internal_dr_cm} cm) ...", flush=True)
318 grid = solve_problem1(environment, grid_config)
319
320 grid_independence = {
321     "note": f"内部网格 {improved_config.internal_dr_cm} vs {grid_config.internal_dr_cm} cm (同一输出网格)",
322     "temperature_c": difference_summary(improved.temperature_c, grid.temperature_c),
323     "moisture": difference_summary(improved.moisture, grid.moisture),
324 }
325 improvement_effect = {
326     "note": "谐波平均 vs 算术平均 (同环境、同网格, 唯一变量=界面平均方式+裁剪); 温度场解耦故逐点一致",
327     "temperature_identical": bool(np.array_equal(improved.temperature_c, baseline.temperature_c)),
328     "moisture": difference_summary(improved.moisture, baseline.moisture),
329 }
330 physical_checks = {
331     "moisture_min": float(improved.moisture.min()),
332     "moisture_max": float(improved.moisture.max()),
333     "temperature_min_c": float(improved.temperature_c.min()),
334     "temperature_max_c": float(improved.temperature_c.max()),
335     "clip_is_noop": bool(improved.moisture.min() > 0.0),
336     "all_finite": bool(np.isfinite(improved.temperature_c).all() and np.isfinite(improved.moisture).all()),
337 }
338 verification = {
339     "source_attachment": str(ATTACHMENT1),
340     "config": asdict(improved_config),
341     "grid_independence": grid_independence,
342     "improvement_effect": improvement_effect,
343     "physical_checks": physical_checks,
344     "selected_tables": {
345         "temperature_c": [[float(t), *[float(v) for v in row]] for t, row in zip(SAMPLE_TIMES,
346             selected_values(improved, improved.temperature_c))],
347         "moisture": [[float(t), *[float(v) for v in row]] for t, row in zip(SAMPLE_TIMES,
348             selected_values(improved, improved.moisture))],
349     },
350 }
351 write_result_xlsx(improved, TEMPLATE, OUT_XLSX)
352 OUT_JSON.write_text(json.dumps(verification, ensure_ascii=False, indent=2), encoding="utf-8")
353
354 bar = "=" * 64
355 radii_header = " ".join(f"{r:>7.1f}" for r in SAMPLE_RADII_CM)
356 print(bar, flush=True)
357 print("问题— 30 min 预热 — 温度/水分径向分布 (谐波平均界面 D + C≥0 裁剪)", flush=True)
358 print(bar, flush=True)
359 print("[表1] 温度/℃ (行=时间/s, 列=距中心半径/cm)", flush=True)
360 print(f"{'t/s':>6} {radii_header}", flush=True)
361 for t, row in zip(SAMPLE_TIMES, selected_values(improved, improved.temperature_c)):
362     print(f"{int(t):>6} " + " ".join(f"{v:>7.4f}" for v in row), flush=True)
363 print("[表2] 水分浓度/(kg/kg)", flush=True)
364 print(f"{'t/s':>6} {radii_header}", flush=True)
365 for t, row in zip(SAMPLE_TIMES, selected_values(improved, improved.moisture)):
366     print(f"{int(t):>6} " + " ".join(f"{v:>7.4f}" for v in row), flush=True)
367 print("-" * 64, flush=True)
368 print(f"[网格无关性] 内部网格减半后: 温度最大变化={grid_independence['temperature_c']}
369     ['maximum_absolute_difference']:.3e} °C, "
370     f"水分最大变化={grid_independence['moisture']['maximum_absolute_difference']:.3e} kg/kg", flush=True)
371 print(f"[改进效果] 算术-谐波平均: 温度场逐点一致={improvement_effect['temperature_identical']}, "
372     f"水分最大变化={improvement_effect['moisture']['maximum_absolute_difference']:.3e} kg/kg", flush=True)
373 print(f"[物理校验] 水分 min={physical_checks['moisture_min']:.6f} max={physical_checks['moisture_max']:.6f}; "
374     f"温度 min={physical_checks['temperature_min_c']:.4f} max={physical_checks['temperature_max_c']:.4f} °C; "
375     f"C≥0 裁剪为 no-op={physical_checks['clip_is_noop']}", flush=True)
376 print(bar, flush=True)
377 print(f"已写出: {OUT_XLSX} (温度/水分两表, 按附件3 模板格式)、{OUT_JSON}", flush=True)
378
379 if __name__ == "__main__":
380     main()

```

```

1 """问题二: 变物性(附录3)热湿耦合, 守恒径向有限体积 + 联合状态自适应 BDF, 求 3 h 烘干的温度/水分场。
2 """
3 from __future__ import annotations
4
5 import json
6 from dataclasses import asdict, dataclass, replace
7 from pathlib import Path
8
9 import numpy as np
10 import openpyxl
11 import pandas as pd
12 from scipy.integrate import solve_ivp
13 from scipy.sparse import bmat, csc_matrix, diags
14
15
16 # ===== 求解核心 =====
17 @dataclass(frozen=True)
18 class EnvironmentSeries:
19     """Piecewise-linear oven temperature and moisture boundary data."""
20
21     time_s: np.ndarray
22     temperature_c: np.ndarray
23     moisture: np.ndarray
24
25     def __post_init__(self) -> None:
26         sizes = {len(self.time_s), len(self.temperature_c), len(self.moisture)}
27         if len(sizes) != 1 or not len(self.time_s):
28             raise ValueError("environment arrays must have the same non-zero length")
29         if np.any(np.diff(self.time_s) <= 0):
30             raise ValueError("environment times must be strictly increasing")
31         if not (
32             np.isfinite(self.time_s).all()
33             and np.isfinite(self.temperature_c).all()
34             and np.isfinite(self.moisture).all()
35         ):
36             raise ValueError("environment arrays must be finite")
37
38     def temperature_at(self, t_s: float) -> float:
39         return float(np.interp(t_s, self.time_s, self.temperature_c))
40
41     def moisture_at(self, t_s: float) -> float:
42         return float(np.interp(t_s, self.time_s, self.moisture))
43
44
45 @dataclass(frozen=True)
46 class SolverConfig:
47     radius_cm: float = 2.0
48     internal_dr_cm: float = 0.025
49     output_dr_cm: float = 0.1
50     duration_s: int = 10800
51     output_dt_s: float = 1.0
52     initial_temperature_c: float = 28.0
53     initial_moisture: float = 2.55
54     heat_transfer_w_m2_k: float = 25.0
55     mass_transfer_m_s: float = 8e-7
56     relative_tolerance: float = 2e-8
57     temperature_absolute_tolerance: float = 2e-9
58     moisture_absolute_tolerance: float = 2e-10
59     max_time_step_s: float = 5.0
60
61
62 @dataclass(frozen=True)
63 class SolverDiagnostics:
64     internal_node_count: int
65     state_size: int
66     function_evaluations: int
67     jacobian_evaluations: int
68     lu_decompositions: int
69     minimum_temperature_c: float
70     minimum_moisture: float
71
72
73 @dataclass(frozen=True)
74 class SimulationResult:
75     time_s: np.ndarray
76     radius_cm: np.ndarray
77     temperature_c: np.ndarray
78     moisture: np.ndarray

```

```

79     diagnostics: SolverDiagnostics
80
81
82 def build_radial_grid_cm(radius_cm: float, dr_cm: float) -> np.ndarray:
83     count = int(round(radius_cm / dr_cm)) + 1
84     if count < 2 or not np.isclose(radius_cm / dr_cm, count - 1):
85         raise ValueError("radius_cm must be an integer multiple of dr_cm")
86     return np.linspace(0.0, radius_cm, count)
87
88
89 def density(c: np.ndarray) -> np.ndarray:
90     return 650.0 + 128.0 * np.asarray(c, dtype=float)
91
92
93 def heat_capacity(c: np.ndarray) -> np.ndarray:
94     c = np.asarray(c, dtype=float)
95     return 1450.0 + 2736.0 * c / (c + 1.0)
96
97
98 def thermal_conductivity(c: np.ndarray) -> np.ndarray:
99     c = np.asarray(c, dtype=float)
100    return 0.21 + 0.38 * c / (c + 1.0)
101
102
103 def moisture_diffusivity(c: np.ndarray, temperature_c: np.ndarray) -> np.ndarray:
104    safe_c = np.maximum(np.asarray(c, dtype=float), 1e-9)
105    temperature_k = np.asarray(temperature_c, dtype=float) + 273.15
106    if np.any(temperature_k <= 0.0):
107        raise ValueError("temperature must be above absolute zero")
108    return 2.4e-3 * np.exp(-0.45 / safe_c) * np.exp(-3850.0 / temperature_k)
109
110
111 def harmonic_mean(left: np.ndarray, right: np.ndarray) -> np.ndarray:
112    left = np.asarray(left, dtype=float)
113    right = np.asarray(right, dtype=float)
114    denominator = left + right
115    result = np.zeros_like(denominator)
116    np.divide(2.0 * left * right, denominator, out=result, where=denominator > 0.0)
117    return result
118
119
120 def radial_geometry(radius_m: float, dr_m: float) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
121    node_count = int(round(radius_m / dr_m)) + 1
122    radius = np.linspace(0.0, radius_m, node_count)
123    left_faces = np.maximum(0.0, radius - dr_m / 2.0)
124    right_faces = np.minimum(radius_m, radius + dr_m / 2.0)
125    volumes = np.pi * (right_faces**2 - left_faces**2)
126    interface_areas = 2.0 * np.pi * (radius[:-1] + dr_m / 2.0)
127    return radius, volumes, interface_areas
128
129
130 def add_internal_fluxes(
131     derivative_numerator: np.ndarray,
132     values: np.ndarray,
133     interface_transport: np.ndarray,
134     interface_areas: np.ndarray,
135     dr_m: float,
136 ) -> None:
137     flux = interface_transport * interface_areas * (values[1:] - values[:-1]) / dr_m
138     derivative_numerator[:-1] += flux
139     derivative_numerator[1:] -= flux
140
141
142 def build_coupled_jacobian_sparsity(node_count: int) -> csc_matrix:
143     """Four nearest-neighbour sparse blocks for [T_0..T_N, C_0..C_N]."""
144
145     tri = diags(
146         [np.ones(node_count - 1), np.ones(node_count), np.ones(node_count - 1)],
147         offsets=[-1, 0, 1],
148         shape=(node_count, node_count),
149         format="csc",
150     )
151     return bmat([[tri, tri], [tri, tri]], format="csc")
152
153
154 def sample_to_output_grid(
155     internal_radius_cm: np.ndarray,
156     output_radius_cm: np.ndarray,
157     states_by_time: np.ndarray,

```

```

158 ) -> np.ndarray:
159     spacing_cm = internal_radius_cm[1] - internal_radius_cm[0]
160     indices = np rint(output_radius_cm / spacing_cm).astype(int)
161     if np.allclose(internal_radius_cm[indices], output_radius_cm):
162         return states_by_time[:, indices]
163     return np.vstack(
164         [np.interp(output_radius_cm, internal_radius_cm, row) for row in states_by_time]
165     )
166
167
168 def solve_problem2(
169     environment: EnvironmentSeries,
170     config: SolverConfig | None = None,
171 ) -> SimulationResult:
172     """Solve the nonlinear T/C system by conservative radial FVM and adaptive BDF."""
173
174     config = config or SolverConfig()
175     if environment.time_s[0] > 0.0 or environment.time_s[-1] < config.duration_s:
176         raise ValueError("environment data must cover the complete simulation interval")
177
178     internal_radius_cm = build_radial_grid_cm(config.radius_cm, config.internal_dr_cm)
179     output_radius_cm = build_radial_grid_cm(config.radius_cm, config.output_dr_cm)
180     dr_m = config.internal_dr_cm / 100.0
181     radius_m = config.radius_cm / 100.0
182     volumes, interface_areas = radial_geometry(radius_m, dr_m)
183     surface_area = 2.0 * np.pi * radius_m
184     node_count = len(internal_radius_cm)
185     jacobian_pattern = build_coupled_jacobian_sparsity(node_count)
186     output_time_s = np.arange(
187         0.0,
188         config.duration_s + config.output_dt_s / 2.0,
189         config.output_dt_s,
190     )
191
192     def coupled_rhs(t_s: float, state: np.ndarray) -> np.ndarray:
193         temperature = state[:node_count]
194         moisture = state[node_count:]
195         safe_moisture = np.maximum(moisture, 1e-9)
196
197         rho_cp = density(safe_moisture) * heat_capacity(safe_moisture)
198         node_k = thermal_conductivity(safe_moisture)
199         node_d = moisture_diffusivity(safe_moisture, temperature)
200         face_k = harmonic_mean(node_k[:-1], node_k[1:])
201         face_d = harmonic_mean(node_d[:-1], node_d[1:])
202
203         heat_numerator = np.zeros(node_count, dtype=float)
204         moisture_numerator = np.zeros(node_count, dtype=float)
205         add_internal_fluxes(heat_numerator, temperature, face_k, interface_areas, dr_m)
206         add_internal_fluxes(moisture_numerator, moisture, face_d, interface_areas, dr_m)
207
208         heat_numerator[-1] += (
209             config.heat_transfer_w_m2_k
210             * surface_area
211             * (environment.temperature_at(t_s) - temperature[-1])
212         )
213         moisture_numerator[-1] += (
214             config.mass_transfer_m_s
215             * surface_area
216             * (environment.moisture_at(t_s) - moisture[-1])
217         )
218
219         temperature_rate = heat_numerator / (rho_cp * volumes)
220         moisture_rate = moisture_numerator / volumes
221         return np.concatenate([temperature_rate, moisture_rate])
222
223     initial_state = np.concatenate(
224         [
225             np.full(node_count, config.initial_temperature_c),
226             np.full(node_count, config.initial_moisture),
227         ]
228     )
229     absolute_tolerance = np.concatenate(
230         [
231             np.full(node_count, config.temperature_absolute_tolerance),
232             np.full(node_count, config.moisture_absolute_tolerance),
233         ]
234     )
235     solution = solve_ivp(
236         coupled_rhs,

```

```

237         (0.0, float(config.duration_s)),
238         initial_state,
239         method="BDF",
240         t_eval=output_time_s,
241         rtol=config.relative_tolerance,
242         atol=absolute_tolerance,
243         max_step=config.max_time_step_s,
244         jac_sparsity=jacobian_pattern,
245     )
246     if not solution.success:
247         raise RuntimeError(f"coupled integration failed: {solution.message}")
248     if not np.isfinite(solution.y).all():
249         raise RuntimeError("coupled integration returned non-finite values")
250
251     internal_temperature = solution.y[:, node_count, :].T
252     internal_moisture = solution.y[:, node_count, :].T
253     if float(internal_moisture.min()) < -1e-8:
254         raise RuntimeError("coupled integration returned materially negative moisture")
255
256     temperature = sample_to_output_grid(internal_radius_cm, output_radius_cm, internal_temperature)
257     moisture = sample_to_output_grid(internal_radius_cm, output_radius_cm, internal_moisture)
258     diagnostics = SolverDiagnostics(
259         internal_node_count=node_count,
260         state_size=2 * node_count,
261         function_evaluations=int(solution.nfev),
262         jacobian_evaluations=int(solution.njev),
263         lu_decompositions=int(solution.nlu),
264         minimum_temperature_c=float(internal_temperature.min()),
265         minimum_moisture=float(internal_moisture.min()),
266     )
267     return SimulationResult(output_time_s, output_radius_cm, temperature, moisture, diagnostics)
268
269
270 # ===== 运行与产物（全部默认，无命令行参数）=====
271 BASE = Path(__file__).resolve().parent # 代码同根目录
272 ATTACHMENT1 = BASE / "附件" / "附件1.xlsx" # 环境数据（输入）
273 TEMPLATE = BASE / "附件" / "附件3" / "result2.xlsx" # 结果格式模板（题目附件3）
274 OUT_XLSX = BASE / "result2.xlsx" # 输出：同根目录
275 OUT_JSON = BASE / "verification2.json"
276 SAMPLE_SECONDS = np.array([1800, 3600, 5400, 7200, 9000, 10800], dtype=float)
277 SAMPLE_RADIUS_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0], dtype=float)
278 TEMP_SHEET = "温度"
279 MOIST_SHEET = "水分浓度"
280
281
282 def load_environment(path: Path) -> EnvironmentSeries:
283     frame = pd.read_excel(path)
284     required = ["时间", "温度", "水分浓度"]
285     missing = [name for name in required if name not in frame.columns]
286     if missing:
287         raise ValueError(f"附件1缺少列: {missing}")
288     return EnvironmentSeries(
289         time_s=frame["时间"].to_numpy(dtype=float),
290         temperature_c=frame["温度"].to_numpy(dtype=float),
291         moisture=frame["水分浓度"].to_numpy(dtype=float),
292     )
293
294
295 def selected_values(result: SimulationResult, values: np.ndarray) -> np.ndarray:
296     time_indices = [int(np.where(np.isclose(result.time_s, value))[0][0]) for value in SAMPLE_SECONDS]
297     radius_indices = [int(np.where(np.isclose(result.radius_cm, value))[0][0]) for value in SAMPLE_RADIUS_CM]
298     return values[np.ix_(time_indices, radius_indices)]
299
300
301 def difference_summary(left: np.ndarray, right: np.ndarray) -> dict[str, float]:
302     absolute = np.abs(left - right)
303     return {
304         "maximum_absolute_difference": float(absolute.max()),
305         "mean_absolute_difference": float(absolute.mean()),
306     }
307
308
309 def write_result_xlsx(result: SimulationResult, template: Path, out_path: Path) -> None:
310     """读附件3 格式模板 → 按完整时间×半径网格覆盖写温度/水分两表（全精度存储，四位小数显示）。"""
311
312     workbook = openpyxl.load_workbook(template)
313     radius = result.radius_cm
314     for sheet_name, field in ((TEMP_SHEET, result.temperature_c), (MOIST_SHEET, result.moisture)):
315         sheet = workbook[sheet_name]

```

```

316     sheet.cell(1, 1).value = "时间\\到药材中心的距离"
317     for j in range(radius.size):
318         sheet.cell(1, 2 + j).value = round(float(radius[j]), 10)
319     for i in range(1, result.time_s.size): # t=1..10800 (跳过 t=0 初值行)
320         sheet.cell(i + 1, 1).value = int(round(result.time_s[i]))
321         for j in range(radius.size):
322             cell = sheet.cell(i + 1, 2 + j)
323             cell.value = float(field[i, j]) # 全精度存储, 仅用 number_format 控制四位小数显示 (与原 result2.xlsx 一致)
324             cell.number_format = "0.0000"
325         sheet.freeze_panes = "B2"
326     out_path.parent.mkdir(parents=True, exist_ok=True)
327     workbook.save(out_path)
328
329
330 def main() -> None:
331     if not ATTACHMENT1.exists():
332         raise SystemExit(f"未找到附件: {ATTACHMENT1}\\n请把 附件1.xlsx 放到本脚本同根目录的 附件/ 下, 再运行: python
problem2.py")
333     if not TEMPLATE.exists():
334         raise SystemExit(f"未找到结果格式模板: {TEMPLATE}\\n请把题目附件3 的 result2.xlsx 放到 附件/附件3/ 下, 再运行: python
problem2.py")
335     environment = load_environment(ATTACHMENT1)
336
337     main_config = SolverConfig(internal_dr_cm=0.025)
338     fine_config = replace(main_config, internal_dr_cm=0.0125)
339     strict_time_config = replace(main_config, relative_tolerance=1e-9, temperature_absolute_tolerance=1e-10,
340                                 moisture_absolute_tolerance=1e-11, max_time_step_s=2.0)
341     early_ultrafine_config = replace(main_config, internal_dr_cm=0.00625, duration_s=60,
342                                     relative_tolerance=1e-9, temperature_absolute_tolerance=1e-10,
343                                     moisture_absolute_tolerance=1e-11, max_time_step_s=0.5)
344
345     print("求解 main grid dr=0.025 cm (主结果)...", flush=True)
346     main_result = solve_problem2(environment, main_config)
347     print("求解 fine grid dr=0.0125 cm...", flush=True)
348     fine_result = solve_problem2(environment, fine_config)
349     print("求解 stricter BDF time settings...", flush=True)
350     strict_time_result = solve_problem2(environment, strict_time_config)
351     print("求解 early-time ultrafine dr=0.00625 cm for 60 s...", flush=True)
352     early_ultrafine_result = solve_problem2(environment, early_ultrafine_config)
353
354     spatial = {
355         "note": "在论文表3/表4 的 30 个采样点上比较",
356         "temperature_c": difference_summary(selected_values(main_result, main_result.temperature_c),
357                                             selected_values(fine_result, fine_result.temperature_c)),
358     }
359     temporal = {
360         "note": "在论文表3/表4 的 30 个采样点上比较",
361         "temperature_c": difference_summary(selected_values(main_result, main_result.temperature_c),
362                                             selected_values(strict_time_result, strict_time_result.temperature_c)),
363         "moisture": difference_summary(selected_values(main_result, main_result.moisture),
364                                       selected_values(strict_time_result, strict_time_result.moisture)),
365     }
366     early = {
367         "comparison": "dr=0.0125 cm versus dr=0.00625 cm over 0-60 s",
368         "temperature_c": difference_summary(fine_result.temperature_c[:61], early_ultrafine_result.temperature_c),
369         "moisture": difference_summary(fine_result.moisture[:61], early_ultrafine_result.moisture),
370     }
371     physical_checks = {
372         "minimum_temperature_c": main_result.diagnostics.minimum_temperature_c,
373         "minimum_moisture": main_result.diagnostics.minimum_moisture,
374         "all_finite": bool(np.isfinite(main_result.temperature_c).all() and
375                             np.isfinite(main_result.moisture).all()),
376     }
377     verification = {
378         "source_attachment": str(ATTACHMENT1),
379         "main_config": asdict(main_config),
380         "diagnostics": {k: asdict(v) for k, v in
381                        ((("main", main_result.diagnostics), ("fine", fine_result.diagnostics),
382                          ("strict_time", strict_time_result.diagnostics), ("early_ultrafine",
383                          early_ultrafine_result.diagnostics))),
384         "spatial_refinement": spatial,
385         "time_solver_sensitivity": temporal,
386         "early_time_spatial_check_0_to_60_s": early,
387         "physical_checks": physical_checks,
388         "selected_tables": {
389             "temperature_c": [[float(t), *[float(v) for v in row]] for t, row in zip(SAMPLE_SECONDS,

```

```

selected_values(main_result, main_result.temperature_c)],
386     "moisture": [[float(t), *[float(v) for v in row]] for t, row in zip(SAMPLE_SECONDS,
selected_values(main_result, main_result.moisture))],
387 ],
388 }
389
390 write_result_xlsx(main_result, TEMPLATE, OUT_XLSX)
391 OUT_JSON.write_text(json.dumps(verification, ensure_ascii=False, indent=2), encoding="utf-8")
392
393 bar = "=" * 64
394 radii_header = " ".join(f"{r:>7.1f}" for r in SAMPLE_RADIUS_CM)
395 print(bar, flush=True)
396 print("问题二 3 h 变物性热湿耦合 — 温度/水分 (守恒径向 FVM + 联合状态自适应 BDF)", flush=True)
397 print(bar, flush=True)
398 print("[表3] 温度/℃ (行=时间/s, 列=距中心半径/cm)", flush=True)
399 print(f"{'t/s':>6} {radii_header}", flush=True)
400 for t, row in zip(SAMPLE_SECONDS, selected_values(main_result, main_result.temperature_c)):
401     print(f"{int(t):>6} " + " ".join(f"{v:>7.4f}" for v in row), flush=True)
402 print("[表4] 水分浓度/(kg/kg)", flush=True)
403 print(f"{'t/s':>6} {radii_header}", flush=True)
404 for t, row in zip(SAMPLE_SECONDS, selected_values(main_result, main_result.moisture)):
405     print(f"{int(t):>6} " + " ".join(f"{v:>7.4f}" for v in row), flush=True)
406 print("-" * 64, flush=True)
407 print(f"[网格加密] dr 0.025→0.0125 cm: 温度最大差={spatial['temperature_c']['maximum_absolute_difference']:.3e}
℃, "
408     f"水分最大差={spatial['moisture']['maximum_absolute_difference']:.3e} kg/kg", flush=True)
409 print(f"[BDF 容差收紧] rtol 2e-8→1e-9、max_step 5→2 s: 温度最大差={temporal['temperature_c']
['maximum_absolute_difference']:.3e}, "
410     f"水分最大差={temporal['moisture']['maximum_absolute_difference']:.3e} (四位小数下采样值不变)", flush=True)
411 print(f"[早期薄层] 0-60 s, dr 0.0125 vs 0.00625 cm: 水分最大差={early['moisture']
['maximum_absolute_difference']:.3e} kg/kg"
412     f" (近表面薄边界层, 正文表格自 0.5 h 起采样不受影响)", flush=True)
413 print(f"[物理核验] 温度 min={physical_checks['minimum_temperature_c']:.4f} °C, 水分 min=
{physical_checks['minimum_moisture']:.6f}, "
414     f"全有限={physical_checks['all_finite']}", flush=True)
415 print(bar, flush=True)
416 print(f"已写出: {OUT_XLSX} (温度/水分两表, 按附件3 模板格式)、{OUT_JSON}", flush=True)
417
418
419 if __name__ == "__main__":
420     main()
421

```

```

1 """问题三：固定半径、附录3变物性热湿耦合，守恒径向有限体积 + 联合状态自适应 BDF，求全截面达标停机的连续临界时间。
2 """
3 from __future__ import annotations
4
5 import json
6 import time
7 from dataclasses import asdict, dataclass, replace
8 from math import ceil
9 from pathlib import Path
10 from typing import Callable
11
12 import numpy as np
13 import openpyxl
14 import pandas as pd
15 from scipy.integrate import solve_ivp
16 from scipy.linalg.lapack import dgtsv
17 from scipy.sparse import bmat, csc_matrix, diags
18
19
20 # ===== 求解核心 · 物性与环境边界（附录3） =====
21 @dataclass(frozen=True)
22 class EnvironmentSeries:
23     """附件1给出的炉内边界序列；附件区间内只做分段线性插值。"""
24
25     time_s: np.ndarray
26     temperature_c: np.ndarray
27     moisture: np.ndarray
28
29     def __post_init__(self) -> None:
30         time = np.asarray(self.time_s, dtype=float)
31         temperature = np.asarray(self.temperature_c, dtype=float)
32         moisture = np.asarray(self.moisture, dtype=float)
33         if not (time.ndim == temperature.ndim == moisture.ndim == 1):
34             raise ValueError("environment arrays must be one-dimensional")
35         if not (len(time) == len(temperature) == len(moisture)) or len(time) < 2:
36             raise ValueError("environment arrays must have equal length >= 2")
37         if np.any(np.diff(time) <= 0.0):
38             raise ValueError("environment time must be strictly increasing")
39         if not (np.isfinite(time).all() and np.isfinite(temperature).all() and np.isfinite(moisture).all()):
40             raise ValueError("environment arrays must be finite")
41         object.__setattr__(self, "time_s", time)
42         object.__setattr__(self, "temperature_c", temperature)
43         object.__setattr__(self, "moisture", moisture)
44
45     @property
46     def terminal_time_s(self) -> float:
47         return float(self.time_s[-1])
48
49     @property
50     def terminal_temperature_c(self) -> float:
51         return float(self.temperature_c[-1])
52
53     @property
54     def terminal_moisture(self) -> float:
55         return float(self.moisture[-1])
56
57     def interpolated_temperature(self, t_s: float) -> float:
58         if t_s < self.time_s[0] or t_s > self.time_s[-1]:
59             raise ValueError("interpolation time is outside the attachment interval")
60         return float(np.interp(t_s, self.time_s, self.temperature_c))
61
62     def interpolated_moisture(self, t_s: float) -> float:
63         if t_s < self.time_s[0] or t_s > self.time_s[-1]:
64             raise ValueError("interpolation time is outside the attachment interval")
65         return float(np.interp(t_s, self.time_s, self.moisture))
66
67
68 @dataclass(frozen=True)
69 class PostBoundary:
70     temperature_c: float
71     moisture: float
72
73
74 def _safe_moisture(c: np.ndarray | float) -> np.ndarray:
75     """只供物性计算使用；不修改求解器中的状态。"""
76
77     return np.maximum(np.asarray(c, dtype=float), 1e-9)
78

```

```

79
80 def density(c: np.ndarray | float) -> np.ndarray:
81     safe_c = _safe_moisture(c)
82     return 650.0 + 128.0 * safe_c
83
84
85 def heat_capacity(c: np.ndarray | float) -> np.ndarray:
86     safe_c = _safe_moisture(c)
87     return 1450.0 + 2736.0 * safe_c / (safe_c + 1.0)
88
89
90 def thermal_conductivity(c: np.ndarray | float) -> np.ndarray:
91     safe_c = _safe_moisture(c)
92     return 0.21 + 0.38 * safe_c / (safe_c + 1.0)
93
94
95 def moisture_diffusivity(c: np.ndarray | float, temperature_c: np.ndarray | float) -> np.ndarray:
96     safe_c = _safe_moisture(c)
97     temperature_k = np.asarray(temperature_c, dtype=float) + 273.15
98     if np.any(temperature_k <= 0.0):
99         raise ValueError("temperature must be above absolute zero")
100     return 2.4e-3 * np.exp(-0.45 / safe_c) * np.exp(-3850.0 / temperature_k)
101
102
103 def post_boundary_from_tail(environment: EnvironmentSeries, averaging_window_s: float | None = None) ->
    PostBoundary:
104     if averaging_window_s is None:
105         return PostBoundary(environment.terminal_temperature_c, environment.terminal_moisture)
106     if averaging_window_s <= 0.0:
107         raise ValueError("averaging_window_s must be positive")
108     start = max(float(environment.time_s[0]), environment.terminal_time_s - averaging_window_s)
109     interior = environment.time_s[(environment.time_s > start) & (environment.time_s <=
        environment.terminal_time_s)]
110     times = np.concatenate([[start], interior])
111     temperature = np.interp(times, environment.time_s, environment.temperature_c)
112     moisture = np.interp(times, environment.time_s, environment.moisture)
113     duration = float(times[-1] - times[0])
114     if duration <= 0.0:
115         raise ValueError("averaging window has zero duration")
116     temperature_integral = float(np.sum(0.5 * (temperature[:-1] + temperature[1:]) * np.diff(times)))
117     moisture_integral = float(np.sum(0.5 * (moisture[:-1] + moisture[1:]) * np.diff(times)))
118     return PostBoundary(
119         temperature_integral / duration,
120         moisture_integral / duration,
121     )
122
123
124 # ===== 求解核心 · 径向 FVM 几何与界面离散 =====
125 @dataclass(frozen=True)
126 class RadialGeometry:
127     radius_m: np.ndarray
128     volumes_m3_per_m: np.ndarray
129     interface_areas_m2_per_m: np.ndarray
130     surface_area_m2_per_m: float
131     dr_m: float
132
133     @property
134     def node_count(self) -> int:
135         return int(self.radius_m.size)
136
137
138 def build_radial_geometry(radius_cm: float, dr_cm: float) -> RadialGeometry:
139     if radius_cm <= 0.0 or dr_cm <= 0.0:
140         raise ValueError("radius_cm and dr_cm must be positive")
141     intervals = int(round(radius_cm / dr_cm))
142     if intervals < 1 or not np.isclose(intervals * dr_cm, radius_cm, rtol=0.0, atol=1e-12):
143         raise ValueError("radius_cm must be an integer multiple of dr_cm")
144     radius_m = np.linspace(0.0, radius_cm / 100.0, intervals + 1)
145     dr_m = dr_cm / 100.0
146     left_faces = np.maximum(0.0, radius_m - dr_m / 2.0)
147     right_faces = np.minimum(radius_m[-1], radius_m + dr_m / 2.0)
148     volumes = np.pi * (right_faces**2 - left_faces**2)
149     interface_areas = 2.0 * np.pi * (radius_m[:-1] + dr_m / 2.0)
150     return RadialGeometry(
151         radius_m=radius_m,
152         volumes_m3_per_m=volumes,
153         interface_areas_m2_per_m=interface_areas,
154         surface_area_m2_per_m=float(2.0 * np.pi * radius_m[-1]),
155         dr_m=dr_m,

```

```

156     )
157
158
159 def harmonic_mean(left: np.ndarray, right: np.ndarray) -> np.ndarray:
160     left = np.asarray(left, dtype=float)
161     right = np.asarray(right, dtype=float)
162     denominator = left + right
163     result = np.zeros(np.broadcast_shapes(left.shape, right.shape), dtype=float)
164     np.divide(2.0 * left * right, denominator, out=result, where=denominator != 0.0)
165     return result
166
167
168 def interface_values(node_values: np.ndarray, mean: str) -> np.ndarray:
169     node_values = np.asarray(node_values, dtype=float)
170     if mean == "harmonic":
171         return harmonic_mean(node_values[:-1], node_values[1:])
172     if mean == "arithmetic":
173         return 0.5 * (node_values[:-1] + node_values[1:])
174     raise ValueError("interface_mean must be 'harmonic' or 'arithmetic'")
175
176
177 def internal_flux_numerator(
178     values: np.ndarray,
179     interface_transport: np.ndarray,
180     geometry: RadialGeometry,
181 ) -> np.ndarray:
182     values = np.asarray(values, dtype=float)
183     numerator = np.zeros_like(values)
184     flux = (
185         np.asarray(interface_transport, dtype=float)
186         * geometry.interface_areas_m2_per_m
187         * (values[1:] - values[:-1])
188         / geometry.dr_m
189     )
190     numerator[:-1] += flux
191     numerator[1:] -= flux
192     return numerator
193
194
195 def build_coupled_jacobian_sparsity(node_count: int) -> csc_matrix:
196     if node_count < 2:
197         raise ValueError("node_count must be at least two")
198     tri = diags(
199         [np.ones(node_count - 1), np.ones(node_count), np.ones(node_count - 1)],
200         offsets=[-1, 0, 1],
201         shape=(node_count, node_count),
202         format="csc",
203     )
204     return bmat([[tri, tri], [tri, tri]], format="csc")
205
206
207 def sample_profiles(
208     internal_radius_cm: np.ndarray,
209     output_radius_cm: np.ndarray,
210     profiles: np.ndarray,
211 ) -> np.ndarray:
212     internal_radius_cm = np.asarray(internal_radius_cm, dtype=float)
213     output_radius_cm = np.asarray(output_radius_cm, dtype=float)
214     profiles = np.asarray(profiles, dtype=float)
215     indices = np rint(output_radius_cm / (internal_radius_cm[1] - internal_radius_cm[0])).astype(int)
216     if np.allclose(internal_radius_cm[indices], output_radius_cm, rtol=0.0, atol=1e-12):
217         return profiles[:, indices]
218     return np.vstack([np.interp(output_radius_cm, internal_radius_cm, row) for row in profiles])
219
220
221
222 # ===== 求解核心 · 联合状态自适应 BDF 求解器（主求解器） =====
223 @dataclass(frozen=True)
224 class SolverConfig:
225     radius_cm: float = 2.0
226     internal_dr_cm: float = 0.025
227     output_dr_cm: float = 0.1
228     output_intervals: int = 60
229     initial_temperature_c: float = 28.0
230     initial_moisture: float = 2.55
231     completion_threshold: float = 0.15
232     heat_transfer_w_m2_k: float = 25.0
233     mass_transfer_m_s: float = 8e-7
234     interface_mean: str = "harmonic"

```

```

235     relative_tolerance: float = 1e-8
236     temperature_absolute_tolerance: float = 1e-8
237     moisture_absolute_tolerance: float = 1e-10
238     max_time_step_s: float = 60.0
239     initial_time_limit_s: float = 72.0 * 3600.0
240     fallback_time_limit_s: float = 96.0 * 3600.0
241     extended_time_limit_s: float | None = None
242     material_negative_tolerance: float = -1e-8
243     monotonicity_tolerance: float = 1e-8
244
245
246 @dataclass(frozen=True)
247 class SolverDiagnostics:
248     internal_node_count: int
249     state_size: int
250     function_evaluations: int
251     jacobian_evaluations: int
252     lu_decompositions: int
253     phase1_end_time_s: float
254     first_limit_reached_without_event: bool
255     fallback_limit_reached_without_event: bool
256     event_count: int
257     event_direction: int
258     minimum_temperature_c: float
259     maximum_temperature_c: float
260     minimum_raw_moisture: float
261     clipped_cell_count: int
262     maximum_clip_magnitude: float
263     radial_monotonicity_all_outputs: bool
264     non_center_argmax_count: int
265     maximum_normalized_mass_balance_residual: float
266
267
268 @dataclass(frozen=True)
269 class SimulationResult:
270     time_s: np.ndarray
271     radius_cm: np.ndarray
272     temperature_c: np.ndarray
273     moisture: np.ndarray
274     argmax_radius_cm: np.ndarray
275     max_moisture: np.ndarray
276     center_moisture: np.ndarray
277     radial_monotonicity: np.ndarray
278     drying_event_time_s: float
279     report_end_time_s: int
280     event_temperature_c: np.ndarray
281     event_moisture: np.ndarray
282     event_internal_radius_cm: np.ndarray
283     event_internal_moisture: np.ndarray
284     diagnostics: SolverDiagnostics
285
286
287 def drying_event_value(state: np.ndarray, node_count: int, threshold: float) -> float:
288     return float(np.max(state[node_count:]) - threshold)
289
290
291 def _boundary_values(
292     t_s: float,
293     environment: EnvironmentSeries,
294     post_boundary: PostBoundary,
295 ) -> tuple[float, float]:
296     if t_s <= environment.terminal_time_s:
297         return environment.interpolated_temperature(t_s), environment.interpolated_moisture(t_s)
298     return post_boundary.temperature_c, post_boundary.moisture
299
300
301 def build_coupled_rhs(
302     geometry: RadialGeometry,
303     environment: EnvironmentSeries,
304     post_boundary: PostBoundary,
305     config: SolverConfig,
306 ) -> Callable[[float, np.ndarray], np.ndarray]:
307     node_count = geometry.node_count
308
309     def rhs(t_s: float, state: np.ndarray) -> np.ndarray:
310         temperature = state[:node_count]
311         moisture = state[node_count:]
312         node_k = thermal_conductivity(moisture)
313         node_d = moisture_diffusivity(moisture, temperature)

```

```

314     face_k = interface_values(node_k, config.interface_mean)
315     face_d = interface_values(node_d, config.interface_mean)
316     heat_numerator = internal_flux_numerator(temperature, face_k, geometry)
317     moisture_numerator = internal_flux_numerator(moisture, face_d, geometry)
318     external_temperature, external_moisture = _boundary_values(t_s, environment, post_boundary)
319     heat_numerator[-1] += (
320         config.heat_transfer_w_m2_k
321         * geometry.surface_area_m2_per_m
322         * (external_temperature - temperature[-1])
323     )
324     moisture_numerator[-1] += (
325         config.mass_transfer_m_s
326         * geometry.surface_area_m2_per_m
327         * (external_moisture - moisture[-1])
328     )
329     rho_cp = density(moisture) * heat_capacity(moisture)
330     temperature_rate = heat_numerator / (rho_cp * geometry.volumes_m3_per_m)
331     moisture_rate = moisture_numerator / geometry.volumes_m3_per_m
332     return np.concatenate([temperature_rate, moisture_rate])
333
334     return rhs
335
336
337 def _evaluate_segments(segments: list, times: np.ndarray) -> np.ndarray:
338     if times.size == 0:
339         raise ValueError("times must not be empty")
340     state_count = int(segments[0].y.shape[0])
341     evaluated = np.empty((state_count, times.size), dtype=float)
342     assigned = np.zeros(times.size, dtype=bool)
343     for segment in segments:
344         mask = (~assigned) & (times >= segment.t[0] - 1e-8) & (times <= segment.t[-1] + 1e-8)
345         if np.any(mask):
346             evaluated[:, mask] = segment.sol(times[mask])
347             assigned[mask] = True
348     if not assigned.all():
349         raise RuntimeError(f"no dense-output segment covers {times[~assigned]}")
350     return evaluated
351
352
353 def _normalized_mass_balance_residual(
354     t_s: float,
355     state: np.ndarray,
356     rhs: Callable[[float, np.ndarray], np.ndarray],
357     geometry: RadialGeometry,
358     environment: EnvironmentSeries,
359     post_boundary: PostBoundary,
360     config: SolverConfig,
361 ) -> float:
362     n = geometry.node_count
363     moisture = state[n:]
364     rate = rhs(t_s, state)[n:]
365     _, external_moisture = _boundary_values(t_s, environment, post_boundary)
366     surface_flux = (
367         config.mass_transfer_m_s
368         * geometry.surface_area_m2_per_m
369         * (external_moisture - moisture[-1])
370     )
371     residual = float(np.sum(rate * geometry.volumes_m3_per_m) - surface_flux)
372     scale = float(np.sum(np.abs(rate * geometry.volumes_m3_per_m)) + abs(surface_flux) + 1e-30)
373     return abs(residual) / scale
374
375
376 def solve_problem3_bdf(
377     environment: EnvironmentSeries,
378     config: SolverConfig | None = None,
379     post_boundary: PostBoundary | None = None,
380 ) -> SimulationResult:
381     """两阶段积分耦合状态 [T_0..T_N, C_0..C_N]，以全域最大水分触发终止事件。"""
382
383     config = config or SolverConfig()
384     post_boundary = post_boundary or post_boundary_from_tail(environment)
385     if environment.time_s[0] > 0.0:
386         raise ValueError("environment must start at or before t=0")
387     if config.initial_time_limit_s <= environment.terminal_time_s:
388         raise ValueError("initial_time_limit_s must exceed the attachment terminal time")
389     if config.fallback_time_limit_s <= config.initial_time_limit_s:
390         raise ValueError("fallback_time_limit_s must exceed initial_time_limit_s")
391     if config.extended_time_limit_s is not None and config.extended_time_limit_s <= config.fallback_time_limit_s:
392         raise ValueError("extended_time_limit_s must exceed fallback_time_limit_s")

```

```

393     if config.output_interval_s <= 0:
394         raise ValueError("output_interval_s must be positive")
395
396     geometry = build_radial_geometry(config.radius_cm, config.internal_dr_cm)
397     internal_radius_m = geometry.radius_m * 100.0
398     output_geometry = build_radial_geometry(config.radius_cm, config.output_dr_cm)
399     output_radius_cm = output_geometry.radius_m * 100.0
400     node_count = geometry.node_count
401     jacobian_pattern = build_coupled_jacobian_sparsity(node_count)
402     rhs = build_coupled_rhs(geometry, environment, post_boundary, config)
403     initial_state = np.concatenate(
404     [
405         np.full(node_count, config.initial_temperature_c, dtype=float),
406         np.full(node_count, config.initial_moisture, dtype=float),
407     ]
408     )
409     atol = np.concatenate(
410     [
411         np.full(node_count, config.temperature_absolute_tolerance),
412         np.full(node_count, config.moisture_absolute_tolerance),
413     ]
414     )
415
416     def integrate(start: float, end: float, state: np.ndarray, with_event: bool):
417         events = None
418         if with_event:
419             def event(t_s: float, y: np.ndarray) -> float:
420                 del t_s
421                 return drying_event_value(y, node_count, config.completion_threshold)
422
423             event.terminal = True
424             event.direction = -1
425             events = event
426         solution = solve_ivp(
427             rhs,
428             (float(start), float(end)),
429             state,
430             method="BDF",
431             events=events,
432             dense_output=True,
433             rtol=config.relative_tolerance,
434             atol=atol,
435             max_step=config.max_time_step_s,
436             jac_sparsity=jacobian_pattern,
437         )
438         if not solution.success:
439             raise RuntimeError(f"BDF integration failed: {solution.message}")
440         if not np.isfinite(solution.y).all():
441             raise RuntimeError("BDF integration returned non-finite values")
442         return solution
443
444     segments = []
445     phase1 = integrate(0.0, environment.terminal_time_s, initial_state, False)
446     segments.append(phase1)
447     phase1_state = phase1.y[:, -1]
448     if drying_event_value(phase1_state, node_count, config.completion_threshold) <= 0.0:
449         raise RuntimeError("drying threshold was reached before the attachment boundary ended")
450
451     phase2 = integrate(environment.terminal_time_s, config.initial_time_limit_s, phase1_state, True)
452     segments.append(phase2)
453     first_limit_reached = len(phase2.t_events[0]) == 0
454     fallback_limit_reached = False
455     event_time = None
456     event_state = None
457     if not first_limit_reached:
458         event_time = float(phase2.t_events[0][0])
459         event_state = phase2.y_events[0][0]
460     else:
461         phase3 = integrate(config.initial_time_limit_s, config.fallback_time_limit_s, phase2.y[:, -1], True)
462         segments.append(phase3)
463         if len(phase3.t_events[0]) == 0:
464             fallback_limit_reached = True
465             if config.extended_time_limit_s is None:
466                 terminal_max = float(np.max(phase3.y[node_count:, -1]))
467                 terminal_argmax = int(np.argmax(phase3.y[node_count:, -1]))
468                 raise RuntimeError(
469                     f"drying threshold was not reached by {config.fallback_time_limit_s / 3600.0:.1f} h; "
470                     f"max moisture={terminal_max:.9g} at r={internal_radius_cm[terminal_argmax]:.9g} cm"
471                 )

```

```

472         phase4 = integrate(
473             config.fallback_time_limit_s,
474             config.extended_time_limit_s,
475             phase3.y[:, -1],
476             True,
477         )
478         segments.append(phase4)
479         if len(phase4.t_events[0]) == 0:
480             terminal_max = float(np.max(phase4.y[node_count:, -1]))
481             terminal_argmax = int(np.argmax(phase4.y[node_count:, -1]))
482             raise RuntimeError(
483                 f"drying threshold was not reached by {config.extended_time_limit_s / 3600.0:.1f} h; "
484                 f"max moisture={terminal_max:.9g} at r={internal_radius_cm[terminal_argmax]:.9g} cm"
485             )
486             event_time = float(phase4.t_events[0][0])
487             event_state = phase4.y_events[0][0]
488         else:
489             event_time = float(phase3.t_events[0][0])
490             event_state = phase3.y_events[0][0]
491
492     assert event_time is not None and event_state is not None
493     report_end = int(config.output_interval_s * ceil(event_time / config.output_interval_s))
494     if report_end <= event_time + 1e-9:
495         report_end += config.output_interval_s
496     extension = integrate(event_time, float(report_end), event_state, False)
497     while drying_event_value(extension.y[:, -1], node_count, config.completion_threshold) >= 0.0:
498         report_end += config.output_interval_s
499         extension = integrate(event_time, float(report_end), event_state, False)
500     segments.append(extension)
501
502     output_times = np.arange(config.output_interval_s, report_end + 1, config.output_interval_s, dtype=float)
503     internal_states = _evaluate_segments(segments, output_times)
504     internal_temperature = internal_states[:node_count, :].T
505     internal_moisture = internal_states[node_count:, :].T
506     minimum_raw_moisture = float(min(np.min(internal_moisture), np.min(event_state[node_count:])))
507     if minimum_raw_moisture < config.material_negative_tolerance:
508         raise RuntimeError(f"materially negative moisture returned: {minimum_raw_moisture:.6g}")
509
510     output_temperature = sample_profiles(internal_radius_cm, output_radius_cm, internal_temperature)
511     output_moisture = sample_profiles(internal_radius_cm, output_radius_cm, internal_moisture)
512     event_temperature = sample_profiles(
513         internal_radius_cm,
514         output_radius_cm,
515         event_state[:node_count][None, :],
516     )[0]
517     event_moisture = sample_profiles(
518         internal_radius_cm,
519         output_radius_cm,
520         event_state[node_count:][None, :],
521     )[0]
522     argmax_indices = np.argmax(internal_moisture, axis=1)
523     argmax_radius = internal_radius_cm[argmax_indices]
524     max_moisture = np.max(internal_moisture, axis=1)
525     monotonicity = np.all(np.diff(internal_moisture, axis=1) <= config.monotonicity_tolerance, axis=1)
526     mass_residuals = np.array(
527         [
528             _normalized_mass_balance_residual(
529                 float(t_s), internal_states[:, index], rhs, geometry, environment, post_boundary, config
530             )
531             for index, t_s in enumerate(output_times)
532         ]
533     )
534     all_solutions = segments
535     diagnostics = SolverDiagnostics(
536         internal_node_count=node_count,
537         state_size=2 * node_count,
538         function_evaluations=int(sum(solution.nfev for solution in all_solutions)),
539         jacobian_evaluations=int(sum(solution.njev for solution in all_solutions)),
540         lu_decompositions=int(sum(solution.nlu for solution in all_solutions)),
541         phase1_end_time_s=environment.terminal_time_s,
542         first_limit_reached_without_event=first_limit_reached,
543         fallback_limit_reached_without_event=fallback_limit_reached,
544         event_count=1,
545         event_direction=-1,
546         minimum_temperature_c=float(min(np.min(internal_temperature), np.min(event_state[:node_count]))),
547         maximum_temperature_c=float(max(np.max(internal_temperature), np.max(event_state[:node_count]))),
548         minimum_raw_moisture=minimum_raw_moisture,
549         clipped_cell_count=0,
550         maximum_clip_magnitude=0.0,

```

```

551     radial_monotonicity_all_outputs=bool(monotonicity.all()),
552     non_center_argmax_count=int(np.count_nonzero(argmax_indices)),
553     maximum_normalized_mass_balance_residual=float(np.max(mass_residuals)),
554 )
555 return SimulationResult(
556     time_s=output_times,
557     radius_cm=output_radius_cm,
558     temperature_c=output_temperature,
559     moisture=output_moisture,
560     argmax_radius_cm=argmax_radius,
561     max_moisture=max_moisture,
562     center_moisture=internal_moisture[:, 0],
563     radial_monotonicity=monotonicity,
564     drying_event_time_s=event_time,
565     report_end_time_s=report_end,
566     event_temperature_c=event_temperature,
567     event_moisture=event_moisture,
568     event_internal_radius_cm=internal_radius_cm,
569     event_internal_moisture=event_state[node_count:].copy(),
570     diagnostics=diagnostics,
571 )
572
573
574
575 # ===== 求解核心 · 后向欧拉-Picard 对照求解器 =====
576 @dataclass(frozen=True)
577 class BESolverConfig:
578     radius_cm: float = 2.0
579     internal_dr_cm: float = 0.025
580     output_dr_cm: float = 0.1
581     time_step_s: float = 10.0
582     output_interval_s: int = 60
583     initial_temperature_c: float = 28.0
584     initial_moisture: float = 2.55
585     completion_threshold: float = 0.15
586     heat_transfer_w_m2_k: float = 25.0
587     mass_transfer_m_s: float = 8e-7
588     interface_mean: str = "harmonic"
589     maximum_picard_iterations: int = 20
590     temperature_picard_tolerance: float = 1e-8
591     moisture_picard_tolerance: float = 1e-10
592     initial_time_limit_s: float = 72.0 * 3600.0
593     fallback_time_limit_s: float = 96.0 * 3600.0
594     event_bracket_tolerance_s: float = 0.5
595     material_negative_tolerance: float = -1e-8
596
597
598 @dataclass(frozen=True)
599 class BEDiagnostics:
600     accepted_steps: int
601     bisection_steps: int
602     maximum_picard_iterations_used: int
603     mean_picard_iterations: float
604     unconverged_steps: int
605     maximum_final_temperature_iteration_error: float
606     maximum_final_moisture_iteration_error: float
607     minimum_raw_moisture: float
608
609
610 @dataclass(frozen=True)
611 class BEResult:
612     time_s: np.ndarray
613     radius_cm: np.ndarray
614     temperature_c: np.ndarray
615     moisture: np.ndarray
616     drying_event_time_s: float
617     event_temperature_c: np.ndarray
618     event_moisture: np.ndarray
619     event_internal_radius_cm: np.ndarray
620     event_internal_moisture: np.ndarray
621     diagnostics: BEDiagnostics
622
623
624 def _solve_tridiagonal(
625     lower: np.ndarray,
626     diagonal: np.ndarray,
627     upper: np.ndarray,
628     right_hand_side: np.ndarray,
629 ) -> np.ndarray:

```

```

630     n = diagonal.size
631     if n < 2:
632         raise ValueError("tridiagonal system must have at least two rows")
633     _, _, _, result, info = dgtsv(
634         lower.copy(),
635         diagonal.copy(),
636         upper.copy(),
637         right_hand_side.copy(),
638         overwrite_dl=True,
639         overwrite_d=True,
640         overwrite_du=True,
641         overwrite_b=True,
642     )
643     if info != 0:
644         raise RuntimeError(f"compiled TDMA failed with LAPACK info={info}")
645     return np.asarray(result, dtype=float)
646
647
648 def _implicit_fvm_step(
649     previous: np.ndarray,
650     node_transport: np.ndarray,
651     capacity: np.ndarray,
652     dt_s: float,
653     boundary_transfer: float,
654     boundary_external: float,
655     geometry: RadialGeometry,
656     interface_mean: str,
657 ) -> np.ndarray:
658     face_transport = interface_values(node_transport, interface_mean)
659     conductance = face_transport * geometry.interface_areas_m2_per_m / geometry.dr_m
660     accumulation = capacity * geometry.volumes_m3_per_m / dt_s
661     lower = -conductance.copy()
662     upper = -conductance.copy()
663     diagonal = accumulation.copy()
664     diagonal[:-1] += conductance
665     diagonal[1:] += conductance
666     right_hand_side = accumulation * previous
667     boundary_conductance = boundary_transfer * geometry.surface_area_m2_per_m
668     diagonal[-1] += boundary_conductance
669     right_hand_side[-1] += boundary_conductance * boundary_external
670     return _solve_tridiagonal(lower, diagonal, upper, right_hand_side)
671
672
673 def _boundary_at(t_s: float, environment: EnvironmentSeries, post_boundary: PostBoundary) -> tuple[float, float]:
674     if t_s <= environment.terminal_time_s:
675         return environment.interpolated_temperature(t_s), environment.interpolated_moisture(t_s)
676     return post_boundary.temperature_c, post_boundary.moisture
677
678
679 def coupled_backward_euler_step(
680     previous_temperature: np.ndarray,
681     previous_moisture: np.ndarray,
682     t_next_s: float,
683     dt_s: float,
684     geometry: RadialGeometry,
685     environment: EnvironmentSeries,
686     post_boundary: PostBoundary,
687     config: BESolverConfig,
688 ) -> tuple[np.ndarray, np.ndarray, int, float, float]:
689     if dt_s <= 0.0:
690         raise ValueError("dt_s must be positive")
691     external_temperature, external_moisture = _boundary_at(t_next_s, environment, post_boundary)
692     temperature_guess = previous_temperature.copy()
693     moisture_guess = previous_moisture.copy()
694     last_temperature_error = np.inf
695     last_moisture_error = np.inf
696     for iteration in range(1, config.maximum_picard_iterations + 1):
697         rho_cp = density(moisture_guess) * heat_capacity(moisture_guess)
698         next_temperature = _implicit_fvm_step(
699             previous_temperature,
700             thermal_conductivity(moisture_guess),
701             rho_cp,
702             dt_s,
703             config.heat_transfer_w_m2_k,
704             external_temperature,
705             geometry,
706             config.interface_mean,
707         )
708         next_moisture = _implicit_fvm_step(

```

```

709         previous_moisture,
710         moisture_diffusivity(moisture_guess, next_temperature),
711         np.ones_like(moisture_guess),
712         dt_s,
713         config.mass_transfer_m_s,
714         external_moisture,
715         geometry,
716         config.interface_mean,
717     )
718     last_temperature_error = float(np.max(np.abs(next_temperature - temperature_guess)))
719     last_moisture_error = float(np.max(np.abs(next_moisture - moisture_guess)))
720     if (
721         last_temperature_error <= config.temperature_picard_tolerance
722         and last_moisture_error <= config.moisture_picard_tolerance
723     ):
724         return next_temperature, next_moisture, iteration, last_temperature_error, last_moisture_error
725     temperature_guess = next_temperature
726     moisture_guess = next_moisture
727     raise RuntimeError(
728         "Picard iteration did not converge: "
729         f"t={t_next_s:.6f}s, dt={dt_s:.6f}s, "
730         f"temperature_error={last_temperature_error:.3e}, moisture_error={last_moisture_error:.3e}"
731     )
732
733
734 def solve_problem3_be(
735     environment: EnvironmentSeries,
736     config: BESolverConfig | None = None,
737     post_boundary: PostBoundary | None = None,
738 ) -> BEResult:
739     config = config or BESolverConfig()
740     post_boundary = post_boundary or post_boundary_from_tail(environment)
741     if environment.time_s[0] > 0.0:
742         raise ValueError("environment must start at or before t=0")
743     geometry = build_radial_geometry(config.radius_cm, config.internal_dr_cm)
744     internal_radius_cm = geometry.radius_m * 100.0
745     output_radius_cm = build_radial_geometry(config.radius_cm, config.output_dr_cm).radius_m * 100.0
746     temperature = np.full(geometry.node_count, config.initial_temperature_c, dtype=float)
747     moisture = np.full(geometry.node_count, config.initial_moisture, dtype=float)
748     time_s = 0.0
749     output_times: list[float] = []
750     output_temperature: list[np.ndarray] = []
751     output_moisture: list[np.ndarray] = []
752     iteration_counts: list[int] = []
753     final_temperature_errors: list[float] = []
754     final_moisture_errors: list[float] = []
755     minimum_raw_moisture = float(np.min(moisture))
756     bisection_steps = 0
757     event_time = None
758     event_temperature = None
759     event_moisture = None
760
761     while time_s < config.fallback_time_limit_s - 1e-12:
762         step_end = min(time_s + config.time_step_s, config.fallback_time_limit_s)
763         if time_s < environment.terminal_time_s < step_end:
764             step_end = environment.terminal_time_s
765         next_output = config.output_interval_s * (np.floor(time_s / config.output_interval_s) + 1.0)
766         if time_s + 1e-10 < next_output < step_end - 1e-10:
767             step_end = float(next_output)
768         dt_s = step_end - time_s
769         next_temperature, next_moisture, iterations, error_t, error_c = coupled_backward_euler_step(
770             temperature,
771             moisture,
772             step_end,
773             dt_s,
774             geometry,
775             environment,
776             post_boundary,
777             config,
778         )
779         iteration_counts.append(iterations)
780         final_temperature_errors.append(error_t)
781         final_moisture_errors.append(error_c)
782         minimum_raw_moisture = min(minimum_raw_moisture, float(np.min(next_moisture)))
783         if minimum_raw_moisture < config.material_negative_tolerance:
784             raise RuntimeError(f"materially negative moisture returned: {minimum_raw_moisture:.6g}")
785
786         previous_event_value = float(np.max(moisture)) - config.completion_threshold
787         next_event_value = float(np.max(next_moisture)) - config.completion_threshold

```

```

788     if previous_event_value > 0.0 and next_event_value <= 0.0:
789         left_time = time_s
790         left_temperature = temperature
791         left_moisture = moisture
792         right_time = step_end
793         right_temperature = next_temperature
794         right_moisture = next_moisture
795         while right_time - left_time > config.event_bracket_tolerance_s:
796             middle_time = 0.5 * (left_time + right_time)
797             middle_temperature, middle_moisture, _, _, _ = coupled_backward_euler_step(
798                 left_temperature,
799                 left_moisture,
800                 middle_time,
801                 middle_time - left_time,
802                 geometry,
803                 environment,
804                 post_boundary,
805                 config,
806             )
807             bisection_steps += 1
808             if float(np.max(middle_moisture) - config.completion_threshold) <= 0.0:
809                 right_time = middle_time
810                 right_temperature = middle_temperature
811                 right_moisture = middle_moisture
812             else:
813                 left_time = middle_time
814                 left_temperature = middle_temperature
815                 left_moisture = middle_moisture
816         event_time = right_time
817         event_temperature = right_temperature
818         event_moisture = right_moisture
819         break
820
821     time_s = step_end
822     temperature = next_temperature
823     moisture = next_moisture
824     if np.isclose(time_s % config.output_interval_s, 0.0, atol=1e-8):
825         output_times.append(time_s)
826         output_temperature.append(temperature.copy())
827         output_moisture.append(moisture.copy())
828
829     if event_time is None or event_temperature is None or event_moisture is None:
830         raise RuntimeError(
831             f"drying threshold was not reached by {config.fallback_time_limit_s / 3600.0:.1f} h"
832         )
833     if not output_times:
834         raise RuntimeError("no regular output was produced")
835
836     output_temperature_internal = np.vstack(output_temperature)
837     output_moisture_internal = np.vstack(output_moisture)
838     diagnostics = BEDiagnostics(
839         accepted_steps=len(iteration_counts),
840         bisection_steps=bisection_steps,
841         maximum_picard_iterations_used=int(max(iteration_counts)),
842         mean_picard_iterations=float(np.mean(iteration_counts)),
843         unconverged_steps=0,
844         maximum_final_temperature_iteration_error=float(max(final_temperature_errors)),
845         maximum_final_moisture_iteration_error=float(max(final_moisture_errors)),
846         minimum_raw_moisture=minimum_raw_moisture,
847     )
848     return BEResult(
849         time_s=np.asarray(output_times),
850         radius_cm=output_radius_cm,
851         temperature_c=sample_profiles(internal_radius_cm, output_radius_cm, output_temperature_internal),
852         moisture=sample_profiles(internal_radius_cm, output_radius_cm, output_moisture_internal),
853         drying_event_time_s=float(event_time),
854         event_temperature_c=sample_profiles(
855             internal_radius_cm, output_radius_cm, event_temperature[None, :]
856         )[0],
857         event_moisture=sample_profiles(
858             internal_radius_cm, output_radius_cm, event_moisture[None, :]
859         )[0],
860         event_internal_radius_cm=internal_radius_cm,
861         event_internal_moisture=event_moisture.copy(),
862         diagnostics=diagnostics,
863     )
864
865
866

```

```

867 # ===== 运行与产物（全部默认，无命令行参数） =====
868 BASE = Path(__file__).resolve().parent # 代码同根目录
869 ATTACHMENT1 = BASE / "附件" / "附件1.xlsx" # 环境数据（输入）
870 TEMPLATE = BASE / "附件" / "附件3" / "result3.xlsx" # 结果格式模板（题目附件3，单 sheet）
871 OUT_XLSX = BASE / "result3.xlsx" # 输出：同根目录
872 OUT_JSON = BASE / "verification3.json"
873 PAPER_RADII_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0])
874
875
876 def load_environment(path: Path) -> EnvironmentSeries:
877     """pandas 读附件1 三列（时间/温度/水分浓度） -> EnvironmentSeries. """
878
879     frame = pd.read_excel(path)
880     required = ["时间", "温度", "水分浓度"]
881     missing = [name for name in required if name not in frame.columns]
882     if missing:
883         raise ValueError(f"附件1缺少列: {missing}")
884     return EnvironmentSeries(
885         frame["时间"].to_numpy(float),
886         frame["温度"].to_numpy(float),
887         frame["水分浓度"].to_numpy(float),
888     )
889
890
891 def values_at_times(result: SimulationResult | BEResult, times_s: np.ndarray) -> np.ndarray:
892     indices = []
893     for value in times_s:
894         matches = np.where(np.isclose(result.time_s, value, rtol=0.0, atol=1e-8))[0]
895         if not len(matches):
896             raise ValueError(f"result has no regular output at t={value}")
897         indices.append(int(matches[0]))
898     radius_indices = [int(np.where(np.isclose(result.radius_cm, radius))[0][0]) for radius in PAPER_RADII_CM]
899     return result.moisture[np.ix_(indices, radius_indices)]
900
901
902 def common_paper_times(*results: SimulationResult | BEResult) -> np.ndarray:
903     latest_common = min(float(result.drying_event_time_s) for result in results)
904     return np.arange(6.0 * 3600.0, np.floor(latest_common / (6.0 * 3600.0)) * 6.0 * 3600.0 + 1.0, 6.0 * 3600.0)
905
906
907 def comparison(left: SimulationResult | BEResult, right: SimulationResult | BEResult) -> dict[str, object]:
908     times = common_paper_times(left, right)
909     left_values = values_at_times(left, times)
910     right_values = values_at_times(right, times)
911     difference = np.abs(left_values - right_values)
912     location = np.unravel_index(int(np.argmax(difference)), difference.shape)
913     return {
914         "event_time_difference_s": float(abs(left.drying_event_time_s - right.drying_event_time_s)),
915         "event_time_relative_difference": float(
916             abs(left.drying_event_time_s - right.drying_event_time_s) / right.drying_event_time_s
917         ),
918         "paper_sample_maximum_moisture_difference": float(difference[location]),
919         "paper_sample_time_h_at_maximum": float(times[location[0]] / 3600.0),
920         "paper_sample_radius_cm_at_maximum": float(PAPER_RADII_CM[location[1]]),
921     }
922
923
924 def summarize_bdf(result: SimulationResult, runtime_s: float) -> dict[str, object]:
925     return {
926         "event_time_s": result.drying_event_time_s,
927         "event_time_h": result.drying_event_time_s / 3600.0,
928         "t60_s": result.report_end_time_s,
929         "event_max_moisture": float(result.event_internal_moisture.max()),
930         "event_argmax_radius_cm": float(
931             result.event_internal_radius_cm[int(np.argmax(result.event_internal_moisture))]
932         ),
933         "event_center_moisture": float(result.event_internal_moisture[0]),
934         "event_surface_moisture": float(result.event_internal_moisture[-1]),
935         "t60_max_moisture": float(result.max_moisture[-1]),
936         "previous_60s_max_moisture": float(result.max_moisture[-2]) if len(result.max_moisture) > 1 else None,
937         "runtime_s": runtime_s,
938         "diagnostics": asdict(result.diagnostics),
939     }
940
941
942 def summarize_be(result: BEResult, runtime_s: float) -> dict[str, object]:
943     return {
944         "event_time_s": result.drying_event_time_s,
945         "event_time_h": result.drying_event_time_s / 3600.0,

```

```

946         "event_max_moisture": float(result.event_internal_moisture.max()),
947         "event_argmax_radius_cm": float(
948             result.event_internal_radius_cm[int(np.argmax(result.event_internal_moisture))]
949         ),
950         "event_center_moisture": float(result.event_internal_moisture[0]),
951         "event_surface_moisture": float(result.event_internal_moisture[-1]),
952         "runtime_s": runtime_s,
953         "diagnostics": asdict(result.diagnostics),
954     }
955
956
957 def bdf_run(label: str, environment: EnvironmentSeries, config: SolverConfig, post: PostBoundary | None = None):
958     print(f"START BDF {label}", flush=True)
959     started = time.perf_counter()
960     result = solve_problem3_bdf(environment, config, post)
961     elapsed = time.perf_counter() - started
962     print(f"DONE BDF {label}: {result.drying_event_time_s / 3600.0:.9f} h ({elapsed:.2f} s)", flush=True)
963     return result, elapsed
964
965
966 def be_run(label: str, environment: EnvironmentSeries, config: BESolverConfig, post: PostBoundary | None = None):
967     print(f"START BE {label}", flush=True)
968     started = time.perf_counter()
969     result = solve_problem3_be(environment, config, post)
970     elapsed = time.perf_counter() - started
971     print(f"DONE BE {label}: {result.drying_event_time_s / 3600.0:.9f} h ({elapsed:.2f} s)", flush=True)
972     return result, elapsed
973
974
975 def write_result_xlsx(result: SimulationResult, template: Path, out_path: Path) -> None:
976     """读附件3 模板 (单 sheet) -> 写达标停机水分表: 22 列 (标签 + 21 个半径 0.0~2.0 cm) 。
977
978     A 列为整数秒 (number_format "0"), B~V 列四位小数 ("0.0000"), 冻结首行, A 列宽 24。
979     结果时间序列从 t=60s 起 (求解器输出不含 t=0 初值行), 逐行写到第 2 行起。
980     """
981
982     workbook = openpyxl.load_workbook(template)
983     sheet = workbook.worksheets[0]
984     radius = result.radius_cm
985     sheet.cell(1, 1).value = "时间\\到药材中心的距离"
986     for j in range(radius.size):
987         sheet.cell(1, 2 + j).value = round(float(radius[j]), 10)
988     for i in range(result.time_s.size):
989         time_cell = sheet.cell(i + 2, 1)
990         time_cell.value = int(round(result.time_s[i]))
991         time_cell.number_format = "0"
992         for j in range(radius.size):
993             cell = sheet.cell(i + 2, 2 + j)
994             cell.value = float(result.moisture[i, j]) # 存全精度, 仅用 number_format 控制四位小数显示 (与原
995             result3.xlsx 一致)
996             cell.number_format = "0.0000"
997         sheet.column_dimensions["A"].width = 24
998         sheet.freeze_panes = "A2"
999         out_path.parent.mkdir(parents=True, exist_ok=True)
1000         workbook.save(out_path)
1001
1002 def main() -> None:
1003     if not ATTACHMENT1.exists():
1004         raise SystemExit(
1005             f"未找到附件: {ATTACHMENT1}\n"
1006             "请把题目附件 附件1.xlsx 放到本脚本同根目录的 附件/ 文件夹下, 再运行: python problem3.py"
1007         )
1008     if not TEMPLATE.exists():
1009         raise SystemExit(
1010             f"未找到结果格式模板: {TEMPLATE}\n"
1011             "请把题目附件3的 result3.xlsx 放到 附件/附件3/ 下, 再运行: python problem3.py"
1012         )
1013     environment = load_environment(ATTACHMENT1)
1014     bar = "=" * 64
1015
1016     # ---- ① 空间网格逐级加密: 观察收敛阶 p、Richardson 外推、调和平均-网格序列 (论文模型检验章) ----
1017     spatial_dr = [0.1, 0.05, 0.025, 0.0125, 0.00625, 0.003125, 0.0015625, 0.00078125]
1018     spatial_config = SolverConfig(
1019         relative_tolerance=1e-8,
1020         temperature_absolute_tolerance=1e-8,
1021         moisture_absolute_tolerance=1e-10,
1022         max_time_step_s=60.0,
1023         extended_time_limit_s=1000.0 * 3600.0,

```

```

1 """问题四：附件2半径收缩 + 附录4物性下的热风烘干达标停机时间（自包含单文件）。
2 """
3
4 from __future__ import annotations
5
6 import json
7 import time
8 from dataclasses import asdict, dataclass, field, replace
9 from math import floor
10 from pathlib import Path
11 from typing import Callable
12
13 import numpy as np
14 import openpyxl
15 import pandas as pd
16 from openpyxl.utils import get_column_letter
17 from scipy.integrate import solve_ivp
18 from scipy.interpolate import PchipInterpolator
19 from scipy.linalg.lapack import dgtsv
20 from scipy.sparse import bmat, csc_matrix, diags
21
22
23 # ===== 求解核心 · 物性与环境边界（附录4） =====
24
25
26 @dataclass(frozen=True)
27 class PhysicalParameters:
28     """附录4物性及收缩幅度的可注入参数；默认值严格保持原模型口径。"""
29
30     density_intercept: float = 760.0
31     density_moisture_coefficient: float = 90.0
32     diffusivity_prefactor: float = 4.2e-4
33     diffusivity_moisture_parameter: float = 0.30
34     activation_temperature_k: float = 3850.0
35     shrinkage_amplitude_factor: float = 1.0
36
37     def __post_init__(self) -> None:
38         values = np.asarray(
39             [
40                 self.density_intercept,
41                 self.density_moisture_coefficient,
42                 self.diffusivity_prefactor,
43                 self.diffusivity_moisture_parameter,
44                 self.activation_temperature_k,
45                 self.shrinkage_amplitude_factor,
46             ],
47             dtype=float,
48         )
49         if not np.isfinite(values).all() or np.any(values <= 0.0):
50             raise ValueError("physical parameters must be positive and finite")
51
52
53 def safe_moisture(c: np.ndarray | float) -> np.ndarray:
54     """只在物性公式中使用的数值保护，不修改求解状态。"""
55
56     return np.maximum(np.asarray(c, dtype=float), 1.0e-9)
57
58
59 def density(
60     c: np.ndarray | float,
61     parameters: PhysicalParameters | None = None,
62 ) -> np.ndarray:
63     parameters = parameters or PhysicalParameters()
64     c_safe = safe_moisture(c)
65     return parameters.density_intercept + parameters.density_moisture_coefficient * c_safe
66
67
68 def heat_capacity(c: np.ndarray | float) -> np.ndarray:
69     c_safe = safe_moisture(c)
70     return 1850.0 + 2150.0 * c_safe / (1.0 + c_safe)
71
72
73 def thermal_conductivity(c: np.ndarray | float) -> np.ndarray:
74     c_safe = safe_moisture(c)
75     return 0.12 + 0.20 * c_safe / (1.0 + c_safe)
76
77
78 def moisture_diffusivity(

```

```

79     c: np.ndarray | float,
80     temperature_c: np.ndarray | float,
81     parameters: PhysicalParameters | None = None,
82 ) -> np.ndarray:
83     parameters = parameters or PhysicalParameters()
84     c_safe = safe_moisture(c)
85     temperature_k = np.asarray(temperature_c, dtype=float) + 273.15
86     if np.any(temperature_k <= 0.0):
87         raise ValueError("temperature must be above absolute zero")
88     return (
89         parameters.diffusivity_prefactor
90         * np.exp(-parameters.diffusivity_moisture_parameter / c_safe)
91         * np.exp(-parameters.activation_temperature_k / temperature_k)
92     )
93
94
95 @dataclass(frozen=True)
96 class EnvironmentSeries:
97     """附件1边界序列：附件区间内仅作分段线性插值。"""
98
99     time_s: np.ndarray
100     temperature_c: np.ndarray
101     moisture: np.ndarray
102
103     def __post_init__(self) -> None:
104         time = np.asarray(self.time_s, dtype=float)
105         temperature = np.asarray(self.temperature_c, dtype=float)
106         moisture = np.asarray(self.moisture, dtype=float)
107         if not (time.ndim == temperature.ndim == moisture.ndim == 1):
108             raise ValueError("environment arrays must be one-dimensional")
109         if len(time) < 2 or not (len(time) == len(temperature) == len(moisture)):
110             raise ValueError("environment arrays must have equal length >= 2")
111         if np.any(np.diff(time) <= 0.0):
112             raise ValueError("environment time must be strictly increasing")
113         if not (np.isfinite(time).all() and np.isfinite(temperature).all() and np.isfinite(moisture).all()):
114             raise ValueError("environment arrays must be finite")
115         object.__setattr__(self, "time_s", time)
116         object.__setattr__(self, "temperature_c", temperature)
117         object.__setattr__(self, "moisture", moisture)
118
119     @property
120     def terminal_time_s(self) -> float:
121         return float(self.time_s[-1])
122
123     @property
124     def terminal_temperature_c(self) -> float:
125         return float(self.temperature_c[-1])
126
127     @property
128     def terminal_moisture(self) -> float:
129         return float(self.moisture[-1])
130
131     def values_at(self, t_s: float) -> tuple[float, float]:
132         if t_s < self.time_s[0] or t_s > self.time_s[-1]:
133             raise ValueError("interpolation time is outside the attachment interval")
134         return (
135             float(np.interp(t_s, self.time_s, self.temperature_c)),
136             float(np.interp(t_s, self.time_s, self.moisture)),
137         )
138
139
140 @dataclass(frozen=True)
141 class PostBoundary:
142     temperature_c: float
143     moisture: float
144
145
146 def post_boundary_from_tail(
147     environment: EnvironmentSeries,
148     averaging_window_s: float | None = None,
149 ) -> PostBoundary:
150     if averaging_window_s is None:
151         return PostBoundary(environment.terminal_temperature_c, environment.terminal_moisture)
152     if averaging_window_s <= 0.0:
153         raise ValueError("averaging_window_s must be positive")
154     start = max(float(environment.time_s[0]), environment.terminal_time_s - averaging_window_s)
155     interior = environment.time_s[(environment.time_s > start) & (environment.time_s <=
environment.terminal_time_s)]
156     times = np.concatenate([[start], interior])

```

```

157 temperature = np.interp(times, environment.time_s, environment.temperature_c)
158 moisture = np.interp(times, environment.time_s, environment.moisture)
159 duration = float(times[-1] - times[0])
160 if duration <= 0.0:
161     raise ValueError("averaging window has zero duration")
162 return PostBoundary(
163     float(np.trapezoid(temperature, times) / duration),
164     float(np.trapezoid(moisture, times) / duration),
165 )
166
167
168 def boundary_at(
169     t_s: float,
170     environment: EnvironmentSeries,
171     post_boundary: PostBoundary,
172 ) -> tuple[float, float]:
173     if t_s <= environment.terminal_time_s:
174         return environment.values_at(t_s)
175     return post_boundary.temperature_c, post_boundary.moisture
176
177
178 @dataclass(frozen=True)
179 class RadiusSeries:
180     """附件2半径序列，默认线性插值，可切换单调 PCHIP。"""
181
182     time_s: np.ndarray
183     radius_cm: np.ndarray
184     interpolation: str = "linear"
185     _pchip: PchipInterpolator | None = field(default=None, init=False, repr=False, compare=False)
186
187     def __post_init__(self) -> None:
188         time = np.asarray(self.time_s, dtype=float)
189         radius = np.asarray(self.radius_cm, dtype=float)
190         if time.ndim != 1 or radius.ndim != 1 or len(time) != len(radius) or len(time) < 2:
191             raise ValueError("radius arrays must be one-dimensional with equal length >= 2")
192         if np.any(np.diff(time) <= 0.0):
193             raise ValueError("radius time must be strictly increasing")
194         if np.any(radius <= 0.0) or not (np.isfinite(time).all() and np.isfinite(radius).all()):
195             raise ValueError("radius data must be positive and finite")
196         if self.interpolation not in {"linear", "pchip"}:
197             raise ValueError("interpolation must be 'linear' or 'pchip'")
198         object.__setattr__(self, "time_s", time)
199         object.__setattr__(self, "radius_cm", radius)
200         if self.interpolation == "pchip":
201             object.__setattr__(self, "_pchip", PchipInterpolator(time, radius, extrapolate=False))
202
203     @property
204     def terminal_time_s(self) -> float:
205         return float(self.time_s[-1])
206
207     def radius_cm_at(self, t_s: float) -> float:
208         if t_s <= self.time_s[0]:
209             return float(self.radius_cm[0])
210         if t_s >= self.time_s[-1]:
211             return float(self.radius_cm[-1])
212         if self.interpolation == "linear":
213             return float(np.interp(t_s, self.time_s, self.radius_cm))
214         assert self._pchip is not None
215         return float(self._pchip(t_s))
216
217     def radius_rate_cm_s_at(self, t_s: float) -> float:
218         if t_s <= self.time_s[0] or t_s >= self.time_s[-1]:
219             return 0.0
220         if self.interpolation == "linear":
221             index = int(np.searchsorted(self.time_s, t_s, side="right") - 1)
222             index = min(max(index, 0), len(self.time_s) - 2)
223             return float(
224                 (self.radius_cm[index + 1] - self.radius_cm[index])
225                 / (self.time_s[index + 1] - self.time_s[index])
226             )
227         assert self._pchip is not None
228         return float(self._pchip.derivative()(t_s))
229
230     def with_interpolation(self, interpolation: str) -> "RadiusSeries":
231         return RadiusSeries(self.time_s, self.radius_cm, interpolation)
232
233
234 @dataclass(frozen=True)
235 class BoundaryScenario:

```

```

236     name: str
237     post_boundary: PostBoundary
238     mass_transfer_factor: float = 1.0
239
240
241 def engineering_boundary_scenarios(default: PostBoundary) -> list[BoundaryScenario]:
242     return [
243         BoundaryScenario(
244             f"T{temperature_delta:g}_C{moisture_factor:.1f}",
245             PostBoundary(
246                 default.temperature_c + temperature_delta,
247                 default.moisture * moisture_factor,
248             ),
249         )
250         for temperature_delta in (-2.0, 0.0, 2.0)
251         for moisture_factor in (0.9, 1.0, 1.1)
252     ]
253
254
255 def mass_transfer_scenarios(default: PostBoundary) -> list[BoundaryScenario]:
256     return [BoundaryScenario(f"hm_{factor:.1f}x", default, factor) for factor in (0.9, 1.0, 1.1)]
257
258 # ===== 求解核心 · 物质坐标 FVM 几何与界面离散 =====
259
260
261
262 @dataclass(frozen=True)
263 class ReferenceGeometry:
264     xi: np.ndarray
265     volumes_hat: np.ndarray
266     interface_areas_hat: np.ndarray
267     dxi: float
268
269     @property
270     def node_count(self) -> int:
271         return int(self.xi.size)
272
273
274 def build_reference_geometry(node_count: int) -> ReferenceGeometry:
275     if node_count < 2:
276         raise ValueError("node_count must be at least two")
277     xi = np.linspace(0.0, 1.0, node_count)
278     dxi = 1.0 / (node_count - 1)
279     left_faces = np.maximum(0.0, xi - 0.5 * dxi)
280     right_faces = np.minimum(1.0, xi + 0.5 * dxi)
281     volumes_hat = 0.5 * (right_faces**2 - left_faces**2)
282     interface_areas_hat = xi[:-1] + 0.5 * dxi
283     return ReferenceGeometry(xi, volumes_hat, interface_areas_hat, dxi)
284
285
286 def harmonic_mean(left: np.ndarray, right: np.ndarray) -> np.ndarray:
287     left = np.asarray(left, dtype=float)
288     right = np.asarray(right, dtype=float)
289     denominator = left + right
290     result = np.zeros(np.broadcast_shapes(left.shape, right.shape), dtype=float)
291     np.divide(2.0 * left * right, denominator, out=result, where=np.abs(denominator) > 0.0)
292     return result
293
294
295 def interface_values(node_values: np.ndarray, mean: str) -> np.ndarray:
296     values = np.asarray(node_values, dtype=float)
297     if mean == "harmonic":
298         return harmonic_mean(values[:-1], values[1:])
299     if mean == "arithmetic":
300         return 0.5 * (values[:-1] + values[1:])
301     raise ValueError("mean must be 'harmonic' or 'arithmetic'")
302
303
304 def internal_flux_numerator(
305     values: np.ndarray,
306     interface_transport: np.ndarray,
307     geometry: ReferenceGeometry,
308 ) -> np.ndarray:
309     values = np.asarray(values, dtype=float)
310     flux = (
311         np.asarray(interface_transport, dtype=float)
312         * geometry.interface_areas_hat
313         * (values[1:] - values[:-1])
314         / geometry.dxi

```

```

315     )
316     numerator = np.zeros_like(values)
317     numerator[:-1] += flux
318     numerator[1:] -= flux
319     return numerator
320
321
322 def build_coupled_jacobian_sparsity(node_count: int) -> csc_matrix:
323     if node_count < 2:
324         raise ValueError("node_count must be at least two")
325     tri = diags(
326         [np.ones(node_count - 1), np.ones(node_count), np.ones(node_count - 1)],
327         [-1, 0, 1],
328         shape=(node_count, node_count),
329         format="csc",
330     )
331     return bmat([[tri, tri], [tri, tri]], format="csc")
332
333
334 def sample_physical_profile(
335     profile: np.ndarray,
336     geometry: ReferenceGeometry,
337     radius_cm: float,
338     fixed_distances_cm: np.ndarray,
339 ) -> np.ndarray:
340     output = np.full(len(fixed_distances_cm) + 1, np.nan, dtype=float)
341     for index, distance_cm in enumerate(fixed_distances_cm):
342         if distance_cm <= radius_cm + 1.0e-12:
343             output[index] = float(np.interp(distance_cm / radius_cm, geometry.xi, profile))
344     output[-1] = float(profile[-1])
345     return output
346
347
348
349 # ===== 求解核心 · 联合状态自适应 BDF 求解器（主求解器）=====
350
351
352 DEFAULT_FIXED_DISTANCES_CM = tuple(float(value) for value in np.round(np.arange(0.0, 2.0, 0.1), 10))
353
354
355 @dataclass(frozen=True)
356 class SolverConfig:
357     node_count: int = 801
358     sample_interval_s: int = 60
359     report_interval_s: int = 60
360     fixed_distances_cm: tuple[float, ...] = DEFAULT_FIXED_DISTANCES_CM
361     initial_temperature_c: float = 28.0
362     initial_moisture: float = 2.55
363     completion_threshold: float = 0.15
364     heat_transfer_w_m2_k: float = 25.0
365     mass_transfer_m_s: float = 8.0e-7
366     k_interface_mean: str = "harmonic"
367     d_interface_mean: str = "harmonic"
368     model_form: str = "material"
369     fixed_radius_cm: float | None = None
370     physical_parameters: PhysicalParameters = field(default_factory=PhysicalParameters)
371     integration_method: str = "BDF"
372     relative_tolerance: float = 1.0e-8
373     temperature_absolute_tolerance: float = 1.0e-8
374     moisture_absolute_tolerance: float = 1.0e-10
375     max_time_step_s: float = 30.0
376     initial_time_limit_s: float = 72.0 * 3600.0
377     fallback_time_limit_s: float = 96.0 * 3600.0
378     material_negative_tolerance: float = -1.0e-8
379     monotonicity_tolerance: float = 1.0e-8
380
381
382 @dataclass(frozen=True)
383 class SolverDiagnostics:
384     internal_node_count: int
385     state_size: int
386     function_evaluations: int
387     jacobian_evaluations: int
388     lu_decompositions: int
389     phase1_end_time_s: float
390     first_limit_reached_without_event: bool
391     event_count: int
392     event_direction: int
393     minimum_temperature_c: float

```

```

394     maximum_temperature_c: float
395     minimum_raw_moisture: float
396     clipped_cell_count: int
397     maximum_clip_magnitude: float
398     radial_monotonicity_all_outputs: bool
399     non_center_argmax_count: int
400     maximum_normalized_balance_residual: float
401     dry_mass_index_relative_range: float
402
403
404 @dataclass(frozen=True)
405 class SimulationResult:
406     time_s: np.ndarray
407     radius_cm: np.ndarray
408     fixed_distance_cm: np.ndarray
409     temperature_c: np.ndarray
410     moisture: np.ndarray
411     argmax_xi: np.ndarray
412     argmax_radius_cm: np.ndarray
413     max_moisture: np.ndarray
414     center_moisture: np.ndarray
415     radial_monotonicity: np.ndarray
416     dry_mass_index: np.ndarray
417     drying_event_time_s: float
418     report_end_time_s: int
419     previous_report_time_s: int
420     previous_report_max_moisture: float
421     report_max_moisture: float
422     event_radius_cm: float
423     event_temperature_internal: np.ndarray
424     event_moisture_internal: np.ndarray
425     event_output_temperature: np.ndarray
426     event_output_moisture: np.ndarray
427     diagnostics: SolverDiagnostics
428
429
430 def drying_event_value(state: np.ndarray, node_count: int, threshold: float) -> float:
431     return float(np.max(state[node_count:]) - threshold)
432
433
434 def _radius_values(t_s: float, radius: RadiusSeries, config: SolverConfig) -> tuple[float, float]:
435     if config.fixed_radius_cm is not None:
436         if config.fixed_radius_cm <= 0.0:
437             raise ValueError("fixed_radius_cm must be positive")
438         return config.fixed_radius_cm / 100.0, 0.0
439     base_radius_cm = float(radius.radius_cm[0])
440     shrinkage_factor = config.physical_parameters.shrinkage_amplitude_factor
441     radius_cm = base_radius_cm + shrinkage_factor * (radius.radius_cm_at(t_s) - base_radius_cm)
442     radius_rate_cm_s = shrinkage_factor * radius.radius_rate_cm_s_at(t_s)
443     if radius_cm <= 0.0:
444         raise ValueError("scaled radius must remain positive")
445     return radius_cm / 100.0, radius_rate_cm_s / max(radius_cm, 1.0e-30)
446
447
448 def build_coupled_rhs(
449     geometry: ReferenceGeometry,
450     environment: EnvironmentSeries,
451     post_boundary: PostBoundary,
452     radius: RadiusSeries,
453     config: SolverConfig,
454 ) -> Callable[[float, np.ndarray], np.ndarray]:
455     if config.model_form not in {"material", "eulerian"}:
456         raise ValueError("model_form must be 'material' or 'eulerian'")
457     node_count = geometry.node_count
458
459     def rhs(t_s: float, state: np.ndarray) -> np.ndarray:
460         temperature = state[:node_count]
461         moisture = state[node_count:]
462         radius_m, radius_rate_ratio_s = _radius_values(t_s, radius, config)
463         external_temperature, external_moisture = boundary_at(t_s, environment, post_boundary)
464
465         node_k = thermal_conductivity(moisture)
466         node_d = moisture_diffusivity(moisture, temperature, config.physical_parameters)
467         heat_numerator = internal_flux_numerator(
468             temperature,
469             interface_values(node_k, config.k_interface_mean),
470             geometry,
471         )
472         moisture_numerator = internal_flux_numerator(

```

```

473         moisture,
474         interface_values(node_d, config.d_interface_mean),
475         geometry,
476     )
477     heat_numerator[-1] += radius_m * config.heat_transfer_w_m2_k * (
478         external_temperature - temperature[-1]
479     )
480     moisture_numerator[-1] += radius_m * config.mass_transfer_m_s * (
481         external_moisture - moisture[-1]
482     )
483
484     rho_cp = density(moisture, config.physical_parameters) * heat_capacity(moisture)
485     temperature_rate = heat_numerator / (radius_m**2 * rho_cp * geometry.volumes_hat)
486     moisture_rate = moisture_numerator / (radius_m**2 * geometry.volumes_hat)
487
488     if config.model_form == "eulerian":
489         temperature_gradient = np.gradient(temperature, geometry.dxi, edge_order=2)
490         moisture_gradient = np.gradient(moisture, geometry.dxi, edge_order=2)
491         temperature_rate += geometry.xi * radius_rate_ratio_s * temperature_gradient
492         moisture_rate += geometry.xi * radius_rate_ratio_s * moisture_gradient
493
494     derivative = np.concatenate([temperature_rate, moisture_rate])
495     if not np.isfinite(derivative).all():
496         raise RuntimeError("RHS returned non-finite values")
497     return derivative
498
499     return rhs
500
501
502 def _evaluate_segments(segments: list, times: np.ndarray) -> np.ndarray:
503     states = np.empty((segments[0].y.shape[0], times.size), dtype=float)
504     assigned = np.zeros(times.size, dtype=bool)
505     for segment in segments:
506         mask = (~assigned) & (times >= segment.t[0] - 1.0e-8) & (times <= segment.t[-1] + 1.0e-8)
507         if np.any(mask):
508             states[:, mask] = segment.sol(times[mask])
509             assigned[mask] = True
510     if not assigned.all():
511         raise RuntimeError(f"no dense-output segment covers {times[~assigned]}")
512     return states
513
514
515 def _sample_times(event_time_s: float, report_end_time_s: int, config: SolverConfig) -> np.ndarray:
516     regular = np.arange(
517         config.sample_interval_s,
518         floor(event_time_s / config.sample_interval_s) * config.sample_interval_s + 1,
519         config.sample_interval_s,
520         dtype=float,
521     )
522     required = np.array(
523         [float(report_end_time_s - config.report_interval_s), float(report_end_time_s)],
524         dtype=float,
525     )
526     return np.unique(np.concatenate([regular, required]))
527
528
529 def _normalized_balance_residual(
530     t_s: float,
531     state: np.ndarray,
532     rhs: Callable[[float, np.ndarray], np.ndarray],
533     geometry: ReferenceGeometry,
534     environment: EnvironmentSeries,
535     post_boundary: PostBoundary,
536     radius: RadiusSeries,
537     config: SolverConfig,
538 ) -> float:
539     if config.model_form != "material":
540         return 0.0
541     n = geometry.node_count
542     moisture = state[n:]
543     radius_m, _ = _radius_values(t_s, radius, config)
544     _, external_moisture = boundary_at(t_s, environment, post_boundary)
545     boundary_term = config.mass_transfer_m_s * (external_moisture - moisture[-1]) / radius_m
546     weighted_rate = float(np.sum(rhs(t_s, state)[n:] * geometry.volumes_hat))
547     residual = weighted_rate - boundary_term
548     scale = abs(weighted_rate) + abs(boundary_term) + 1.0e-30
549     return abs(residual) / scale
550
551

```

```

552 def _dry_mass_index(radius_m: float, moisture: np.ndarray, geometry: ReferenceGeometry) -> float:
553     return _dry_mass_index_with_parameters(radius_m, moisture, geometry, PhysicalParameters())
554
555
556 def _dry_mass_index_with_parameters(
557     radius_m: float,
558     moisture: np.ndarray,
559     geometry: ReferenceGeometry,
560     parameters: PhysicalParameters,
561 ) -> float:
562     return float(
563         radius_m**2
564         * np.sum(
565             density(moisture, parameters)
566             / (1.0 + np.asarray(moisture, dtype=float))
567             * geometry.volumes_hat
568         )
569     )
570
571
572 def solve_problem4_bdf(
573     environment: EnvironmentSeries,
574     radius: RadiusSeries,
575     config: SolverConfig | None = None,
576     post_boundary: PostBoundary | None = None,
577 ) -> SimulationResult:
578     """严格物质坐标 FVM + 联合状态自适应 BDF + 连续终止事件。"""
579
580     config = config or SolverConfig()
581     post_boundary = post_boundary or post_boundary_from_tail(environment)
582     if environment.time_s[0] > 0.0:
583         raise ValueError("environment must start at or before t=0")
584     if config.initial_time_limit_s <= environment.terminal_time_s:
585         raise ValueError("initial_time_limit_s must exceed attachment terminal time")
586     if config.fallback_time_limit_s <= config.initial_time_limit_s:
587         raise ValueError("fallback_time_limit_s must exceed initial_time_limit_s")
588     if config.sample_interval_s <= 0 or config.report_interval_s <= 0:
589         raise ValueError("output intervals must be positive")
590     if config.integration_method not in {"BDF", "Radau"}:
591         raise ValueError("integration_method must be 'BDF' or 'Radau'")
592
593     geometry = build_reference_geometry(config.node_count)
594     node_count = geometry.node_count
595     jacobian_pattern = build_coupled_jacobian_sparsity(node_count)
596     rhs = build_coupled_rhs(geometry, environment, post_boundary, radius, config)
597     initial_state = np.concatenate(
598         [
599             np.full(node_count, config.initial_temperature_c, dtype=float),
600             np.full(node_count, config.initial_moisture, dtype=float),
601         ]
602     )
603     atol = np.concatenate(
604         [
605             np.full(node_count, config.temperature_absolute_tolerance),
606             np.full(node_count, config.moisture_absolute_tolerance),
607         ]
608     )
609
610     def integrate(start: float, end: float, state: np.ndarray, with_event: bool):
611         events = None
612         if with_event:
613             def event(_: float, y: np.ndarray) -> float:
614                 return drying_event_value(y, node_count, config.completion_threshold)
615
616             event.terminal = True
617             event.direction = -1
618             events = event
619         solution = solve_ivp(
620             rhs,
621             (float(start), float(end)),
622             state,
623             method=config.integration_method,
624             events=events,
625             dense_output=True,
626             rtol=config.relative_tolerance,
627             atol=atol,
628             max_step=config.max_time_step_s,
629             jac_sparsity=jacobian_pattern,
630         )

```

```

631         if not solution.success:
632             raise RuntimeError(f"{config.integration_method} integration failed: {solution.message}")
633         if not np.isfinite(solution.y).all():
634             raise RuntimeError(f"{config.integration_method} integration returned non-finite values")
635         return solution
636
637     segments = []
638     phase1 = integrate(0.0, environment.terminal_time_s, initial_state, False)
639     segments.append(phase1)
640     if drying_event_value(phase1.y[:, -1], node_count, config.completion_threshold) <= 0.0:
641         raise RuntimeError("drying threshold was reached before attachment boundary ended")
642
643     phase2 = integrate(
644         environment.terminal_time_s,
645         config.initial_time_limit_s,
646         phase1.y[:, -1],
647         True,
648     )
649     segments.append(phase2)
650     first_limit_reached = len(phase2.t_events[0]) == 0
651     if first_limit_reached:
652         phase3 = integrate(
653             config.initial_time_limit_s,
654             config.fallback_time_limit_s,
655             phase2.y[:, -1],
656             True,
657         )
658         segments.append(phase3)
659         if len(phase3.t_events[0]) == 0:
660             terminal_max = float(np.max(phase3.y[node_count:, -1]))
661             raise RuntimeError(
662                 f"drying threshold was not reached by {config.fallback_time_limit_s / 3600.0:.1f} h; "
663                 f"max moisture={terminal_max:.9g}"
664             )
665         event_time = float(phase3.t_events[0][0])
666         event_state = phase3.y_events[0][0]
667     else:
668         event_time = float(phase2.t_events[0][0])
669         event_state = phase2.y_events[0][0]
670
671     report_end = int(config.report_interval_s * (floor(event_time / config.report_interval_s) + 1))
672     extension = integrate(event_time, float(report_end), event_state, False)
673     while drying_event_value(extension.y[:, -1], node_count, config.completion_threshold) >= 0.0:
674         report_end += config.report_interval_s
675         extension = integrate(event_time, float(report_end), event_state, False)
676     segments.append(extension)
677
678     previous_report = report_end - config.report_interval_s
679     sample_times = _sample_times(event_time, report_end, config)
680     sample_states = _evaluate_segments(segments, sample_times)
681     sample_temperature = sample_states[:node_count, :].T
682     sample_moisture = sample_states[node_count:, :].T
683     radius_cm = np.array(
684         [100.0 * _radius_values(float(t_s), radius, config)[0] for t_s in sample_times],
685         dtype=float,
686     )
687     fixed_distance = np.asarray(config.fixed_distances_cm, dtype=float)
688     output_temperature = np.vstack(
689         [
690             sample_physical_profile(profile, geometry, r_cm, fixed_distance)
691             for profile, r_cm in zip(sample_temperature, radius_cm, strict=True)
692         ]
693     )
694     output_moisture = np.vstack(
695         [
696             sample_physical_profile(profile, geometry, r_cm, fixed_distance)
697             for profile, r_cm in zip(sample_moisture, radius_cm, strict=True)
698         ]
699     )
700
701     argmax_indices = np.argmax(sample_moisture, axis=1)
702     argmax_xi = geometry.xi[argmax_indices]
703     argmax_radius_cm = argmax_xi * radius_cm
704     max_moisture = np.max(sample_moisture, axis=1)
705     monotonicity = np.all(np.diff(sample_moisture, axis=1) <= config.monotonicity_tolerance, axis=1)
706     balance_residuals = np.array(
707         [
708             _normalized_balance_residual(
709                 float(t_s), sample_states[:, index], rhs, geometry, environment, post_boundary, radius, config

```

```

710         )
711         for index, t_s in enumerate(sample_times)
712     ],
713     dtype=float,
714 )
715 dry_mass = np.array(
716     [
717         _dry_mass_index_with_parameters(
718             r_cm / 100.0,
719             profile,
720             geometry,
721             config.physical_parameters,
722         )
723         for r_cm, profile in zip(radius_cm, sample_moisture, strict=True)
724     ],
725     dtype=float,
726 )
727 initial_radius_m, _ = _radius_values(0.0, radius, config)
728 initial_dry_mass = _dry_mass_index_with_parameters(
729     initial_radius_m,
730     initial_state[node_count:],
731     geometry,
732     config.physical_parameters,
733 )
734 dry_mass_with_initial = np.concatenate([[initial_dry_mass], dry_mass])
735 dry_mass_relative_range = float(
736     (np.max(dry_mass_with_initial) - np.min(dry_mass_with_initial))
737     / max(abs(initial_dry_mass), 1.0e-30)
738 )
739
740 event_radius_cm = 100.0 * _radius_values(event_time, radius, config)[0]
741 event_temperature = event_state[:node_count].copy()
742 event_moisture = event_state[node_count:].copy()
743 event_output_temperature = sample_physical_profile(
744     event_temperature, geometry, event_radius_cm, fixed_distance
745 )
746 event_output_moisture = sample_physical_profile(
747     event_moisture, geometry, event_radius_cm, fixed_distance
748 )
749 previous_state = _evaluate_segments(segments, np.array([float(previous_report)]))[:, 0]
750 report_state = extension.sol(float(report_end))
751 minimum_raw_moisture = float(
752     min(np.min(sample_moisture), np.min(event_moisture), np.min(report_state[node_count:]))
753 )
754 if minimum_raw_moisture < config.material_negative_tolerance:
755     raise RuntimeError(f"materially negative moisture returned: {minimum_raw_moisture:.6g}")
756
757 diagnostics = SolverDiagnostics(
758     internal_node_count=node_count,
759     state_size=2 * node_count,
760     function_evaluations=int(sum(segment.nfev for segment in segments)),
761     jacobian_evaluations=int(sum(segment.njev for segment in segments)),
762     lu_decompositions=int(sum(segment.nlu for segment in segments)),
763     phase1_end_time_s=environment.terminal_time_s,
764     first_limit_reached_without_event=first_limit_reached,
765     event_count=1,
766     event_direction=-1,
767     minimum_temperature_c=float(
768         min(config.initial_temperature_c, np.min(sample_temperature), np.min(event_temperature))
769     ),
770     maximum_temperature_c=float(max(np.max(sample_temperature), np.max(event_temperature))),
771     minimum_raw_moisture=minimum_raw_moisture,
772     clipped_cell_count=0,
773     maximum_clip_magnitude=0.0,
774     radial_monotonicity_all_outputs=bool(monotonicity.all()),
775     non_center_argmax_count=int(np.count_nonzero(argmax_indices)),
776     maximum_normalized_balance_residual=float(np.max(balance_residuals)),
777     dry_mass_index_relative_range=dry_mass_relative_range,
778 )
779 return SimulationResult(
780     time_s=sample_times,
781     radius_cm=radius_cm,
782     fixed_distance_cm=fixed_distance,
783     temperature_c=output_temperature,
784     moisture=output_moisture,
785     argmax_xi=argmax_xi,
786     argmax_radius_cm=argmax_radius_cm,
787     max_moisture=max_moisture,
788     center_moisture=sample_moisture[:, 0],

```

```

789         radial_monotonicity=monotonicity,
790         dry_mass_index=dry_mass,
791         drying_event_time_s=event_time,
792         report_end_time_s=report_end,
793         previous_report_time_s=previous_report,
794         previous_report_max_moisture=float(np.max(previous_state[node_count:])),
795         report_max_moisture=float(np.max(report_state[node_count:])),
796         event_radius_cm=event_radius_cm,
797         event_temperature_internal=event_temperature,
798         event_moisture_internal=event_moisture,
799         event_output_temperature=event_output_temperature,
800         event_output_moisture=event_output_moisture,
801         diagnostics=diagnostics,
802     )
803
804
805
806 # ===== 求解核心 · 后向欧拉-Picard 对照求解器 =====
807
808
809 @dataclass(frozen=True)
810 class BESolverConfig:
811     node_count: int = 801
812     time_step_s: float = 10.0
813     sample_interval_s: int = 21600
814     fixed_distances_cm: tuple[float, ...] = DEFAULT_FIXED_DISTANCES_CM
815     initial_temperature_c: float = 28.0
816     initial_moisture: float = 2.55
817     completion_threshold: float = 0.15
818     heat_transfer_w_m2_k: float = 25.0
819     mass_transfer_m_s: float = 8.0e-7
820     k_interface_mean: str = "harmonic"
821     d_interface_mean: str = "harmonic"
822     fixed_radius_cm: float | None = None
823     maximum_picard_iterations: int = 20
824     temperature_picard_tolerance: float = 1.0e-8
825     moisture_picard_tolerance: float = 1.0e-10
826     fallback_time_limit_s: float = 96.0 * 3600.0
827     event_bracket_tolerance_s: float = 0.5
828     material_negative_tolerance: float = -1.0e-8
829
830
831 @dataclass(frozen=True)
832 class BEDiagnostics:
833     accepted_steps: int
834     bisection_steps: int
835     maximum_picard_iterations_used: int
836     mean_picard_iterations: float
837     unconverged_steps: int
838     maximum_final_temperature_iteration_error: float
839     maximum_final_moisture_iteration_error: float
840     minimum_raw_moisture: float
841
842
843 @dataclass(frozen=True)
844 class BEResult:
845     time_s: np.ndarray
846     radius_cm: np.ndarray
847     fixed_distance_cm: np.ndarray
848     temperature_c: np.ndarray
849     moisture: np.ndarray
850     drying_event_time_s: float
851     event_radius_cm: float
852     event_temperature_internal: np.ndarray
853     event_moisture_internal: np.ndarray
854     event_output_temperature: np.ndarray
855     event_output_moisture: np.ndarray
856     diagnostics: BEDiagnostics
857
858
859 def _solve_tridiagonal(
860     lower: np.ndarray,
861     diagonal: np.ndarray,
862     upper: np.ndarray,
863     right_hand_side: np.ndarray,
864 ) -> np.ndarray:
865     _, _, _, result, info = dgtsv(
866         lower.copy(),
867         diagonal.copy(),

```

```

868         upper.copy(),
869         right_hand_side.copy(),
870         overwrite_d1=True,
871         overwrite_d=True,
872         overwrite_du=True,
873         overwrite_b=True,
874     )
875     if info != 0:
876         raise RuntimeError(f"compiled TDMA failed with LAPACK info={info}")
877     return np.asarray(result, dtype=float)
878
879
880 def _radius_m_at(t_s: float, radius: RadiusSeries, fixed_radius_cm: float | None) -> float:
881     if fixed_radius_cm is not None:
882         if fixed_radius_cm <= 0.0:
883             raise ValueError("fixed_radius_cm must be positive")
884         return fixed_radius_cm / 100.0
885     return radius.radius_cm_at(t_s) / 100.0
886
887
888 def _implicit_reference_step(
889     previous: np.ndarray,
890     node_transport: np.ndarray,
891     capacity: np.ndarray,
892     dt_s: float,
893     radius_m: float,
894     boundary_transfer: float,
895     boundary_external: float,
896     geometry: ReferenceGeometry,
897     interface_mean: str,
898 ) -> np.ndarray:
899     face_transport = interface_values(node_transport, interface_mean)
900     conductance = face_transport * geometry.interface_areas_hat / (geometry.dxi * radius_m**2)
901     accumulation = capacity * geometry.volumes_hat / dt_s
902     lower = -conductance.copy()
903     upper = -conductance.copy()
904     diagonal = accumulation.copy()
905     diagonal[:-1] += conductance
906     diagonal[1:] += conductance
907     right_hand_side = accumulation * previous
908     boundary_conductance = boundary_transfer / radius_m
909     diagonal[-1] += boundary_conductance
910     right_hand_side[-1] += boundary_conductance * boundary_external
911     return _solve_tridiagonal(lower, diagonal, upper, right_hand_side)
912
913
914 def coupled_backward_euler_step(
915     previous_temperature: np.ndarray,
916     previous_moisture: np.ndarray,
917     t_next_s: float,
918     dt_s: float,
919     geometry: ReferenceGeometry,
920     environment: EnvironmentSeries,
921     post_boundary: PostBoundary,
922     radius: RadiusSeries,
923     config: BESolverConfig,
924 ) -> tuple[np.ndarray, np.ndarray, int, float, float]:
925     if dt_s <= 0.0:
926         raise ValueError("dt_s must be positive")
927     external_temperature, external_moisture = boundary_at(t_next_s, environment, post_boundary)
928     radius_m = _radius_m_at(t_next_s, radius, config.fixed_radius_cm)
929     temperature_guess = previous_temperature.copy()
930     moisture_guess = previous_moisture.copy()
931     last_temperature_error = np.inf
932     last_moisture_error = np.inf
933     for iteration in range(1, config.maximum_picard_iterations + 1):
934         rho_cp = density(moisture_guess) * heat_capacity(moisture_guess)
935         next_temperature = _implicit_reference_step(
936             previous_temperature,
937             thermal_conductivity(moisture_guess),
938             rho_cp,
939             dt_s,
940             radius_m,
941             config.heat_transfer_w_m2_k,
942             external_temperature,
943             geometry,
944             config.k_interface_mean,
945         )
946         next_moisture = _implicit_reference_step(

```

```

947         previous_moisture,
948         moisture_diffusivity(moisture_guess, next_temperature),
949         np.ones_like(moisture_guess),
950         dt_s,
951         radius_m,
952         config.mass_transfer_m_s,
953         external_moisture,
954         geometry,
955         config.d_interface_mean,
956     )
957     last_temperature_error = float(np.max(np.abs(next_temperature - temperature_guess)))
958     last_moisture_error = float(np.max(np.abs(next_moisture - moisture_guess)))
959     if (
960         last_temperature_error <= config.temperature_picard_tolerance
961         and last_moisture_error <= config.moisture_picard_tolerance
962     ):
963         return next_temperature, next_moisture, iteration, last_temperature_error, last_moisture_error
964     temperature_guess = next_temperature
965     moisture_guess = next_moisture
966     raise RuntimeError(
967         "Picard iteration did not converge: "
968         f"t={t_next_s:.6f}s, dt={dt_s:.6f}s, "
969         f"temperature_error={last_temperature_error:.3e}, moisture_error={last_moisture_error:.3e}"
970     )
971
972
973 def solve_problem4_be(
974     environment: EnvironmentSeries,
975     radius: RadiusSeries,
976     config: BESolverConfig | None = None,
977     post_boundary: PostBoundary | None = None,
978 ) -> BEResult:
979     config = config or BESolverConfig()
980     post_boundary = post_boundary or post_boundary_from_tail(environment)
981     geometry = build_reference_geometry(config.node_count)
982     fixed_distance = np.asarray(config.fixed_distances_cm, dtype=float)
983     temperature = np.full(config.node_count, config.initial_temperature_c, dtype=float)
984     moisture = np.full(config.node_count, config.initial_moisture, dtype=float)
985     time_s = 0.0
986     output_times: list[float] = []
987     output_radii: list[float] = []
988     output_temperature: list[np.ndarray] = []
989     output_moisture: list[np.ndarray] = []
990     iteration_counts: list[int] = []
991     temperature_errors: list[float] = []
992     moisture_errors: list[float] = []
993     minimum_raw_moisture = float(np.min(moisture))
994     bisection_steps = 0
995     event_time = None
996     event_temperature = None
997     event_moisture = None
998
999     while time_s < config.fallback_time_limit_s - 1.0e-12:
1000         step_end = min(time_s + config.time_step_s, config.fallback_time_limit_s)
1001         if time_s < environment.terminal_time_s < step_end:
1002             step_end = environment.terminal_time_s
1003         next_sample = config.sample_interval_s * (np.floor(time_s / config.sample_interval_s) + 1.0)
1004         if time_s + 1.0e-10 < next_sample < step_end - 1.0e-10:
1005             step_end = float(next_sample)
1006         dt_s = step_end - time_s
1007         next_temperature, next_moisture, iterations, error_t, error_c = coupled_backward_euler_step(
1008             temperature,
1009             moisture,
1010             step_end,
1011             dt_s,
1012             geometry,
1013             environment,
1014             post_boundary,
1015             radius,
1016             config,
1017         )
1018         iteration_counts.append(iterations)
1019         temperature_errors.append(error_t)
1020         moisture_errors.append(error_c)
1021         minimum_raw_moisture = min(minimum_raw_moisture, float(np.min(next_moisture)))
1022         if minimum_raw_moisture < config.material_negative_tolerance:
1023             raise RuntimeError(f"materially negative moisture returned: {minimum_raw_moisture:.6g}")
1024
1025     previous_event_value = float(np.max(moisture) - config.completion_threshold)

```

```

1026     next_event_value = float(np.max(next_moisture) - config.completion_threshold)
1027     if previous_event_value > 0.0 and next_event_value <= 0.0:
1028         left_time = time_s
1029         left_temperature = temperature.copy()
1030         left_moisture = moisture.copy()
1031         right_time = step_end
1032         right_temperature = next_temperature.copy()
1033         right_moisture = next_moisture.copy()
1034         while right_time - left_time > config.event_bracket_tolerance_s:
1035             middle_time = 0.5 * (left_time + right_time)
1036             middle_temperature, middle_moisture, _, _, _ = coupled_backward_euler_step(
1037                 left_temperature,
1038                 left_moisture,
1039                 middle_time,
1040                 middle_time - left_time,
1041                 geometry,
1042                 environment,
1043                 post_boundary,
1044                 radius,
1045                 config,
1046             )
1047             bisection_steps += 1
1048             if float(np.max(middle_moisture) - config.completion_threshold) <= 0.0:
1049                 right_time = middle_time
1050                 right_temperature = middle_temperature
1051                 right_moisture = middle_moisture
1052             else:
1053                 left_time = middle_time
1054                 left_temperature = middle_temperature
1055                 left_moisture = middle_moisture
1056         event_time = right_time
1057         event_temperature = right_temperature
1058         event_moisture = right_moisture
1059         break
1060
1061     time_s = step_end
1062     temperature = next_temperature
1063     moisture = next_moisture
1064     if np.isclose(time_s % config.sample_interval_s, 0.0, atol=1.0e-8):
1065         radius_cm = 100.0 * _radius_m_at(time_s, radius, config.fixed_radius_cm)
1066         output_times.append(time_s)
1067         output_radii.append(radius_cm)
1068         output_temperature.append(sample_physical_profile(temperature, geometry, radius_cm, fixed_distance))
1069         output_moisture.append(sample_physical_profile(moisture, geometry, radius_cm, fixed_distance))
1070
1071     if event_time is None or event_temperature is None or event_moisture is None:
1072         raise RuntimeError(
1073             f"drying threshold was not reached by {config.fallback_time_limit_s / 3600.0:.1f} h"
1074         )
1075     event_radius_cm = 100.0 * _radius_m_at(event_time, radius, config.fixed_radius_cm)
1076     diagnostics = BEDiagnostics(
1077         accepted_steps=len(iteration_counts),
1078         bisection_steps=bisection_steps,
1079         maximum_picard_iterations_used=int(max(iteration_counts)),
1080         mean_picard_iterations=float(np.mean(iteration_counts)),
1081         unconverged_steps=0,
1082         maximum_final_temperature_iteration_error=float(max(temperature_errors)),
1083         maximum_final_moisture_iteration_error=float(max(moisture_errors)),
1084         minimum_raw_moisture=minimum_raw_moisture,
1085     )
1086     return BEResult(
1087         time_s=np.asarray(output_times, dtype=float),
1088         radius_cm=np.asarray(output_radii, dtype=float),
1089         fixed_distance_cm=fixed_distance,
1090         temperature_c=np.vstack(output_temperature) if output_temperature else np.empty((0, len(fixed_distance) +
1091 1)),
1092         moisture=np.vstack(output_moisture) if output_moisture else np.empty((0, len(fixed_distance) + 1)),
1093         drying_event_time_s=float(event_time),
1094         event_radius_cm=event_radius_cm,
1095         event_temperature_internal=event_temperature.copy(),
1096         event_moisture_internal=event_moisture.copy(),
1097         event_output_temperature=sample_physical_profile(
1098             event_temperature, geometry, event_radius_cm, fixed_distance
1099         ),
1100         event_output_moisture=sample_physical_profile(event_moisture, geometry, event_radius_cm, fixed_distance),
1101         diagnostics=diagnostics,
1102     )
1103

```

```

1104
1105 # ===== 运行与产物（全部默认，无命令行参数）=====
1106
1107
1108 BASE = Path(__file__).resolve().parent
1109 ATTACHMENT1 = BASE / "附件" / "附件1.xlsx"
1110 ATTACHMENT2 = BASE / "附件" / "附件2.xlsx"
1111 TEMPLATE = BASE / "附件" / "附件3" / "result4.xlsx"
1112 OUT_XLSX = BASE / "result4.xlsx"
1113 OUT_JSON = BASE / "verification4.json"
1114
1115
1116 def json_dump(path: Path, value) -> None:
1117     path.parent.mkdir(parents=True, exist_ok=True)
1118     path.write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8")
1119
1120
1121 def load_inputs(attachment1: Path, attachment2: Path) -> tuple[EnvironmentSeries, RadiusSeries]:
1122     environment_frame = pd.read_excel(attachment1)
1123     radius_frame = pd.read_excel(attachment2)
1124     environment = EnvironmentSeries(
1125         environment_frame.iloc[:, 0].to_numpy(dtype=float),
1126         environment_frame.iloc[:, 1].to_numpy(dtype=float),
1127         environment_frame.iloc[:, 2].to_numpy(dtype=float),
1128     )
1129     radius = RadiusSeries(
1130         radius_frame.iloc[:, 0].to_numpy(dtype=float),
1131         radius_frame.iloc[:, 1].to_numpy(dtype=float),
1132         "linear",
1133     )
1134     return environment, radius
1135
1136
1137 def timed_bdf(
1138     label: str,
1139     environment: EnvironmentSeries,
1140     radius: RadiusSeries,
1141     config: SolverConfig,
1142     post_boundary: PostBoundary | None = None,
1143 ) -> tuple[SimulationResult, float]:
1144     print(f"START BDF {label}", flush=True)
1145     started = time.perf_counter()
1146     result = solve_problem4_bdf(environment, radius, config, post_boundary)
1147     runtime = time.perf_counter() - started
1148     print(f"DONE BDF {label}: {result.drying_event_time_s / 3600.0:.9f} h ({runtime:.2f} s)", flush=True)
1149     return result, runtime
1150
1151
1152 def timed_be(
1153     label: str,
1154     environment: EnvironmentSeries,
1155     radius: RadiusSeries,
1156     config: BESolverConfig,
1157     post_boundary: PostBoundary | None = None,
1158 ) -> tuple[BESResult, float]:
1159     print(f"START BE {label}", flush=True)
1160     started = time.perf_counter()
1161     result = solve_problem4_be(environment, radius, config, post_boundary)
1162     runtime = time.perf_counter() - started
1163     print(f"DONE BE {label}: {result.drying_event_time_s / 3600.0:.9f} h ({runtime:.2f} s)", flush=True)
1164     return result, runtime
1165
1166
1167 def bdf_summary(result: SimulationResult, runtime_s: float) -> dict:
1168     return {
1169         "node_count": result.diagnostics.internal_node_count,
1170         "event_time_s": result.drying_event_time_s,
1171         "event_time_h": result.drying_event_time_s / 3600.0,
1172         "report_end_time_s": result.report_end_time_s,
1173         "event_radius_cm": result.event_radius_cm,
1174         "event_center_moisture": float(result.event_moisture_internal[0]),
1175         "event_surface_moisture": float(result.event_moisture_internal[-1]),
1176         "event_argmax_xi": float(np.argmax(result.event_moisture_internal) / (len(result.event_moisture_internal)
1177 - 1)),
1178         "runtime_s": runtime_s,
1179         "diagnostics": asdict(result.diagnostics),
1180     }
1181

```

```

1182 def be_summary(result: BEResult, runtime_s: float) -> dict:
1183     return {
1184         "node_count": len(result.event_moisture_internal),
1185         "event_time_s": result.drying_event_time_s,
1186         "event_time_h": result.drying_event_time_s / 3600.0,
1187         "event_radius_cm": result.event_radius_cm,
1188         "event_center_moisture": float(result.event_moisture_internal[0]),
1189         "event_surface_moisture": float(result.event_moisture_internal[-1]),
1190         "event_argmax_xi": float(np.argmax(result.event_moisture_internal) / (len(result.event_moisture_internal)
1191 - 1)),
1192         "runtime_s": runtime_s,
1193         "diagnostics": asdict(result.diagnostics),
1194     }
1195
1196 def common_profile_difference(
1197     left_times: np.ndarray,
1198     left_values: np.ndarray,
1199     right_times: np.ndarray,
1200     right_values: np.ndarray,
1201 ) -> float:
1202     common = sorted(set(left_times.astype(int)).intersection(right_times.astype(int)))
1203     common = [value for value in common if value % 21600 == 0]
1204     if not common:
1205         return float("inf")
1206     maximum = 0.0
1207     for time_s in common:
1208         left = left_values[int(np.where(np.isclose(left_times, time_s))[0][0])]
1209         right = right_values[int(np.where(np.isclose(right_times, time_s))[0][0])]
1210         finite = np.isfinite(left) & np.isfinite(right)
1211         if np.any(finite):
1212             maximum = max(maximum, float(np.max(np.abs(left[finite] - right[finite]))))
1213     return maximum
1214
1215
1216 def compare_bdf(left: SimulationResult, right: SimulationResult) -> dict:
1217     difference_s = abs(left.drying_event_time_s - right.drying_event_time_s)
1218     return {
1219         "event_time_difference_s": difference_s,
1220         "event_time_relative_difference": difference_s / right.drying_event_time_s,
1221         "paper_sample_maximum_moisture_difference": common_profile_difference(
1222             left.time_s, left.moisture, right.time_s, right.moisture
1223         ),
1224         "event_location_same": bool(
1225             np.argmax(left.event_moisture_internal) == 0 and np.argmax(right.event_moisture_internal) == 0
1226         ),
1227     }
1228
1229
1230 def compare_bdf_be(left: SimulationResult, right: BEResult) -> dict:
1231     difference_s = abs(left.drying_event_time_s - right.drying_event_time_s)
1232     return {
1233         "event_time_difference_s": difference_s,
1234         "event_time_relative_difference": difference_s / left.drying_event_time_s,
1235         "paper_sample_maximum_moisture_difference": common_profile_difference(
1236             left.time_s, left.moisture, right.time_s, right.moisture
1237         ),
1238     }
1239
1240
1241 def write_result_xlsx(result: SimulationResult, template: Path, out_path: Path) -> None:
1242     """按附件3模板格式写出全截面水分浓度剖面。
1243
1244     全精度存储、仅用 number_format 控制四位小数显示（与原 result4.xlsx 一致）；
1245     半径收缩后已退出药材内部的固定距离留空（NaN -> 空单元格）。
1246     """
1247     workbook = openpyxl.load_workbook(template)
1248     sheet = workbook.worksheets[0]
1249     fixed = result.fixed_distance_cm
1250     sheet.cell(1, 1).value = "时间\\到药材中心的距离"
1251     for j in range(fixed.size):
1252         sheet.cell(1, 2 + j).value = round(float(fixed[j]), 10)
1253     sheet.cell(1, 2 + fixed.size).value = "药材表面"
1254     for i in range(result.time_s.size):
1255         time_cell = sheet.cell(i + 2, 1)
1256         time_cell.value = int(round(result.time_s[i]))
1257         time_cell.number_format = "0"
1258         values = result.moisture[i]
1259         for j in range(values.size):

```